

Node.JS – CouchDB

Ubuntu Server



Compilation de notes et d'articles de « jihem » , _rev : 2.

```
--[[
  _
o|'--'|o this file is licensed under cc4-by-sa international license.
( 1 ) to view this license, visit https://creativecommons.org/licenses/by-sa/4.0/
  = author: jean-marc "jihem" quere 2011 - jean-marc.quere@codyssea.com
--]]
```

4 Introduction

5 Installation d'Ubuntu Server 12.04 LTS

Installation, Configuration, Pare-Feu, DNS

38 Installation de Node.JS 0.6.18

npm, Express, Nano

47 Installation de CouchDB 1.2.0

50 Introduction à CouchDB

Accès http, Tunnel SSH, Overview, Gestion des documents

74 CouchDB à partir de Node.JS

Gestion des documents

89 SQL vers CouchDB

110 Views

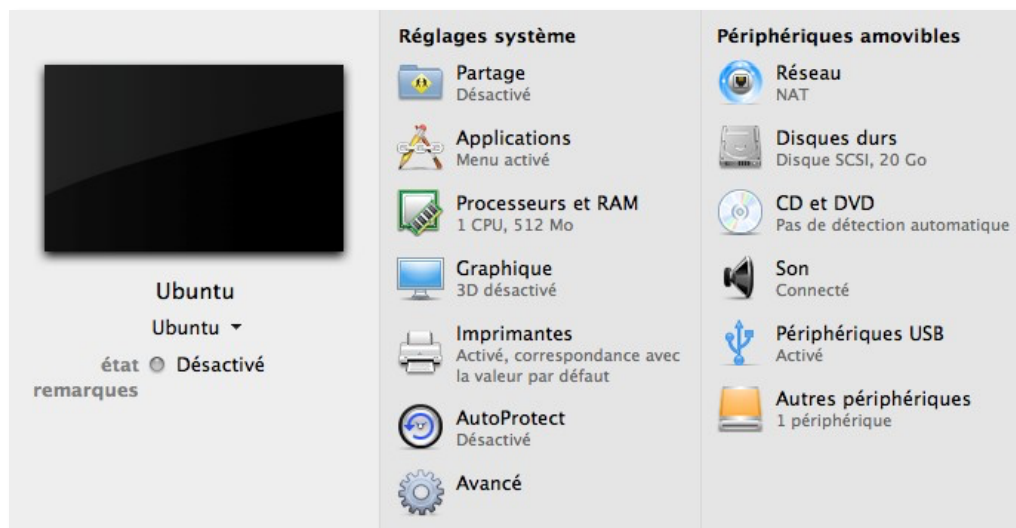
Map, Reduce, Update Handler

Introduction

L'ouvrage présente la mise en œuvre conjointe de Node.JS et CouchDB sous Ubuntu. Les différentes étapes guident l'utilisateur à travers un ensemble d'exemples concrets pour obtenir une plate-forme de production à partir d'un système vierge : ordinateur personnel, serveur ou machine virtuelle. Les activités sont regroupées en trois modules : installation du système d'exploitation et des applications, prise en main de CouchDB (interface et notions connexes) et interaction avec Node.JS. IMPORTANT : des rudiments de programmation en général et de JavaScript en particulier sont requis à la compréhension des concepts décrits. Sans être indispensable, la lecture préalable de **Hands-on Node** (de Pedro Teixeira) est conseillée.

Installation d'Ubuntu Server 12.04 LTS

La version du système d'exploitation retenue est Ubuntu Server 12.04 LTS (Precise Pangolin). Elle est disponible en version 32 et 64 bits sous la forme d'un fichier «.iso» à partir de <http://releases.ubuntu.com/12.04> (Server Install CD). Le choix 32-bit (PC Intel x86) ou 64-bit (PC AMD64) est lié à la nature du processeur de la plateforme matérielle cible. L'image disque «.iso» (ubuntu-12.04-serveri386.iso ou ubuntu-12.04-server-amd64.iso) doit être gravée sur un DVD pour être amorcée. Elle peut également être exploitée en l'état par une machine virtuelle. La mise en œuvre décrite dans cet ouvrage exploite une machine virtuelle VMware Fusion.



Lors de l'activation du système, l'accès au BIOS permet de sélectionner le périphérique de démarrage adéquat (contenant l'image à amorcer).



Lorsqu'il s'agit d'une machine virtuelle, l'accès au BIOS permet également d'inhiber les composants matériels émulés non requis : [MAIN] Legacy Diskette A : **Disabled**, [Advanced-I/O Device Configuration] Serial port A / Serial port B / Parallel port / Floppy Disk controller : **Disabled**, [EXIT] Saving changes.

L'installation d'Ubuntu Server débute. Elle consiste en une succession de choix décrivant la configuration souhaitée et les modalités d'usage des ressources disponibles sur le système hôte : le premier concerne la langue (Français).

Language			
Amharic	Gaeilge	Malayalam	Thai
Arabic	Galego	Marathi	Tagalog
Asturianu	Gujarati	Nepali	Türkçe
Беларуская	ᱚᱠᱟᱨ	Nederlands	Українська
Български	Hindi	Norsk bokmål	Tiếng Việt
Bengali	Hrvatski	Norsk nynorsk	中文(简体)
Bosanski	Magyar	Punjabi (Gurmukhi)	中文(繁體)
Català	Bahasa Indonesia	Polski	
Čeština	Íslenska	Português do Brasil	
Dansk	Italiano	Português	
Deutsch	日本語	Română	
Dzongkha	தமிழ்	Русский	
Ελληνικά	Қазақ	Sámegiellii	
English	Khmer	සිංහල	
Esperanto	ಕನ್ನಡ	Slovenčina	
Español	한국어	Slovenščina	
Eesti	Kurdî	Shqip	
Euskara	Lao	Српски	
فارسی	Lietuviškai	Svenska	
Suomi	Latviski	Tamil	
Français	Македонски	தமிழ்	

F1 Help F2 Language F3 Keymap F4 Modes F5 Accessibility F6 Other Options

Les puristes linuxiens et autres adeptes de l'administration système préféreront une installation en anglais (English) pour disposer d'une configuration ANSI standard. L'adaptation des procédures décrites ne devrait toutefois pas leur poser de problème : seuls les paramètres linguistiques et la langue des messages affichés changent.

La sélection s'effectue avec les touches fléchées du clavier. [Entrée] valide l'option retenue. L'interface utilisateur adopte alors la langue choisie (*en général... sinon l'anglais est de mise*) et propose la liste des actions gérées par l'image disque amorcée. Le premier choix correspond à l'opération désirée : Installer Ubuntu Server. Les vignettes ci-après présentent chaque étape et les actions associées.

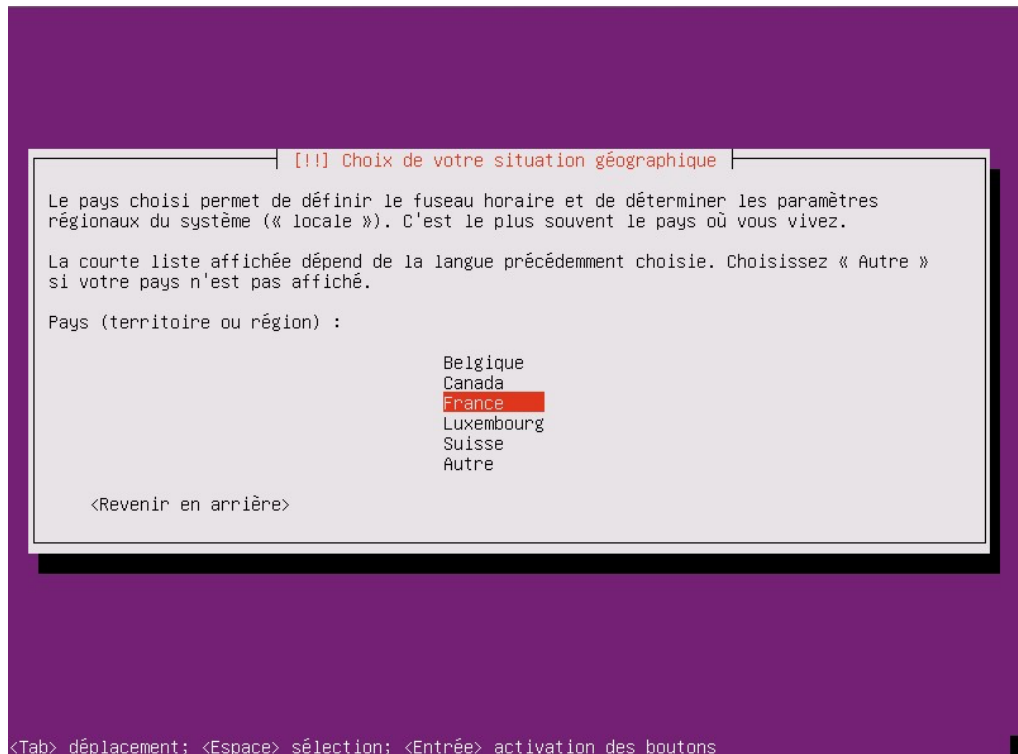


Installer Ubuntu Server

[Entrée]

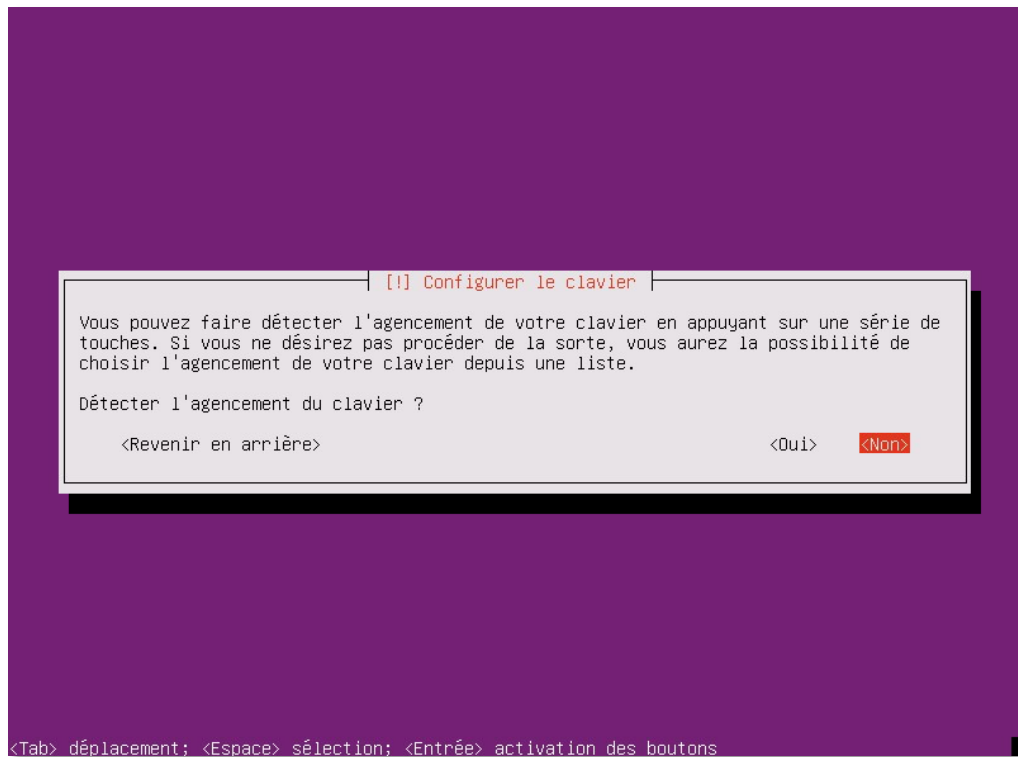


Sur un nouvel ordinateur (neuf ou non encore éprouvé en production), **Tester la mémoire** peut éviter bien des déconvenues : comportements aléatoires, corruptions et pertes de données. Un dysfonctionnement affectant les barrettes mémoire est l'une des causes de pannes les plus difficiles à diagnostiquer : s'assurer de bon fonctionnement de celles-ci constitue une garantie non négligeable.



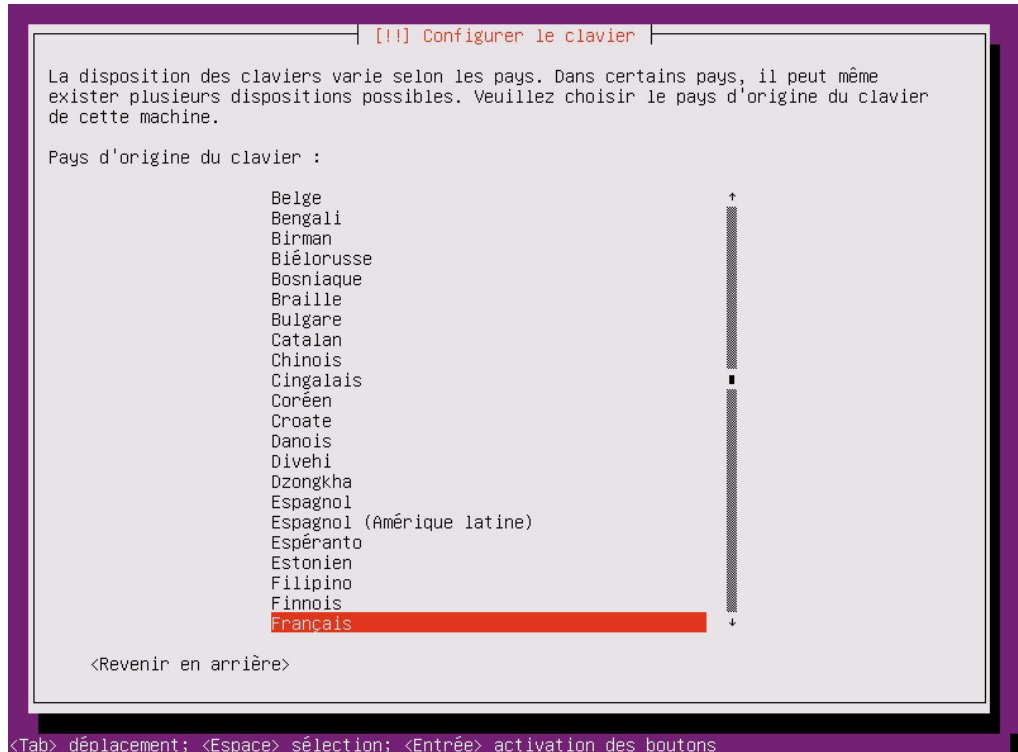
France

[Entrée]



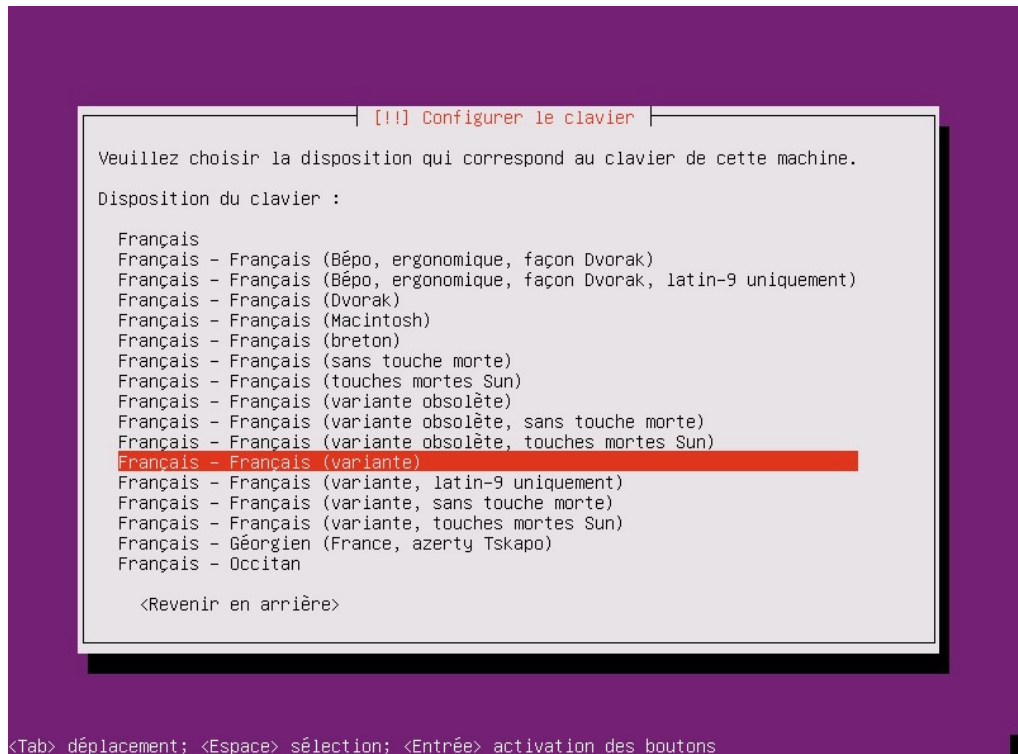
Non

[Entrée]



Français

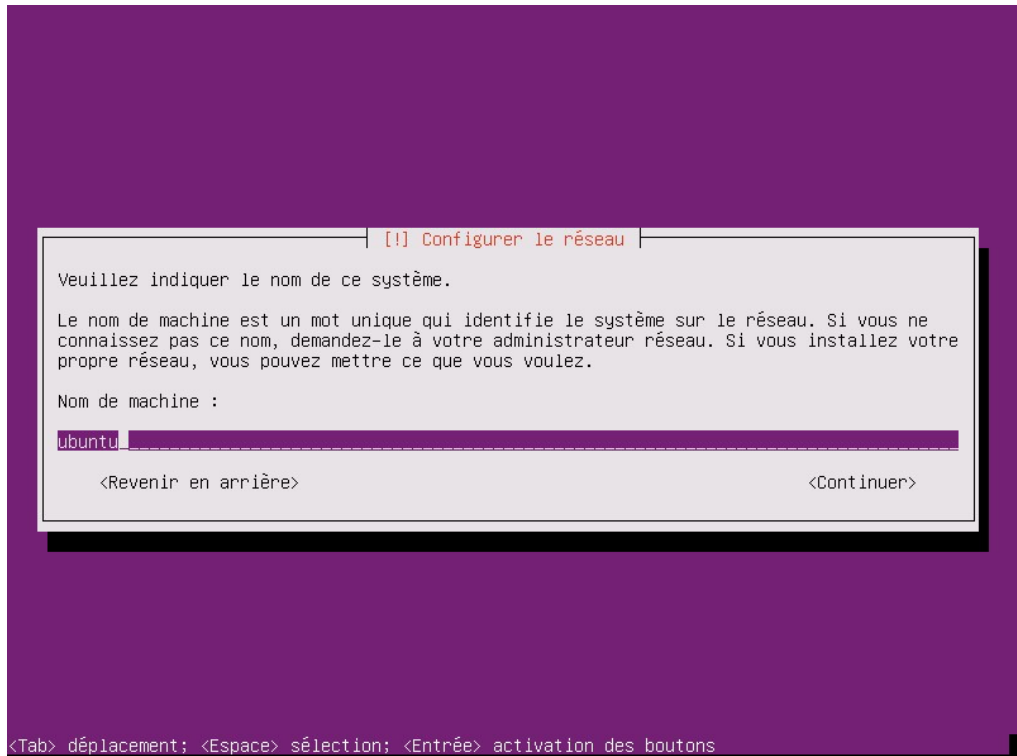
[Entrée]



Français - Français (variante)

[Entrée]

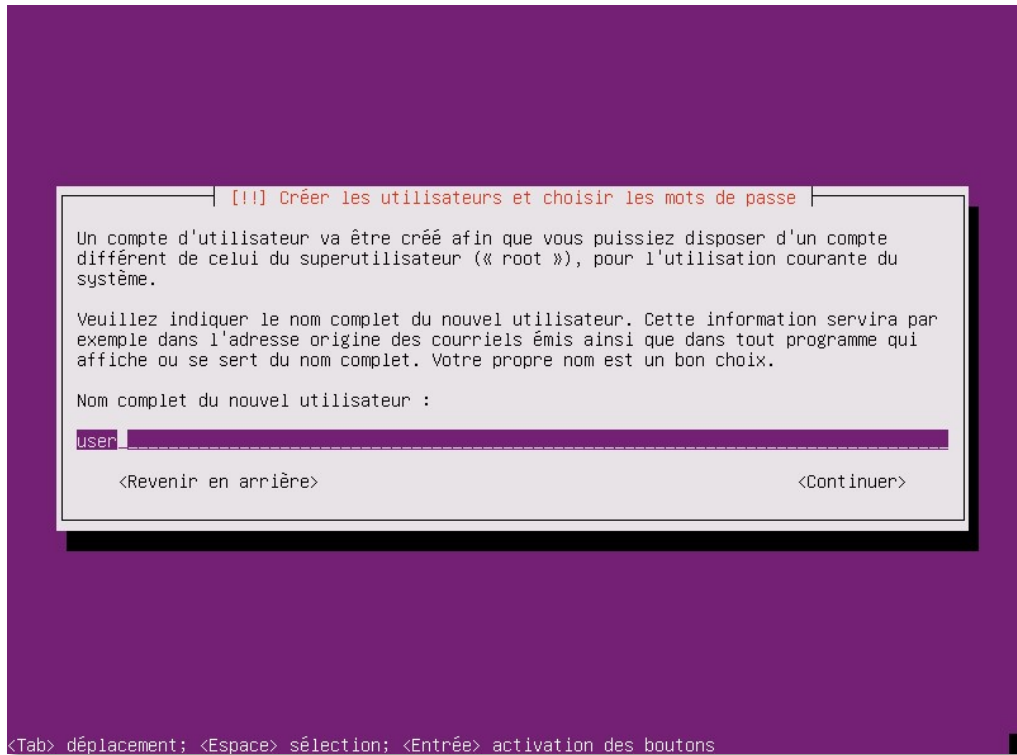
Le système d'exploitation charge en mémoire les modules génériques et procède à la détection et l'activation des équipements détectés. Les cartes Ethernet sont automatiquement dotées des paramètres TCP/IP fournis par le serveur DHCP du réseau (réseau, masque, passerelle et DNS). Le système doit ensuite est «baptisé». Un nom univoque doit lui être attribué afin de l'identifier, ex. ***ubuntu***.



ubuntu

[Entrée]

Linux est un système administré. Les actions des usagers sont gérées : tout le monde ne peut pas faire n'importe quoi. Les notions rattachées aux droits d'accès et d'usage impliquent la déclaration de deux comptes utilisateur : un usager de base (vous) et un super-administrateur (aux pouvoirs illimités = responsabilités).



[[!]] Créer les utilisateurs et choisir les mots de passe

Un compte d'utilisateur va être créé afin que vous puissiez disposer d'un compte différent de celui du superutilisateur (« root »), pour l'utilisation courante du système.

Veuillez indiquer le nom complet du nouvel utilisateur. Cette information servira par exemple dans l'adresse origine des courriels émis ainsi que dans tout programme qui affiche ou se sert du nom complet. Votre propre nom est un bon choix.

Nom complet du nouvel utilisateur :

user

<Revenir en arrière

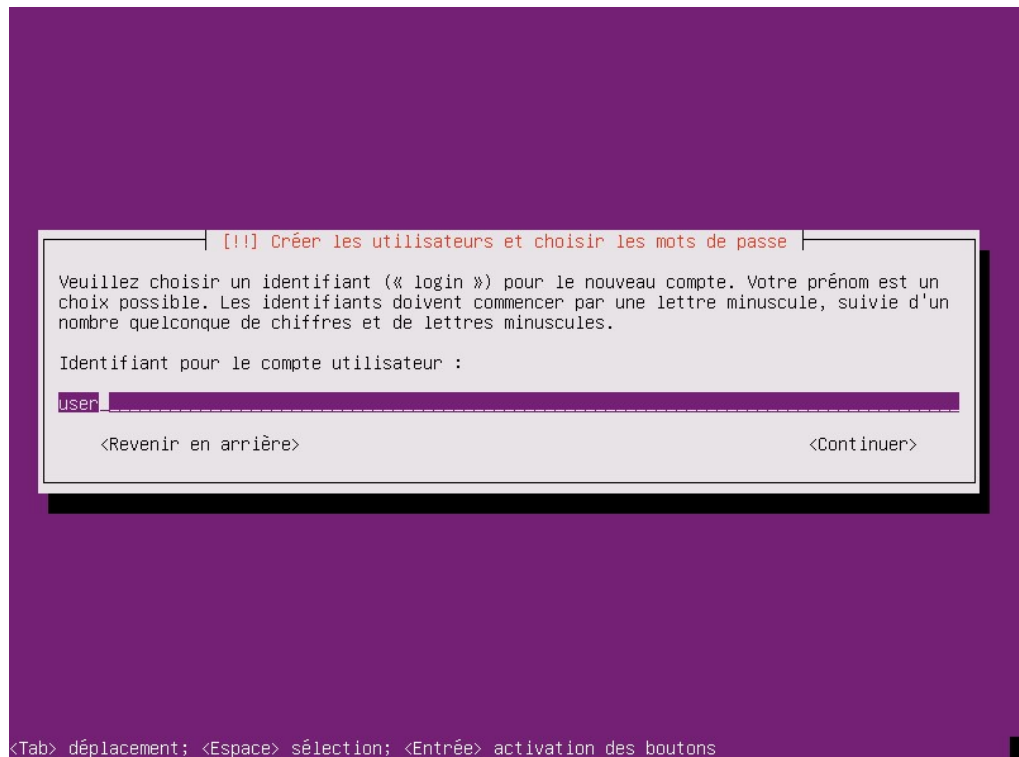
<Continuer>

<Tab> déplacement; <Espace> sélection; <Entrée> activation des boutons



user

[Entrée]



[[!]] Créer les utilisateurs et choisir les mots de passe

Veillez choisir un identifiant (« login ») pour le nouveau compte. Votre prénom est un choix possible. Les identifiants doivent commencer par une lettre minuscule, suivie d'un nombre quelconque de chiffres et de lettres minuscules.

Identifiant pour le compte utilisateur :

user

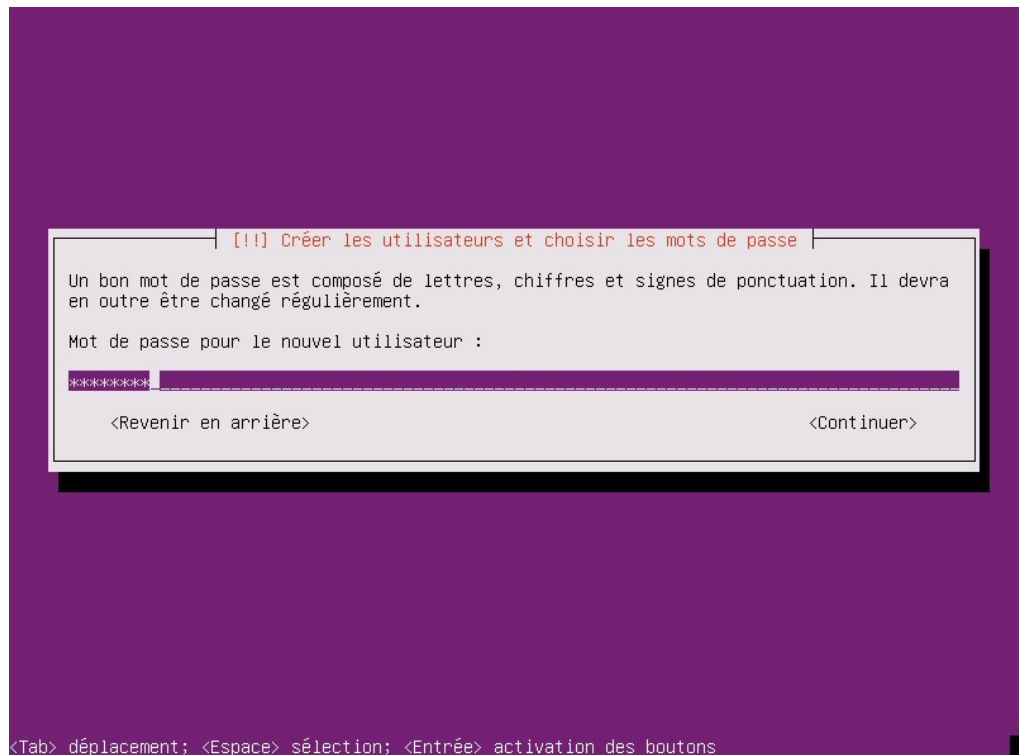
<Revenir en arrière> <Continuer>

<Tab> déplacement; <Espace> sélection; <Entrée> activation des boutons



user

[Entrée]



The screenshot shows a web application interface with a purple background. A central white box contains the following text:

[!!] Créer les utilisateurs et choisir les mots de passe

Un bon mot de passe est composé de lettres, chiffres et signes de ponctuation. Il devra en outre être changé régulièrement.

Mot de passe pour le nouvel utilisateur :

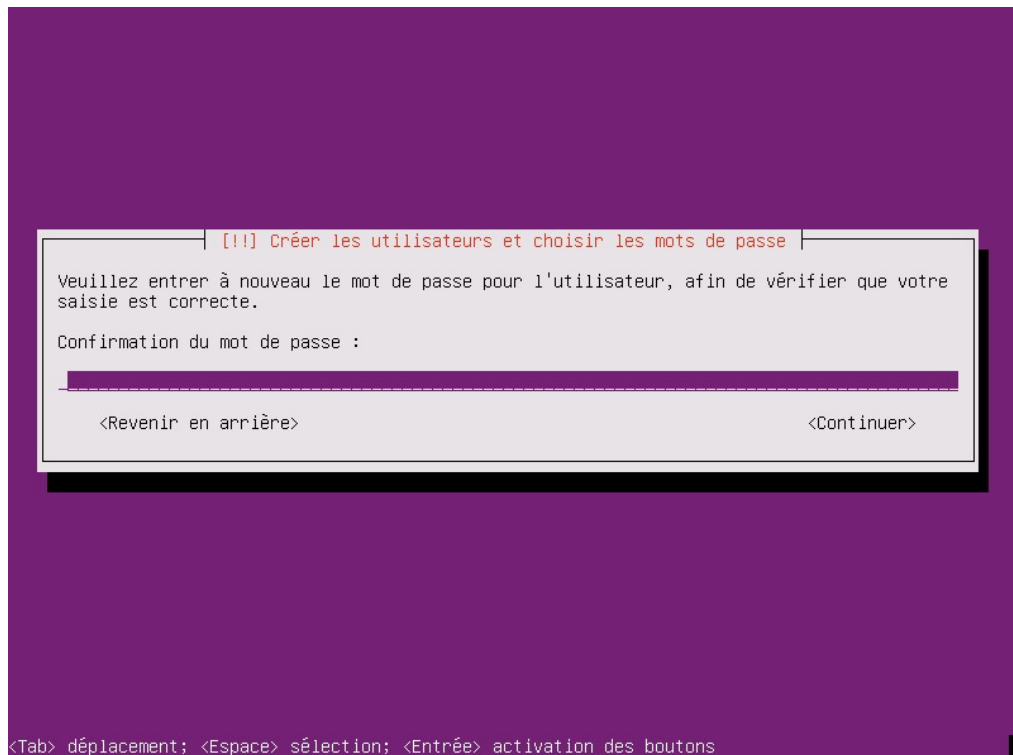
<Revenir en arrière> <Continuer>

At the bottom of the screen, a small text box contains the instruction: <Tab> déplacement; <Espace> sélection; <Entrée> activation des boutons



pass

[Entrée]



[[!]] Créer les utilisateurs et choisir les mots de passe

Veillez entrer à nouveau le mot de passe pour l'utilisateur, afin de vérifier que votre saisie est correcte.

Confirmation du mot de passe :

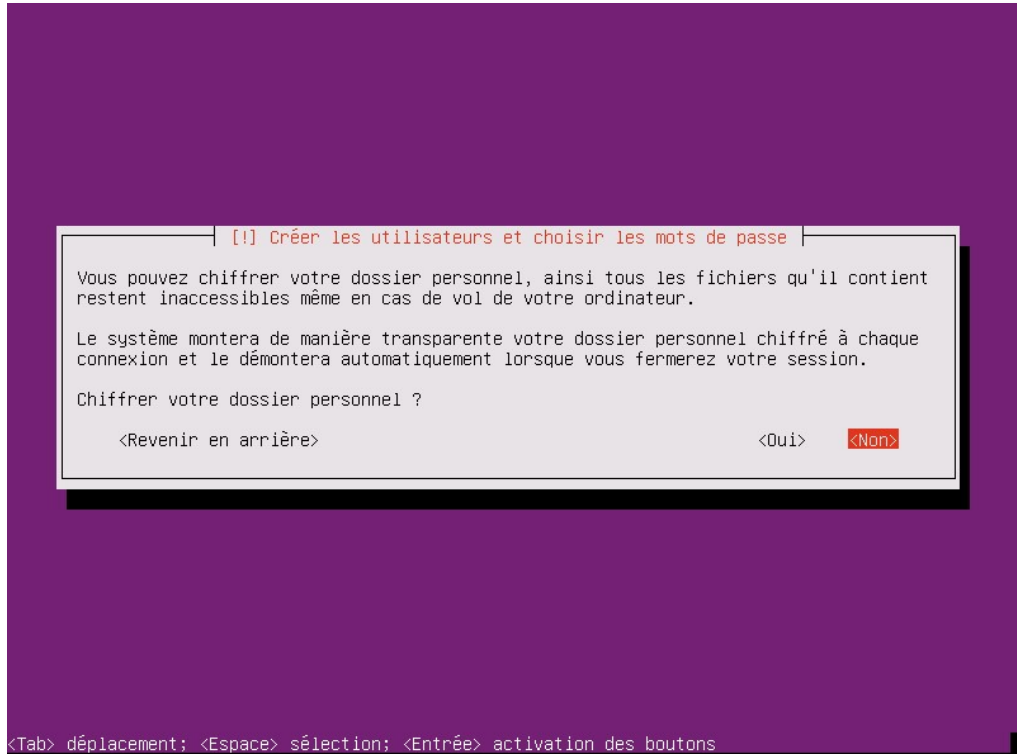
<Revenir en arrière> <Continuer>

<Tab> déplacement; <Espace> sélection; <Entrée> activation des boutons



pass

[Entrée]

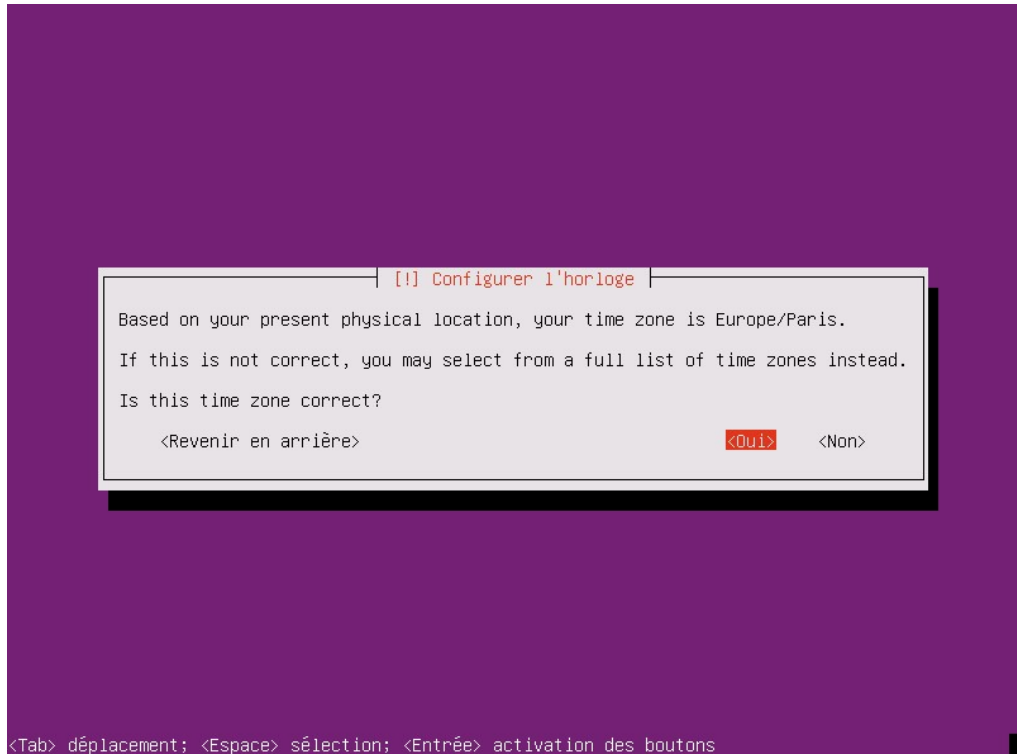


Non

[Entrée]



Le choix du couple (**user**, **pass**) est effectué par soucis de simplicité. Vous devez IMPERATIVEMENT définir vos propres identifiants et mots de passe. Le mot de passe indiqué est valable pour l'usager **user** et le superadministrateur **root**. En pratique, il est nécessaire de passer par un compte intermédiaire pour devenir super-administrateur. L'objectif est d'identifier l'usager à l'origine de l'installation selon le contexte.

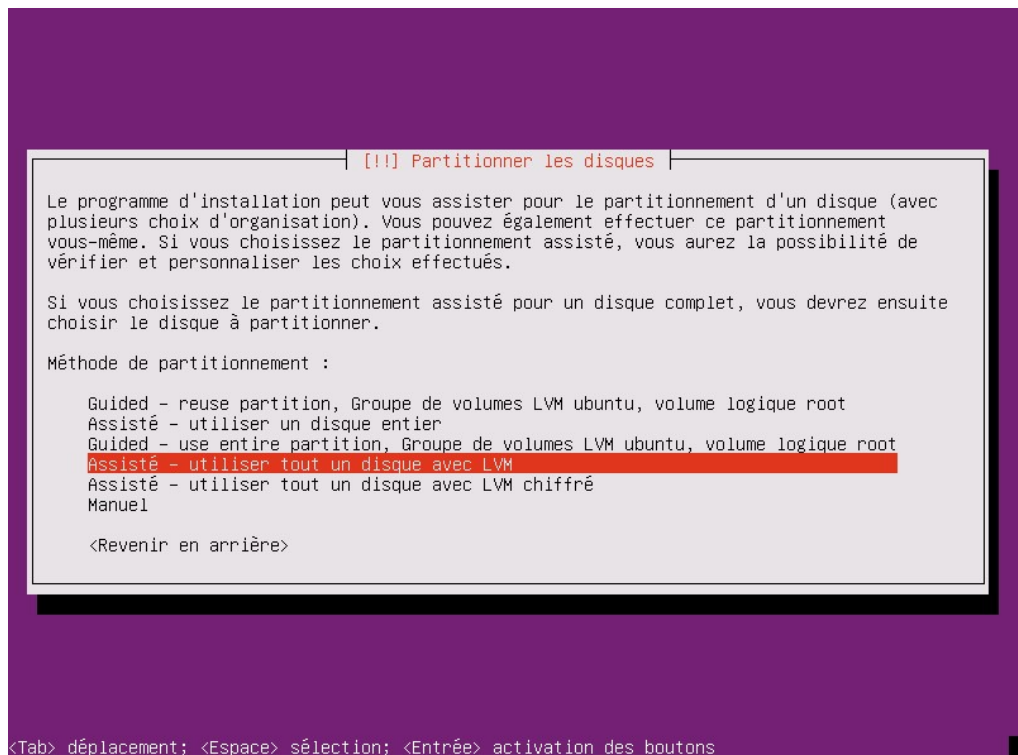


Europe/Paris

[Entrée]

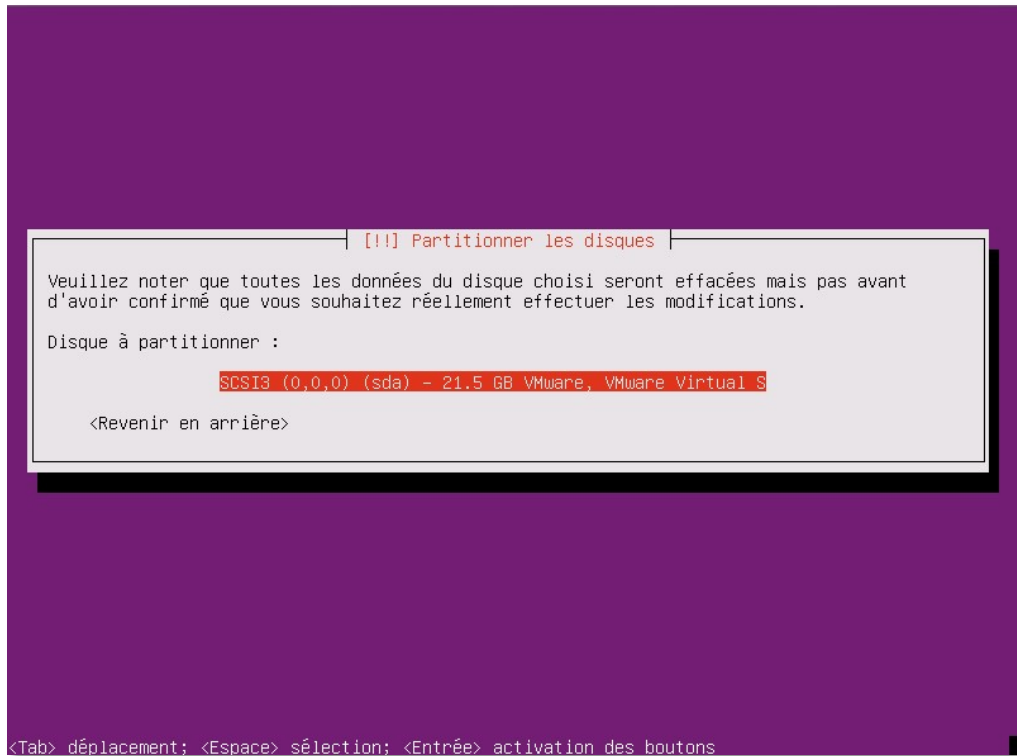
Le fuseau horaire proposé par défaut correspond à celui de la zone géographique où l'installation est effectuée : France Métropolitaine GMT+1 Paris/Berlin (avec gestion de l'heure d'été).

ATTENTION : La mise en œuvre proposée utilise l'intégralité du disque dur. Les données présentes sont EFFACEES DE FAÇON IRREMEDIABLE. Les partitions et unités disques proposés correspondent à la machine virtuelle précédemment décrite (disque dur virtuel de 20Go). Les désignations peuvent selon les caractéristiques physiques ou virtuelles de votre propre système. La démarche reste toutefois similaire.

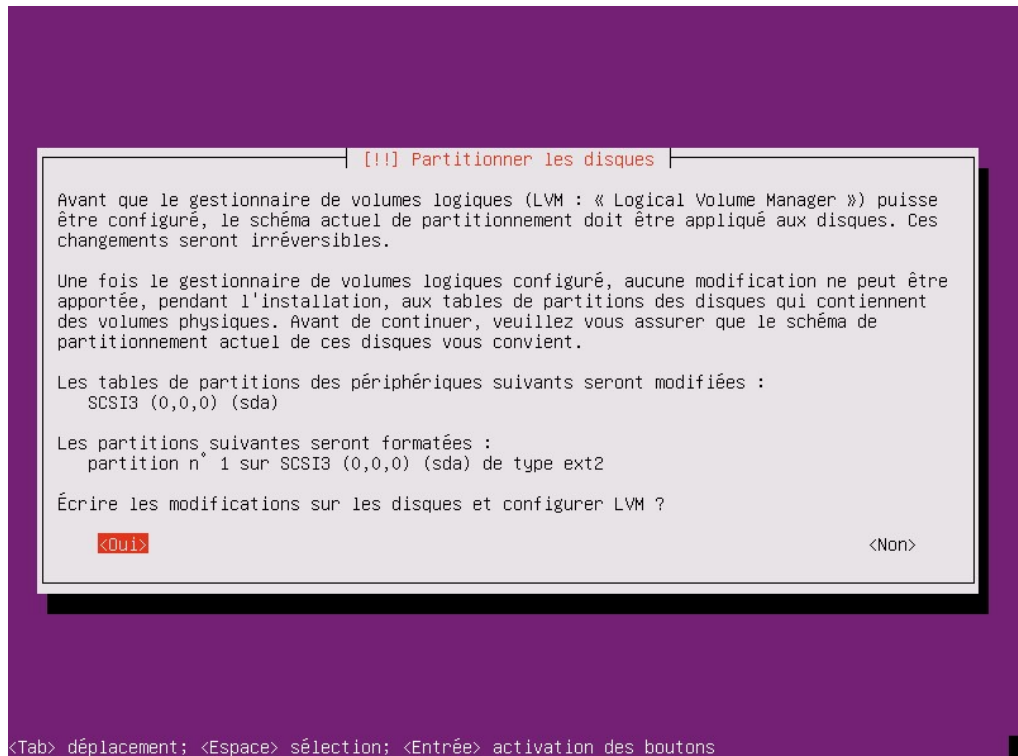


Assisté - utiliser tout un disque avec LVM

[Entrée]

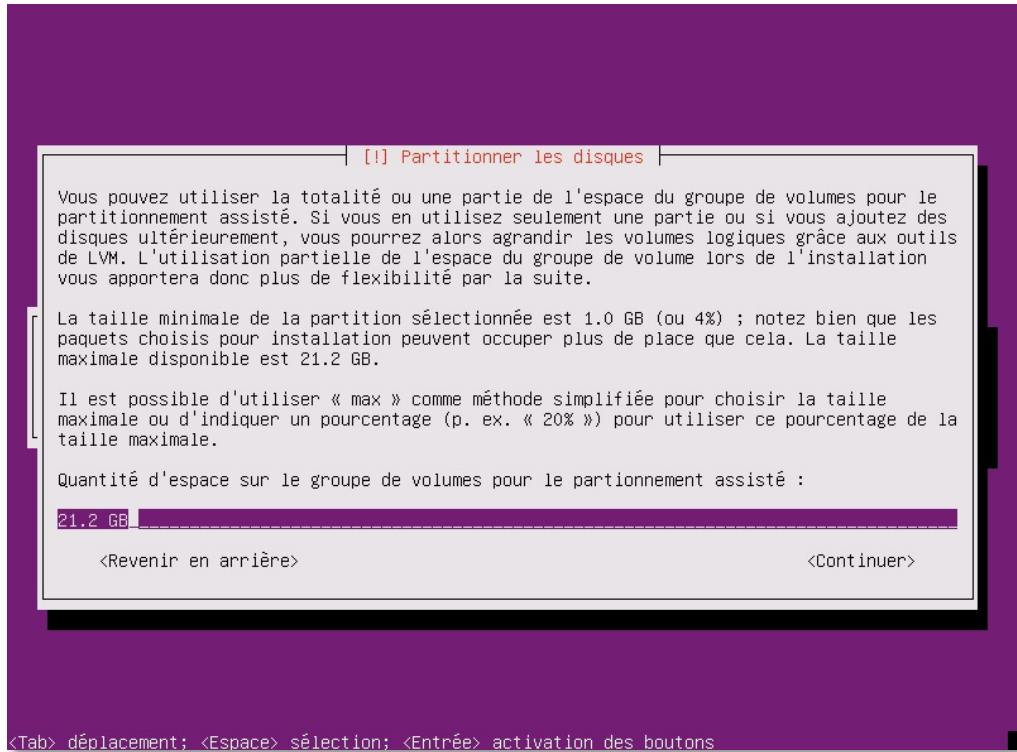


SCSI3 (0,0,0) (sda) – 21.5 GB, VMware, VMware Virtual S
[Entrée]

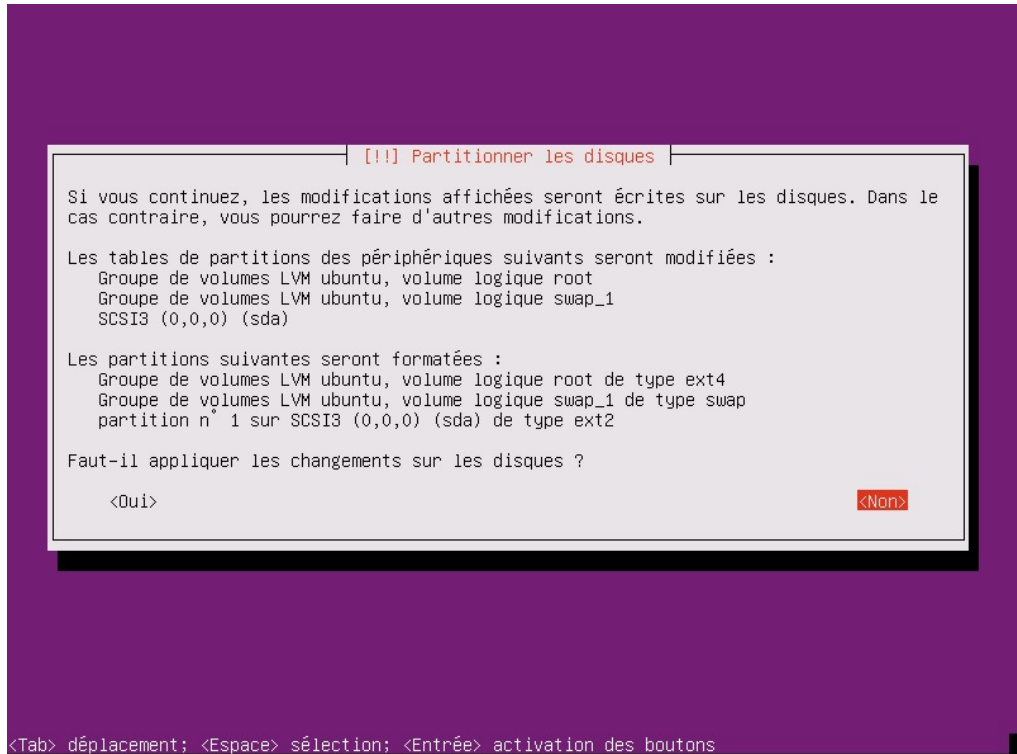


Oui

[Entrée]



Conservez la valeur proposée : **21.2 GB** dans le cas de l'exemple.
[Entrée]



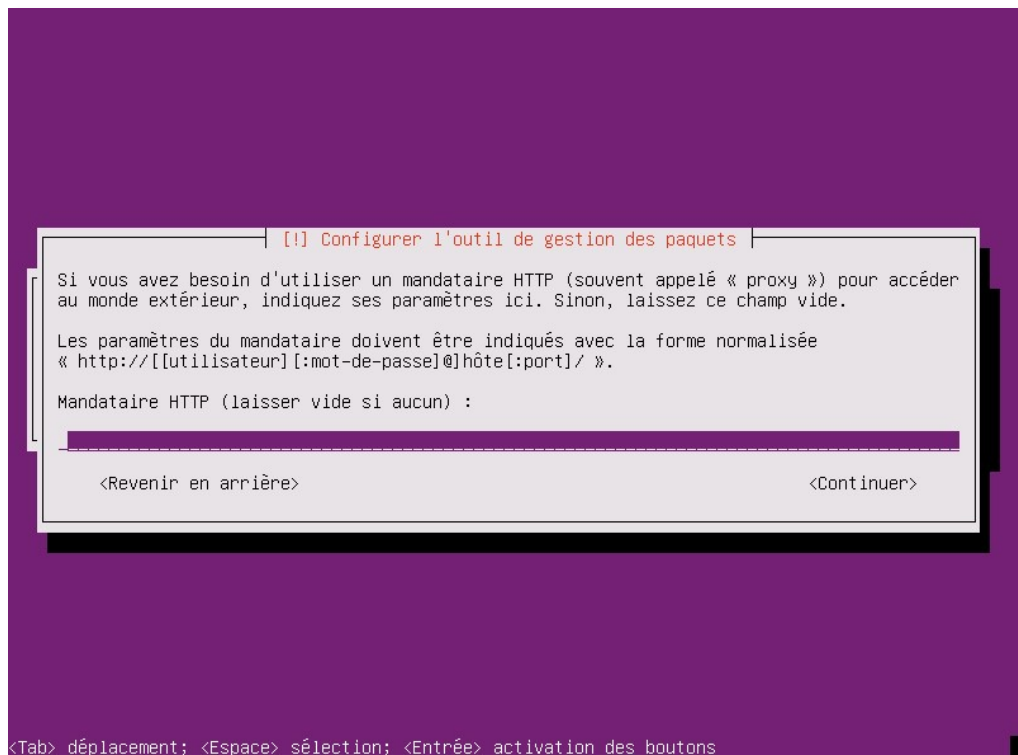
Dernière possibilité de faire marche arrière et/ou ultime confirmation...

Oui

[Entrée]

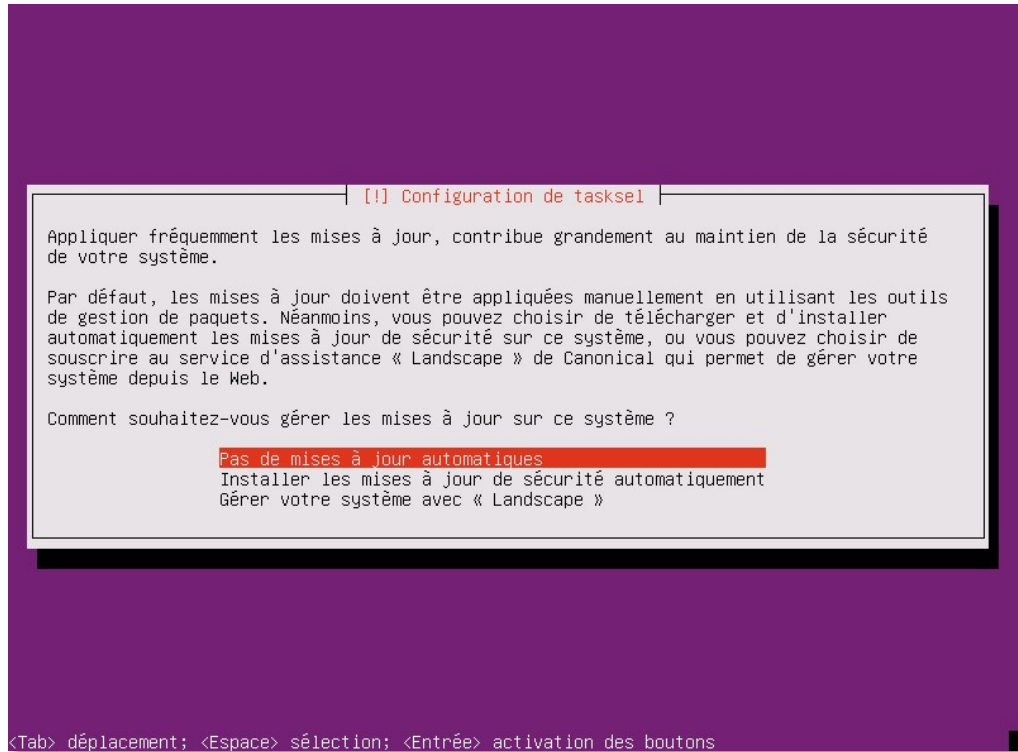
Le reconditionnement du système hôte (partitionnement, formatage des unités disques) requiert quelques instants (selon la taille des disques et ses performances).

L'installation peut recourir à des serveurs de dépôts (recueils de mises à jours et logiciels connexes) pour l'installation. Certains réseaux (essentiellement d'entreprises) nécessitent l'emploi de dispositif sécurisant (limitant) les accès à Internet : mandataire HTTP (proxy). En dehors de ce contexte, cette information ne doit pas être renseignée. Le raccordement du système à un réseau TCP/IP offrant une passerelle vers Internet est vivement recommandé.



[Entrée]

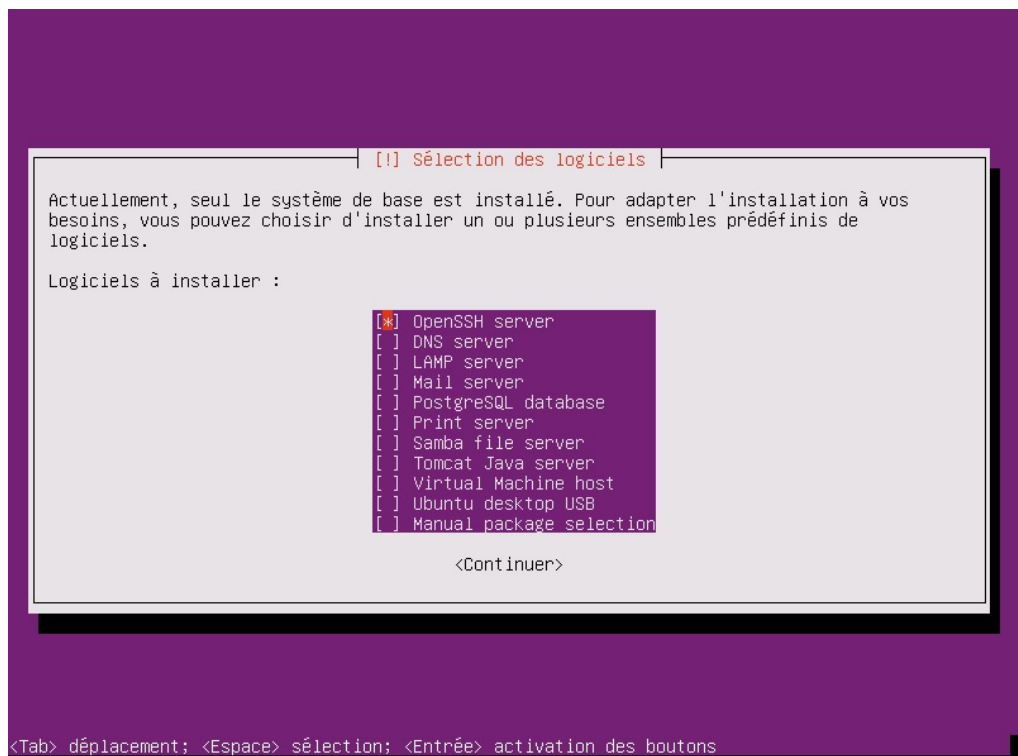
La configuration de l'outil de gestion des paquets est complétée d'après les informations collectées sur les serveurs d'Internet dédiés à la diffusion des applications et des mises à jour d'Ubuntu. La société éditrice d'Ubuntu (<http://www.canonical.com>) propose différents types de gestion des mises à jour : manuelle, automatisée (avec ou sans valeur ajoutée). La destination du système, au terme de son installation, influe sur ce choix qui contribue au maintien de la sécurité. En l'état (découverte et d'expérimentation), une mise à jour manuelle est préférée.



Pas de mise à jour automatique

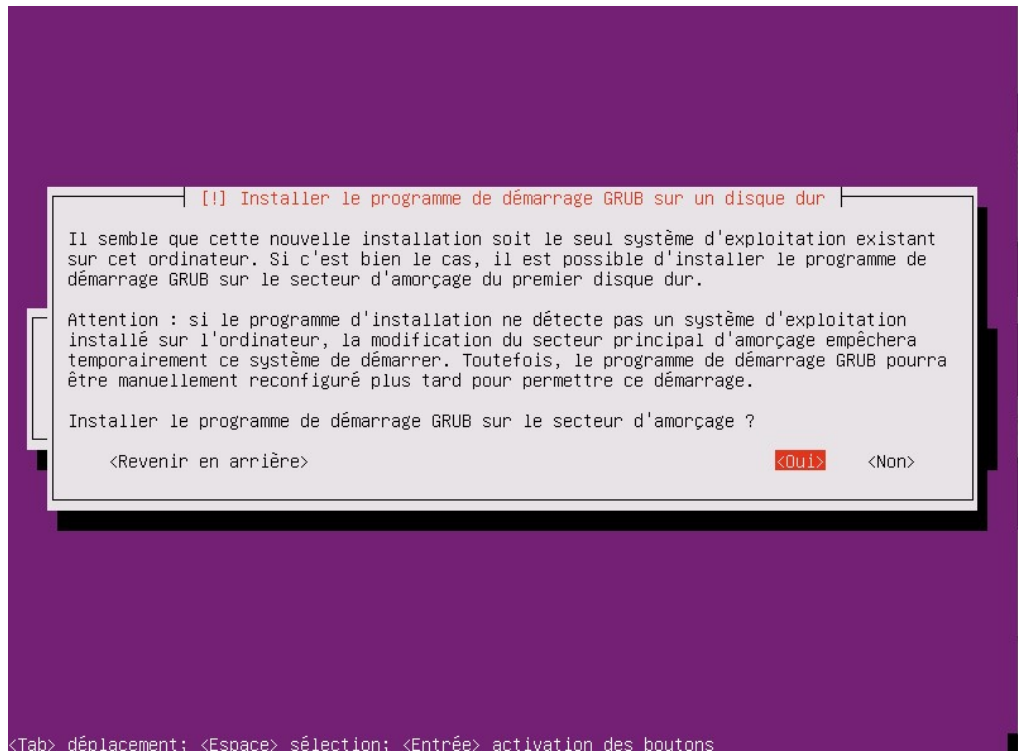
[Entrée]

En complément du système de base, une sélection de logiciels propose des solutions prêtes à l'emploi aux besoins les plus courants (en entreprise) : serveur DNS, serveur Web, serveur de Mail, serveur de bases de données (MySQL, PostgreSQL), serveur d'impression, serveur d'applications Java, serveur de virtualisation, etc. **OpenSSH** (accès distant via le réseau local ou Internet) est INDISPENSABLE.



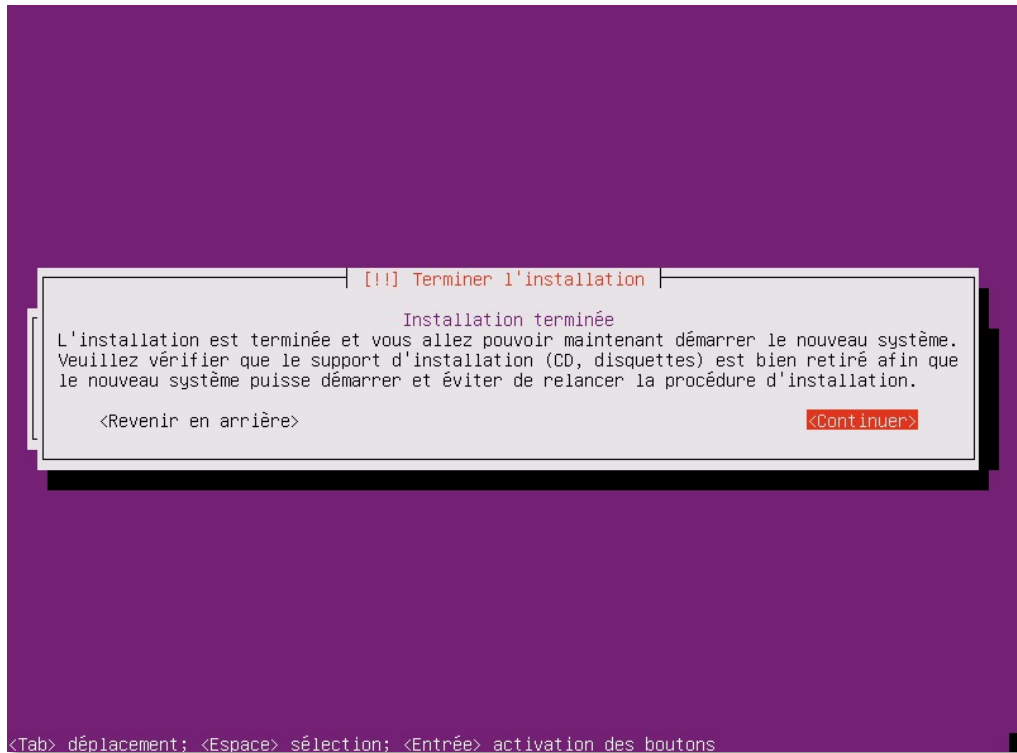
OpenSSH server [Espace]
[Entrée]

Le processus d'installation copie sur le disque dur tous les éléments nécessaires à la bonne marche du système. Le cas échéant des modules et logiciels connexes sont téléchargés. La durée de cette étape varie de quelques secondes à plusieurs minutes. Les dernières étapes de l'installation consistent à installer un logiciel d'amorçage du système sur le disque dur et... à redémarrer.



Oui

[Entrée]

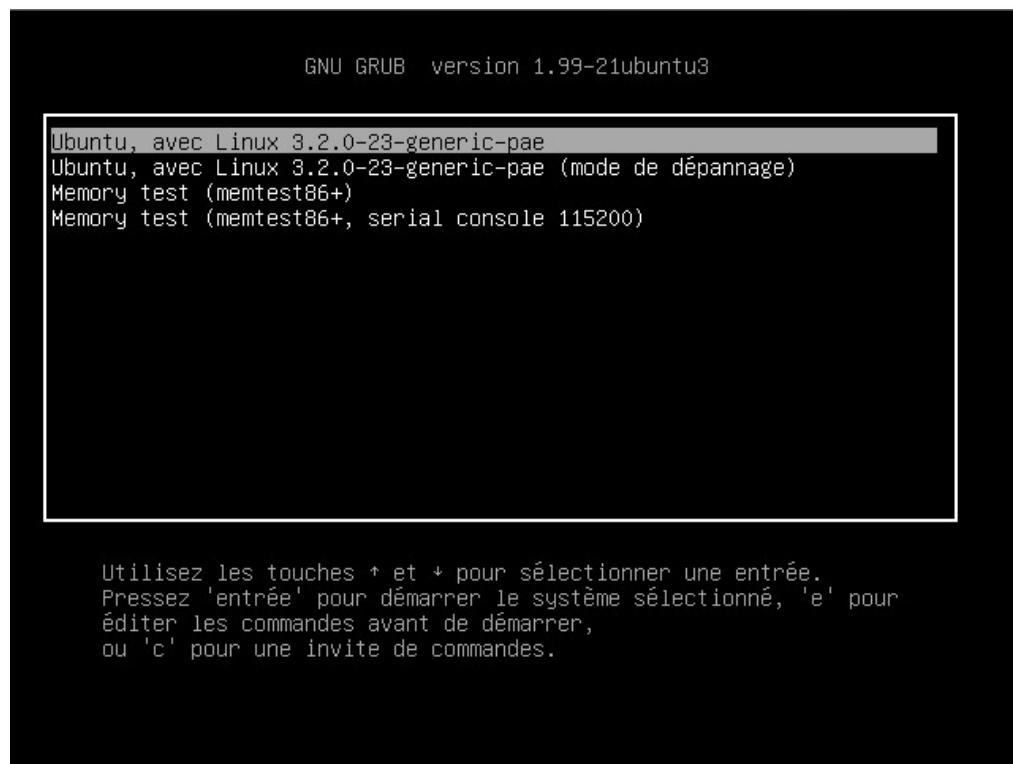


Continuer

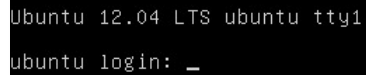
[Entrée]



Pensez à retirer le DVD... ou lors de la séquence de redémarrage, accédez au BIOS pour redéfinir l'ordre de prise en compte des périphériques d'amorçage : placer le disque dur en premier.



L'écran (Grup) de sélection du mode d'amorçage s'affiche... et...

A terminal window with a black background. The text 'Ubuntu 12.04 LTS ubuntu tty1' is displayed on the first line, and 'ubuntu login: _' is on the second line.

```
Ubuntu 12.04 LTS ubuntu tty1
ubuntu login: _
```

C'est terminé.

Enfin, l'installation est terminée. Maintenant, il faut configurer...

Configuration

La version installée d'Ubuntu (à partir du support) doit être mise à jour afin d'intégrer les derniers correctifs disponibles sur Internet. Suite au redémarrage, le message **login** invite l'utilisateur à s'identifier. L'administration requiert l'ensemble des droits sur le système. Toutefois, il n'est pas possible de se connecter directement en **root**. Il est nécessaire de passer par un compte utilisateur intermédiaire (afin d'identifier nominativement dans les journaux système qui requièrent ces droits). Le compte utilisateur **user** est donc utilisé à cet effet (associé avec le mot de passe **pass**).

```
Ubuntu 12.04 LTS ubuntu tty1
ubuntu login: user
Password:
Last login: Wed May 30 10:53:40 CEST 2012 on tty1
Welcome to Ubuntu 12.04 LTS (GNU/Linux 3.2.0-23-generic-pae i686)

 * Documentation:  https://help.ubuntu.com/

System information as of Wed May 30 10:54:11 CEST 2012

System load:  0.05          Processes:            63
Usage of /:   5.4% of 19.18GB Users logged in:          0
Memory usage: 6%           IP address for eth0: 192.168.99.175
Swap usage:   0%

Graph this data and manage this system at https://landscape.canonical.com/

26 packages can be updated.
9 updates are security updates.

user@ubuntu:~$ _
```



Login: user [Entrée]

Password: pass [Entrée]

En cas de succès, l'invite de commande \$ (spécifique aux usagers) s'affiche. La commande **sudo su** – permet devenir administrateur (après une nouvelle saisie du mot de passe).



```
user@ubuntu:~$ sudo su – [Entrée]
```

```
password for user: pass [Entrée]
```

```
root@ubuntu:~#
```

L'invite de commande passe du \$ au # confirmant l'obtention des droits d'administration. Le préfixe de l'invite de commande comporte des indications relatives au compte utilisateur utilisé (user ou root) au nom du système (ubuntu), à l'emplacement actuel (~ = dossier personnel).



~ ne correspond pas au même emplacement selon le compte utilisateur (puisque'il est propre à ce dernier). Il s'agit respectivement de **/home/user** et **/root** pour les comptes **user** et **root**. La commande **pwd** indique l'emplacement actuel (**ls** en liste le contenu). **man** donne accès à la documentation en ligne (et en anglais) d'une commande, ex. : **man ls** [Entrée].

ATTENTION : à partir de maintenant \$ et # identifient le contexte d'emploi des commandes : **user** et **root**. La touche de validation [Entrée] en fin de ligne de commande n'est plus indiquée. En l'absence de remarque spécifique, il faut confirmer les messages intermédiaires, ex. **Continuer (O/N)**: O [Entrée].

La commande relative à la gestion des paquets logiciels (à installer, désinstaller, mettre à jour) est **apt-get**. L'option **update** met à jour la liste des serveurs sources (à partir desquels des données peuvent être téléchargées). L'option **upgrade** actualise l'ensemble des paquets déployés au dernier niveau de version disponible.



```
#apt-get update  
#apt-get upgrade
```

Les outils de développement **build-essential** sont disponibles, comme l'ensemble des autres applications, sous la forme de paquets. La compilation de **node.js** à partir de ses sources requiert l'installation de l'ensemble des paquets listés ciaprès.



```
#apt-get install build-essential  
#apt-get install libcurl4-openssl-dev
```

Les utilisateurs de **node.js** sont - à raison - très attachés à la plate-forme collaborative de gestion de projets **github**. L'ajout du paquet **git-core** permet de disposer de l'ensemble des commandes requises à la publication et à la synchronisation de sources.



```
#apt-get install git-core
```

Pare-feu

Le pare-feu protège le système hôte en limitant les flux à destination ou en provenance du monde extérieur (réseau local, Internet). La configuration proposée ouvre les ports TCP 22 (ssh) et TCP 80 (web). Le système peut ainsi être contrôlé à distance (via ssh) et les services Web sont accessibles. Les événements liés au pare-feu sont consignés dans un journal. L'option **status numbered** liste les règles établies à l'écran. Le numéro associé à chaque règle peut être utilisé pour la supprimer (**sudo ufw delete numéro**). Pour créer une règle d'interdiction, il suffit de remplacer **enable** par **deny**. Initialement tous les flux émanent du système sont autorisés et tous ceux qui lui sont destinés (à partir du monde extérieur) sont interdits (*je peux sortir mais personne n'entre sans mon accord*).



```
# ufw allow ssh/tcp
```

Les règles ont été mises à jour

Les règles ont été mises à jour (IPv6)

```
# ufw allow 80/tcp
```

Les règles ont été mises à jour

Les règles ont été mises à jour (IPv6)

```
# ufw logging on
```

Journalisation activée

```
# ufw enable
```

Le pare-feu est actif et lancé au démarrage du système

```
# ufw status numbered
```

Etat : actif

... liste des règles ...

DNS

La configuration DNS ci-après illustre l'emploi d'un joker (*) qui redirige tous les hôtes vers la même adresse. Ce réglage est très pratique car il permet de prendre en charge n'importe quel serveur virtuel (**www.mondomaine.fr**, **demo.mondomaine.fr**, **monsite.mondomaine.fr**) sans aucune déclaration connexe (coté DNS).



.mondomaine.fr NS dns20.registrar.net

.mondomaine.fr NS ns20.registrar.net

.mondomaine.fr A 98.76.54.32

*.mondomaine.fr CNAME mondomaine.fr



La découverte et l'expérimentation de la plateforme Node.JS – CouchDB sous Ubuntu Server ne requiert pas de déclaration DNS. En production (ensemble accessible sur Internet), l'affectation d'un nom d'hôte s'avère nécessaire : les internautes rechignent à saisir des adresses IP dans leur barre d'adresse. L'astuce proposée permet d'affecter non pas un nom d'hôte mais un domaine entier au serveur.

Le nom de domaine (mondomaine.fr), l'adresse IP (98.76.54.32) et les serveurs de nom du registrar (dns20.registrar.net, ns20.registrat.net) sont à adapter aux conditions réelles d'usage.

Installation de Node.JS 0.6.18

(incluant npm 1.1.21)

L'interpréteur **node.js** est indissociable de l'utilitaire de gestion de paquets **npm** qui permet de compléter l'environnement de développement à l'aide de bibliothèque de fonctions (écrites par d'autres utilisateurs). L'ajout, la mise à jour ou le retrait des bibliothèques et de leurs dépendances sont intégralement gérés par ce dernier afin d'assurer la cohérence de l'ensemble. L'équipe de développement de **node.js** est active et prolixe. Les mises à jour se succèdent rapidement. Les systèmes d'exploitation sont souvent distribués avec des versions obsolètes. Le site officiel <http://nodejs.org> donne accès aux dernières versions stables pour Windows et MacOS (incluant **npm**). Les utilisateurs d'autres systèmes d'exploitation (Linux) doivent compiler les sources. La préparation et l'installation s'effectuent en ligne de commande (sous le compte **user**, invite **\$**). La commande **wget** transfère l'archive contenant les sources du site officiel vers le système hôte. Le chemin et le nom de l'archive (à adapter) dépendent de la version disponible. Dans le cadre de la réalisation de cet ouvrage, la dernière version stable est 0.6.18. L'archive téléchargée peut ensuite être décompressée (**tar**).

```
System information as of Wed May 30 12:37:34 CEST 2012

System load: 0.16          Processes:           70
Usage of /: 6.4% of 19.18GB Users logged in:      0
Memory usage: 5%          IP address for eth0: 192.168.99.175
Swap usage: 0%

Graph this data and manage this system at https://landscape.canonical.com/

user@ubuntu:~$ wget http://nodejs.org/dist/v0.6.18/node-v0.6.18.tar.gz
--2012-05-30 12:37:38-- http://nodejs.org/dist/v0.6.18/node-v0.6.18.tar.gz
Résolution de nodejs.org (nodejs.org)... 8.12.44.238
Connexion vers nodejs.org (nodejs.org)[8.12.44.238]:80... connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
Longueur: 10261465 (9,8M) [application/octet-stream]
Sauvegarde en : «node-v0.6.18.tar.gz»

100%[=====] 10 261 465 315K/s ds 33s

2012-05-30 12:38:12 (304 KB/s) - «node-v0.6.18.tar.gz» sauvegardé [10261465/10261465]

user@ubuntu:~$ tar xzf node-v0.6.18.tar.gz
user@ubuntu:~$ _
```



```
$cd ~
```

```
$wget http://nodejs.org/dist/v0.6.15/node-v0.6.15.tar.gz
```

```
$tar xzf node-v0.6.15.tar.gz
```

Le dossier cible d'extraction **node-v0.6.18** est automatiquement créé. Les commandes utiles à la compilation (et les sources) se trouvent à l'intérieur de ce dernier. La commande **configure** identifie le contexte (propre au système utilisé) et vérifie la présence de l'ensemble des composantes requis : chaîne de développement et bibliothèques connexes.

```
user@ubuntu:~$ cd node-v0.6.18/
user@ubuntu:~/node-v0.6.18$ ./configure
Checking for program g++ or c++      : /usr/bin/g++
Checking for program cpp             : /usr/bin/cpp
Checking for program ar              : /usr/bin/ar
Checking for program ranlib          : /usr/bin/ranlib
Checking for g++                    : ok
Checking for program gcc or cc       : /usr/bin/gcc
Checking for program ar              : /usr/bin/ar
Checking for program ranlib          : /usr/bin/ranlib
Checking for gcc                    : ok
Checking for library dl              : yes
Checking for openssl                : yes
Checking for library util            : yes
Checking for library rt              : yes
Checking for fdatsync(2) with c++    : yes
'configure' finished successfully (0.171s)
user@ubuntu:~/node-v0.6.18$ _
```



```
$cd node-v0.6.18
$./configure
```

Le compte rendu ci-après décrit un environnement favorable. En cas d'anomalie, le composant incriminé doit être installé ou mis à jour. Le système, résultant de l'installation d'Ubuntu Server décrite, comporte les éléments requis : buildessential, libcurl4-openssl-dev.

La commande **make** produit - après quelques instants - les programmes sous leur forme exécutable (**node**, **node-waf**).



```
$make
```

La mise à disposition de ces programmes à l'ensemble des usagers nécessite leur déplacement (ou copie) au sein du dossier usuel des commandes utilisateur. L'opération requiert des droits élevés (d'où le passage en **root**, invite **#**). Le passage d'un utilisateur à un autre s'effectue à l'aide de **su - nom de l'utilisateur** (en l'occurrence **su – root** équivalent à **su -**). L'installateur est inclus dans le script géré par make (Makefile).

Version longue (avec **su**)



```
$su -  
#make install  
#exit  
$
```

Version courte (avec **sudo**)



```
$sudo make install
```



La version courte (plus rapide) doit être préférée. L'intérêt de la présentation des deux versions est d'explicitier l'action de **sudo**. Certaine distribution Linux et quelques administrateurs rendent la commande **su** inaccessible aux usagers. Ils préfèrent n'autoriser que la commande **sudo** pour quelques comptes (listés dans **/etc/sudoers**). Le **su** n'est alors possible que par l'intermédiaire de : **sudo su -**

npm 1.1.22

Le gestionnaire de packages de node.js est maintenant inclus (et distribué) avec l'installation de **node.js**. L'exécution de la procédure décrite ci-après n'est plus requise. Vous pouvez passer à l'installation des packages, voir **Express**, **Nano...** sauf si vous souhaitez installer la toute dernière version de **npm**. Les codes source de **npm** sont hébergés sur la plate-forme collaborative de gestion de projet logiciel **github**. La commande **git** clone l'arborescence distante du projet en local dans le répertoire **npm**. La démarche pour compiler l'application est similaire à celle déjà décrite pour **node.js**.



```
$cd ~  
$git clone http://github.com/isaacs/npm.git  
$cd npm  
$./configure  
$make
```

L'installation est scriptée (dans le fichier Makefile). Elle s'effectue également à l'aide de **make** (en **root** pour disposer des droits requis).



```
$sudo make install
```

Express, Nano...

Le couple **node.js** - **npm** est maintenant pleinement fonctionnel. La commande **npm** peut être utilisée pour importer quelques bibliothèques usuelles. L'import peut être lié à l'utilisateur à l'origine de la requête ou commun à l'ensemble des utilisateurs (option **-g**). L'activation de cette option requiert des droits élevés (**root**).

Express est un Framework Web permettant de programmer selon le modèle MVC. **Jade** est moteur de rendu produisant du code HTML à partir de vues définies par l'utilisateur. Session gère les sessions des utilisateurs de façon rémanente ou non selon le pilote de stockage rattaché (par défaut les données sont stockées en mémoire). Nano assure l'interface entre **node.js** et la base de données NoSQL **CouchDB**. L'identification des usagers (internauts) et l'authentification des échanges sont assurées par **Passport** (en local ou par l'intermédiaire de service standard : twitter, facebook). Le dernier module **socket.io** facilite l'invocation de méthodes JavaScript du server par le browser et vice-versa (Ajax puissance infinie).



```
$cd ~
```

```
$sudo npm -g install express jade session nano passport socket.io
```

La découverte et la vérification du bon fonctionnement de la plate-forme peuvent commencer en exécutant un script (à saisir dans l'éditeur de votre choix et à enregistrer sous le nom *server.js*). En ligne de commande (sous Ubuntu), l'un des éditeurs les plus efficace est convivial est **nano**. Les options **-w** et **-N** précisent respectivement que les lignes longues ne seront pas découpées sur plusieurs lignes (word wrap) et que les retours-chariot des autres systèmes d'exploitation (Windows, Mac OS) ne seront par interprétés comme tels (ils seront affichés).



```
$nano -w -N server.js
```

Le code source ci-après doit être saisi dans l'éditeur (ou copier-coller).

```
var http = require('http'); http.createServer(function(req,
res){
res.writeHead(200, {'Content-Type': 'text/plain'});
res.end('Hello World\n'); }).listen(3000,
'127.0.0.1'); console.log('Server running at
http://127.0.0.1:3000');
```

La séquence **[Ctrl]-[X]** permet, après confirmation (**y**), de sauvegarder le fichier. La première ligne charge le module `http` (dans l'instance de même nom). Un serveur HTTP à l'écoute en local sur le port 3000 est ensuite créé. La fonction rattachée à la réception des données renvoie *Hello World*. Les modules (bibliothèques) sont recherchés selon les emplacements définis dans la variable d'environnement `NODE_PATH`. Elle peut être créée automatiquement à chaque ouverture de session en ajoutant sa définition au fichier **~/.profile**.

```
user@ubuntu:~$ echo 'export NODE_PATH="$("$npm root)":"$("$npm root -g)'"
export NODE_PATH="/home/user/node_modules":"/usr/local/lib/node_modules"
user@ubuntu:~$ echo 'export NODE_PATH="$("$npm root)":"$("$npm root -g)'" >> ~/
.profile
user@ubuntu:~$ _
```



```
$echo 'export NODE_PATH=..:"$($npm root)":"$($npm root -g)'" >>
~/.profile
```



La commande est d'un seul tenant (sur **1 ligne**). Elle comprend **2 signes « > »** pour compléter le fichier .profile (avec 1 seul « > » le fichier est écrasé).

Pour l'utilisateur **user**, la ligne **export NODE_PATH =.:~/ "nodev0.6.18/node_modules":"/usr/local/lib/node_modules"** est ajoutée. Les chemins indiqués correspondent respectivement au dossier courant (.) à celui des packages installés uniquement pour l'utilisateur et à ceux accessibles à tous. Le chemin rattaché à l'utilisateur dépend de l'emplacement à partir duquel l'archive

node-v0.6.18.tar.gz a été décompressée (ici ~). La variable peut être transférée dans l'environnement d'exécution lors de l'exécution du script sous un compte utilisateur distinct (ex. **user**).



```
$sudo su -  
cd /home/user  
#sudo -u user env NODE_PATH=$NODE_PATH node server.js &
```

La première commande (**sudo su -**) permet de passer en **root** pour l'expérimentation. Elle doit être omise lorsque l'utilisateur est déjà identifié en administrateur ou si la commande est exécutée à l'amorçage du système pour démarrer l'application en **daemon** avec des droits restreints (non **root**).

```
user@ubuntu:~$ sudo su -  
root@ubuntu:~# cd /home/user  
root@ubuntu:/home/user# sudo -u user env NODE_PATH=$NODE_PATH node server.js &  
[1] 6241  
root@ubuntu:/home/user# Server running at http://127.0.0.1:3000  
  
root@ubuntu:/home/user# exit  
déconnexion  
user@ubuntu:~$ wget http://127.0.0.1:3000  
--2012-05-30 13:54:18-- http://127.0.0.1:3000/  
Connexion vers 127.0.0.1:3000... connecté.  
requête HTTP transmise, en attente de la réponse... 200 OK  
Longueur: non spécifié [text/plain]  
Sauvegarde en : «index.html»  
  
[ <=> ] 12 --.-K/s ds 0s  
2012-05-30 13:54:18 (1,37 MB/s) - «index.html» sauvegardé [12]  
  
user@ubuntu:~$ more index.html  
Hello World  
user@ubuntu:~$ _
```

Lancement et consultation du service *Hello World*... v(^_^)v

Installation de CouchDB 1.2.0

Apache CouchDB est une base de données NoSQL. Contrairement aux bases de données “classiques” SQL, elle n'utilise pas une structure en table où chaque ligne représente une entité et chaque colonne une propriété de celle-ci. Les entités manipulées sont des documents JSON. Ils décrivent les objets (sous une forme sérialisée). Chaque document peut adopter la structure la mieux adaptée à sa représentation. Le stockage n'impose aucune contrainte de structure. Le nombre, la nature, le type et l'imbrication des caractéristiques d'un document (d'un objet ou d'une entité pour conserver le parallèle avec SQL) sont libres. Les requêtes sont effectuées à l'aide du protocole HTTP (le même que celui des navigateurs Internet). Le langage d'interrogation utilisé est JavaScript. Le projet est hébergé par la fondation Apache (gage de sérieux et de pérennité). La page d'accueil du projet peut être consultée à l'adresse <http://couchdb.apache.org>. En dehors de la présentation sommaire, l'information la plus importante est le lien vers le téléchargement (*Download*). Ce dernier est disponible sous la forme d'une archive des sources (à destination de Linux ou des curieux) et de deux distributions binaires respectivement dédiées à Mac OS X et Windows.

L'adresse **<http://mirror.speednetwork.de/apache/couchdb/releases/1.2.0/apache-couchdb-1.2.0.tar.gz>** permet de télécharger les sources (à partir d'un miroir situé en Allemagne). Les commandes ci-après permettent de créer l'exécutable pour Linux.



```
$cd ~
```

```
$sudo apt-get install libmozjs185-dev libicu-dev libcurl4-gnutls-dev  
libtool erlang-dev erlang-inets erlang-os-mon erlang-eunit
```

```
$wget
```

```
http://mirror.speednetwork.de/apache/couchdb/releases/1.2.0/apache-  
couchdb-1.2.0.tar.gz
```

```
$tar xzf apache-couchdb-1.2.0.tar.gz
```

Le dossier cible d'extraction **apache-couchdb-1.2.0** est automatiquement créé. Les commandes utiles à la compilation (et les sources) se trouvent à l'intérieur de ce dernier. La commande **configure** identifie le contexte (propre au système utilisé) et vérifie la présence de l'ensemble des composantes requises : chaîne de développement et bibliothèques connexes.



```
$cd apache-couchdb-1.2.0
```

```
$./configure
```

```
$make
```

La dernière étape consiste à déployer CouchDB sur le système hôte (et à associer les droits d'usage requis à chaque dossier utilisé).



```
$sudo make install
```

```
$sudo useradd -d /usr/local/var/lib/couchdb couchdb
```

```
$sudo chown -R couchdb:couchdb /usr/local/var/lib/couchdb
$sudo chown -R couchdb:couchdb /usr/local/lib/couchdb
$sudo chown -R couchdb:couchdb /usr/local/var/log/couchdb
$sudo chown -R couchdb:couchdb /usr/local/var/run/couchdb
$sudo chown -R couchdb:couchdb /usr/local/etc/couchdb
$sudo /usr/local/etc/init.d/couchdb start
```



La mise en place des droits doit être effectuée avec une très grande vigilance. Le moindre oubli, la moindre erreur est synonyme de dysfonctionnement (plus ou moins simple à diagnostiquer par la suite).

Comment démarrer/arrêter CouchDB automatiquement ?

La mise en place d'un lien symbolique dans `/etc/init.d` vers le fichier `/usr/local/etc/init.d/couchdb` et la déclaration du service (avec les réglages par défaut) assure le démarrage et l'arrêt du serveur de base de données conjointement au système hôte.



```
$sudo ln -s /usr/local/etc/init.d/couchdb /etc/init.d/couchdb
$sudo update-rc.d couchdb defaults
```

Pourquoi ne pas utiliser le paquet CouchDB ?

La version distribuée par Ubuntu (`#apt-get couchdb`) est obsolète : 1.0.1.

Introduction à CouchDB

CouchDB est une base de données orientée document. Elle se distingue des bases de données traditionnelles (SQL) par la méthode de stockage des données et les mécanismes inhérents à leur accès. La comparaison et le développement des théories inhérentes aux deux approches dépassent le cadre de cette introduction. Seuls les aspects pratiques, liés à sa mise en œuvre y sont abordés.

À la suite de son installation, la base de données est accessible sur le système hôte (et uniquement à partir de celui-ci) à l'adresse **http://localhost:5984/_utils**. L'accès à partir d'un poste client s'effectue selon deux méthodes.

Méthode laxiste

En remplaçant **localhost** de l'URL par l'adresse IP (ou le nom DNS) du système hôte, exemple : **http://192.168.0.8:5984/_utils**.



La commande **ifconfig** permet de connaître l'adresse IP associée à l'interface réseau (usuellement **eth0**) du système hôte.

La valeur de l'adresse écoutée par CouchDB (**bind_address**) dans le fichier **/usr/local/etc/couchdb/default.ini** doit impérativement être modifiée au préalable. La valeur **127.0.0.1** (**localhost**) doit être remplacée par **0.0.0.0**. Le fichier peut être édité à l'invite de commande du shell à l'aide de **nano** en root. La valeur se situe dans la section **[httpd]** ([Ctrl]-[X] pour quitter **nano**).

```
bind_address = 0.0.0.0_
authentication_handlers = {couch_httpd_oauth, oauth_authentication_handler}, {cs
default_handler = {couch_httpd_db, handle_request}
secure_rewrites = true
whost_global_handlers = _utils, _uuids, _session, _oauth, _users
allow_jsonp = false
; Options for the MochiWeb HTTP server.
;server_options = [{backlog, 128}, {acceptor_pool_size, 16}]
; For more socket options, consult Erlang's module 'inet' man page.
;socket_options = [{recbuf, 262144}, {sndbuf, 262144}, {nodelay, true}]
log_max_chunk_size = 1000000

[ssl]
port = 6984

[log]
file = /usr/local/var/log/couchdb/couch.log
level = info
include_sasl = true
```

^G Aide ^O Écrire ^R Lire fich. ^Y Page préc. ^K Couper ^C Pos. cur.
^X Quitter ^J Justifier ^W Chercher ^V Page suiv. ^U Coller ^T Orthograp.



```
#nano -w /usr/local/etc/couchdb/default.ini
```

Le redémarrage de CouchDB est nécessaire pour prendre en compte la modification des paramètres. Le port doit également être ouvert au niveau du pare-feu (d'où l'appel à **ufw**).



```
$sudo /usr/local/etc/init.d/couchdb restart
```

```
$sudo ufw allow 5984/TCP
```

Et hop, la porte est ouverte... en grand... °_O"

Méthode sécurisée

En production, si les applications sont installées sur le serveur doté de la base de données (CouchDB), les bonnes pratiques recommandent de conserver la valeur **127.0.0.1**. Elle limite les accès des bases aux seules applications s'exécutant sur le serveur hôte. L'accès à distance peut être obtenu par l'intermédiaire d'un tunnel SSH sur le client distant (poste de travail ou autre)



```
$ssh user:pass@désignation_du_serveur_ou_ip -L  
5984:localhost:5984
```

La valeur 5984 est celle attribuée à **port** dans **default.ini** par défaut : vérifier la valeur réellement renseignée. Suite à l'activation du tunnel SSH, l'accès sur le client s'effectue comme si CouchDB était en local (sur le poste client). Lorsque le client dispose également d'une installation locale (sur le même port 5984), il faut utiliser un port distinct pour le tunnel SSH (ex. 5985)



```
$ssh user:pass@désignation_du_serveur_ou_ip -L  
5985:localhost:5984
```

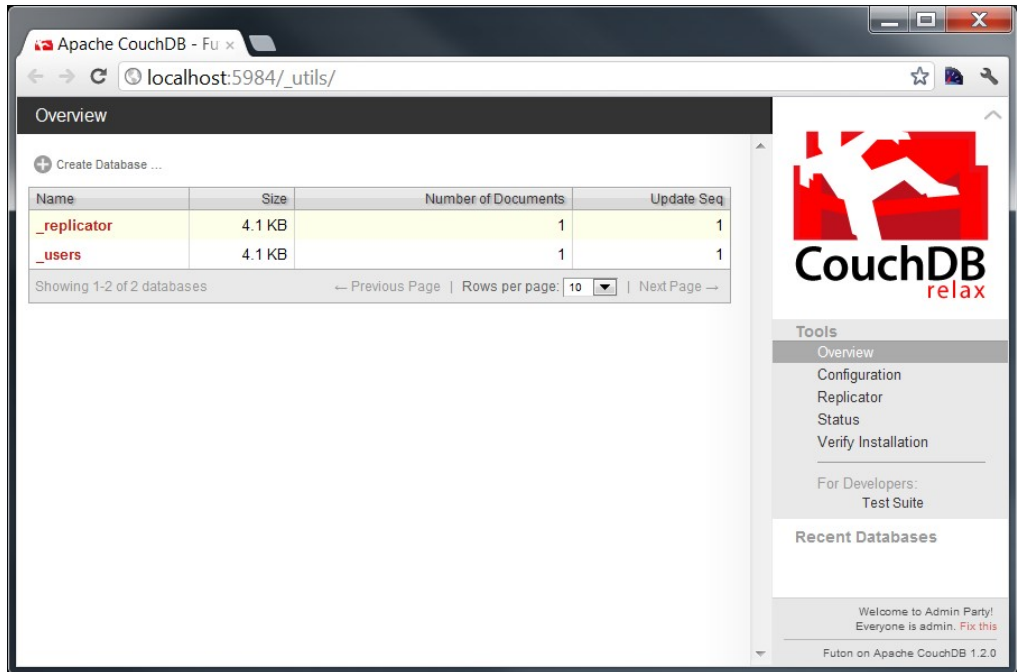
La base locale du client reste accessible en 5984 (<http://localhost:5984>). Le port 5985 est associé à l'instance distante (<http://localhost:5985>).

Futon

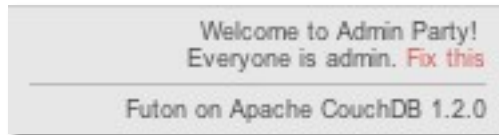
Une fois les problématiques d'accès résolues (avec ou sans laxisme), l'appel de l'une ou l'autre des URL ci-après :

- http://localhost:5984/_utils (via tunnel SSH) ou
- http://désignation_du_serveur_ou_ip:5984 (en direct) donne accès

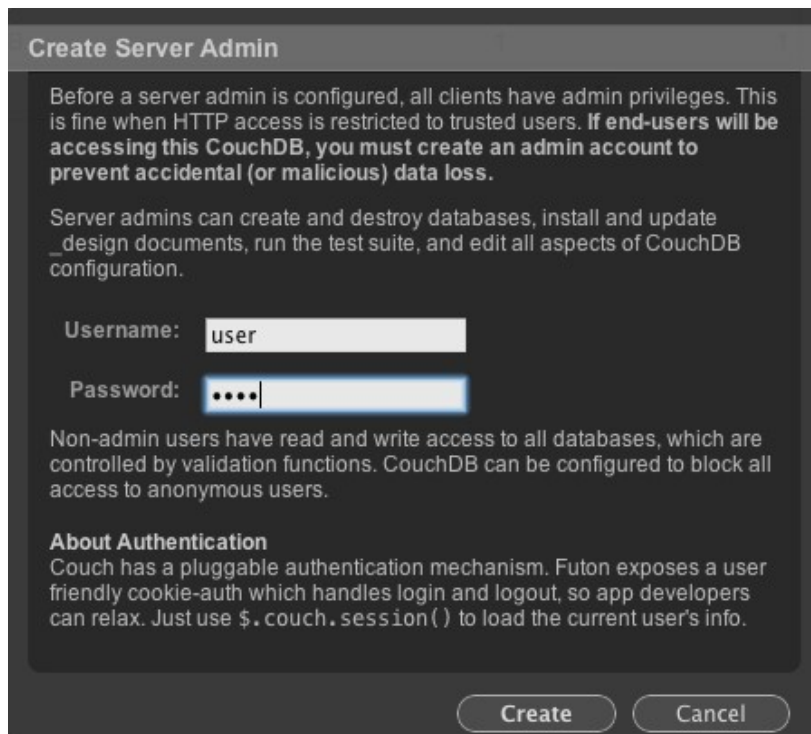
l'administration de CouchDB sous la forme d'une interface Web. Elle est compatible avec les principaux navigateurs Web (chrome, firefox, internet explorer) même si - dans le cas des illustrations - Chrome est utilisé.



L'interface affichée se nomme **Futon**. Elle comporte dans le coin inférieur droit un message de bienvenue et d'avertissement "Welcome to Admin Party ! Everyone is Admin." : tous les usagers sont assimilés à des administrateurs.



Pour sécuriser l'accès, un clic sur **Fix It** permet de déclarer un administrateur. Il suffit d'indiquer son nom (*username*) et son mot de passe (*password*) et d'activer le bouton **Create**.

A screenshot of a "Create Server Admin" dialog box. The title bar says "Create Server Admin". The main text reads: "Before a server admin is configured, all clients have admin privileges. This is fine when HTTP access is restricted to trusted users. If end-users will be accessing this CouchDB, you must create an admin account to prevent accidental (or malicious) data loss." Below this, it says: "Server admins can create and destroy databases, install and update _design documents, run the test suite, and edit all aspects of CouchDB configuration." There are two input fields: "Username:" with the value "user" and "Password:" with masked characters "....". Below the inputs, it says: "Non-admin users have read and write access to all databases, which are controlled by validation functions. CouchDB can be configured to block all access to anonymous users." At the bottom, there is a section titled "About Authentication" with text: "Couch has a pluggable authentication mechanism. Futon exposes a user friendly cookie-auth which handles login and logout, so app developers can relax. Just use \$.couch.session() to load the current user's info." At the very bottom, there are two buttons: "Create" and "Cancel".

Le message (dans le coin en bas à droite) s'actualise afin de proposer des liens pour ajouter d'autres administrateurs (*Setup more admins*), changer le mot de passe (*Change password*) ou se déconnecter (*Logout*).



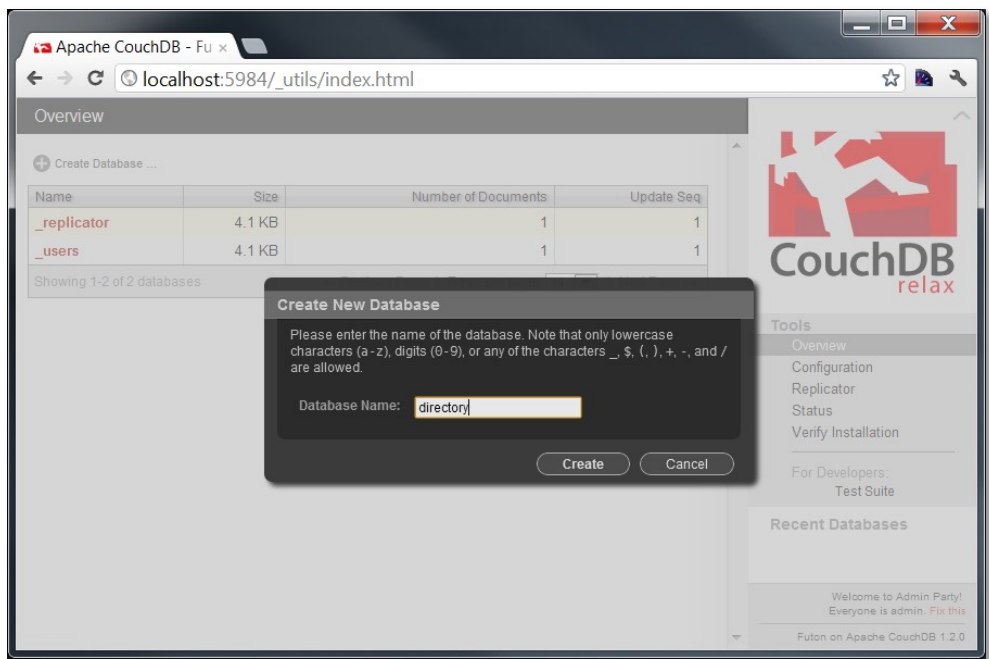
La déclaration du premier administrateur active la gestion des accès aux bases de données. Lorsque l'utilisateur n'est pas connecté, le message lui propose de s'enregistrer comme nouvel utilisateur "lambda" (*signup*) ou de se connecter (*login*). Les exemples ci-après requièrent les droits d'administration : vous devez soit vous connecter avec un compte d'administrateur (si vous en avez créé un), soit poursuivre en l'état (tous les usagers sont des administrateurs).

Overview

Suite à l'installation deux bases de données (de type) "système" sont proposées. La première, **_replicator**, concerne la mise en place et l'exécution de répliquions entre différentes bases de données (locales et/ou distantes) afin d'effectuer une sauvegarde, une copie ou une synchronisation de plusieurs nœuds dans le cadre d'une répartition de charge. La seconde, **_users**, comporte la définition des usagers : nom, mot de passe, droits. L'usage des bases systèmes et les actions connexes sont abordés en **Réplication**. Les actions décrites ci-après présentent les fonctions élémentaires : création d'une base de données, ajout de documents, modification, etc. Les actions, effectuées à l'aide de l'interface **Futon**, sont également réalisables directement par appel d'URL (à la main en ligne de commande ou par l'intermédiaire d'une application). L'invite de commande et l'utilitaire cURL sont utilisés en parallèle à cet effet pour présenter les deux contextes : interface Web conviviale, ligne de commande (austère ?). La réalisation de scripts, d'applications et l'automatisation des traitements requièrent souvent les deux approches. Le module **nano** est introduit à cet effet pour permettre l'Accès à partir de node.js.

Création d'une base de données

La vue d'ensemble (*Overview*) liste les bases de données déclarées (`_replicator`, `_user`). Le bouton **Create Database** affiche une boîte de dialogue ne comportant qu'une seule zone de saisie (*Database Name*). L'indication du nom choisi, constitué de chiffres et des caractères "a" à "z", suivi d'un clic sur le bouton **Create**, crée la base de données et en propose un aperçu (initialement vide) comportant les *n* premiers documents. La valeur *n* précise le nombre de documents affichés (*Rows per page*). Les valeurs possibles sont 10, 25, 50 et 100. Pour consulter les documents au-delà de la limite, il faut naviguer entre les pages à l'aide des liens : **Previous Page**, **Next Page**.



La création d'une base de données est également possible en HTTP. L'utilitaire cURL est utilisé car il permet de spécifier la méthode à utiliser, les données associées, les en-têtes, etc. Disponible nativement sous Linux, des versions sont également proposées pour d'autres systèmes d'exploitation (dont Windows). La réponse **unauthorized** appelle à compléter l'URL soumise avec le nom et le mot de passe d'un usager (dans ce cas d'un administrateur).



```
$curl -X PUT http://localhost:5984/directory
```

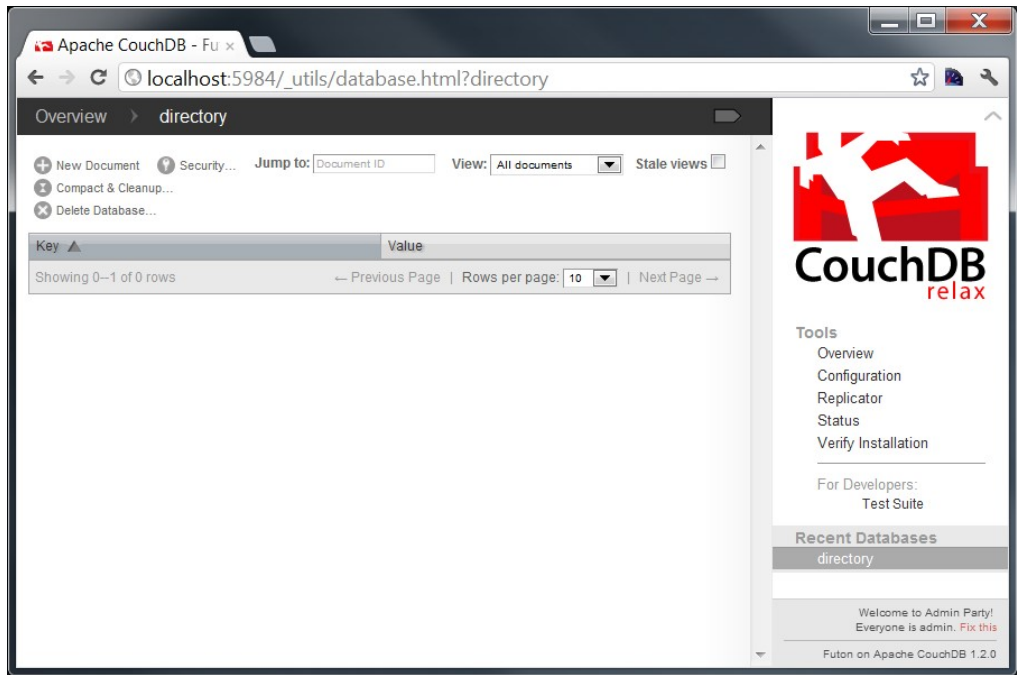
```
{"error":"unauthorized","reason":"You are not a server admin."}
```

```
$curl -X PUT http://user:pass@localhost:5984/directory
```

```
{"ok":true}
```

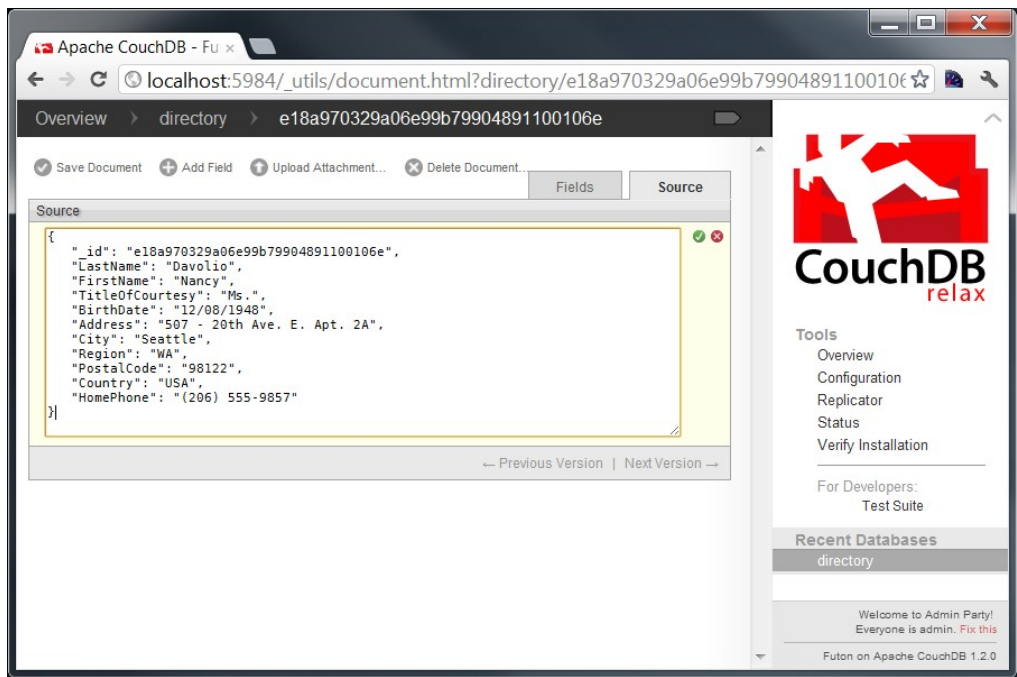
Création d'un document

L'aperçu de la base de données (directory dans le cadre de l'exemple) comporte un bandeau regroupant l'ensemble des actions possibles : création d'un nouveau document (*New Document*), définition des droits (*Security*), maintenance (*Compact & Cleanup*), suppression (*Delete Database*), navigation, etc. La création d'un document s'effectue en deux étapes. **New Document** génère un document vierge et propose un identificateur univoque pour ce dernier (**_id**). L'expression de ce dernier varie selon le mode de consultation choisi : mode "classique" en fiche (*Fields*) ou mode JSON (*Source*). Le passage d'un mode à l'autre s'effectue à l'aide des onglets en haut à droite de la zone de saisie. L'identifiant généré est un GUID (global unique identifier) , exemple : a8e72441092cc97a78b3cd973400126a (à vos souhaits). Il peut être remplacé par une valeur au choix (dès lors qu'elle est unique). L'_id correspond à l'immatriculation du document. Une fois défini, l'identifiant est inaltérable (jusqu'à la destruction du document). Lors du remplacement de la valeur proposée par une expression plus "humaine" (genre "ean_3256220171373"), il convient de s'assurer de la rectitude et de la pérennité des règles de nommage choisies.

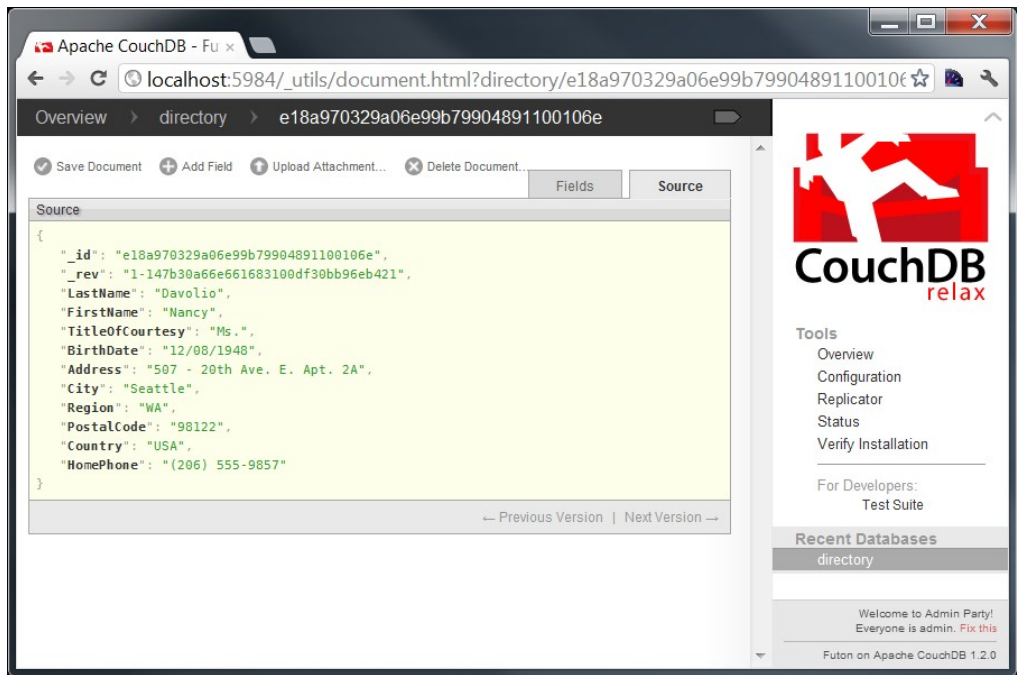


La création d'un document peut s'effectuer en mode fiche (*Fields*). **Add Field** propose de nommer la caractéristique ajoutée et de lui affecter une valeur (bascule en mode fiche). Le mode JSON (Source) propose de saisir l'intégralité du document selon la norme JSON (ce qui s'avère à l'usage beaucoup plus efficace). Un document est en réalité un tableau associatif dont les clefs (placées entre guillemets) sont les noms des propriétés. Elles sont suivies par le caractère ":" et la valeur associée. La valeur peut être un booléen (ex. true, false), un nombre (ex. 10.2), une chaîne (ex. "Exemple"), un tableau de valeurs (ex. avec des nombres [1, 2, 3]), un tableau associatif (ex. { "Taille" : 170, "Poids" : 60 }) ou une combinaison de l'ensemble. L'identifiant ("_id") et l'indice de version **révision** ("_rev") doivent être

conservés sans modification. La version (**révision**) est actualisée par le serveur à chaque modification du document. Lors de la soumission d'une modification, ce mécanisme permet de s'assurer que la version (**révision**) du document sauvegardé correspond à celle qui est soumise avec les modifications. Dans le cas contraire, la modification est refusée par CouchDB (car le document a fait l'objet d'une modification intermédiaire). Les données sont donc toujours cohérentes. La révision apparaît sur un nouveau document après sauvegarde.



L'enregistrement du nouveau document s'effectue en deux étapes : le bouton de validation de la saisie (coche verte) doit être cliqué puis le lien **Save Document**. La présentation du document est actualisée : la révision **_rev** apparaît immédiatement après l'identifiant **_id**.



La création d'un document en HTTP emploie la même méthode que la création de la base de données : **PUT**. L'expression JSON décrivant le document suit le paramètre **-d** introduisant la partie des données (data) de la méthode appelée. La commande est à saisir d'un bloc sur une seule ligne avec des séquences d'échappement pour les guillemets. La réponse comporte l'identifiant **_id** généré pour le document et sa révision **_rev**. Le source JSON du document créé peut

également comporter la valeur de l'identifiant souhaité (ex. ...

```
-d "{ \"_id\": \"code_unique\", \"LastName\": \"Daviolo\", ...}).
```

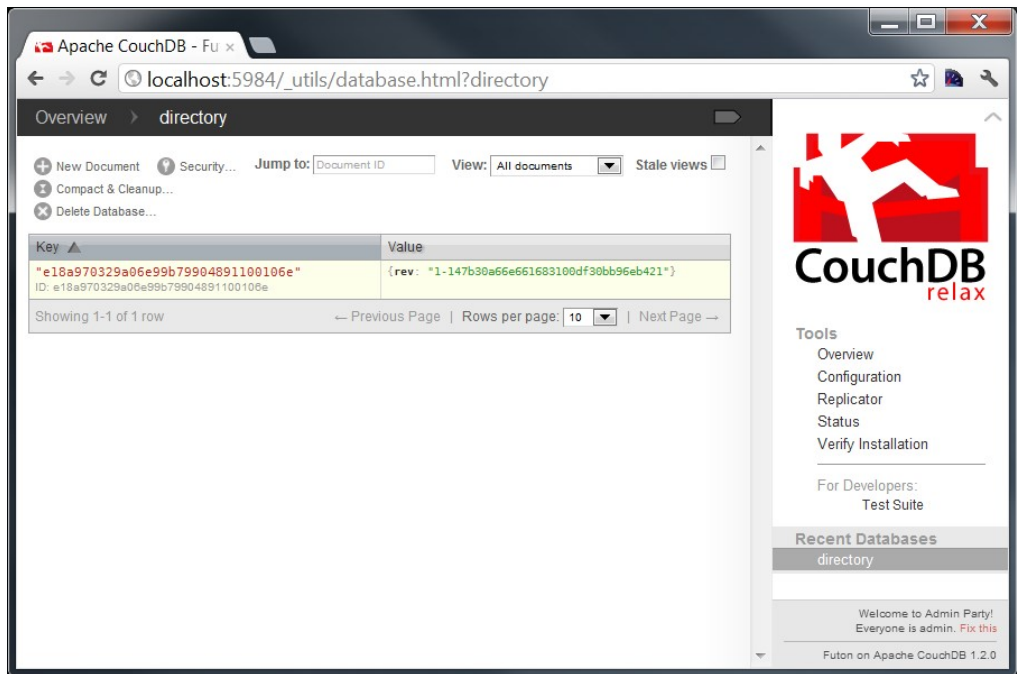


```
$curl -X PUT -d "{ \"LastName\": \"Daviolo\", \"FirstName\": \"Nancy\",  
\"TitleOfCourtesy\": \"Ms.\", \"BirthDate\": \"12/08/1948\", \"Address\":  
\"507 - 20th Ave. E. Apt. 2A\", \"City\": \"Seattle\", \"Region\": \"WA\",  
\"PostalCode\": \"98122\", \"Country\": \"USA\", \"HomePhone\":  
\"(206) 555-9857\" }"  
http://user:pass@localhost:5984/directory/d9b756d7befc595d821bad  
dc9e00022b
```

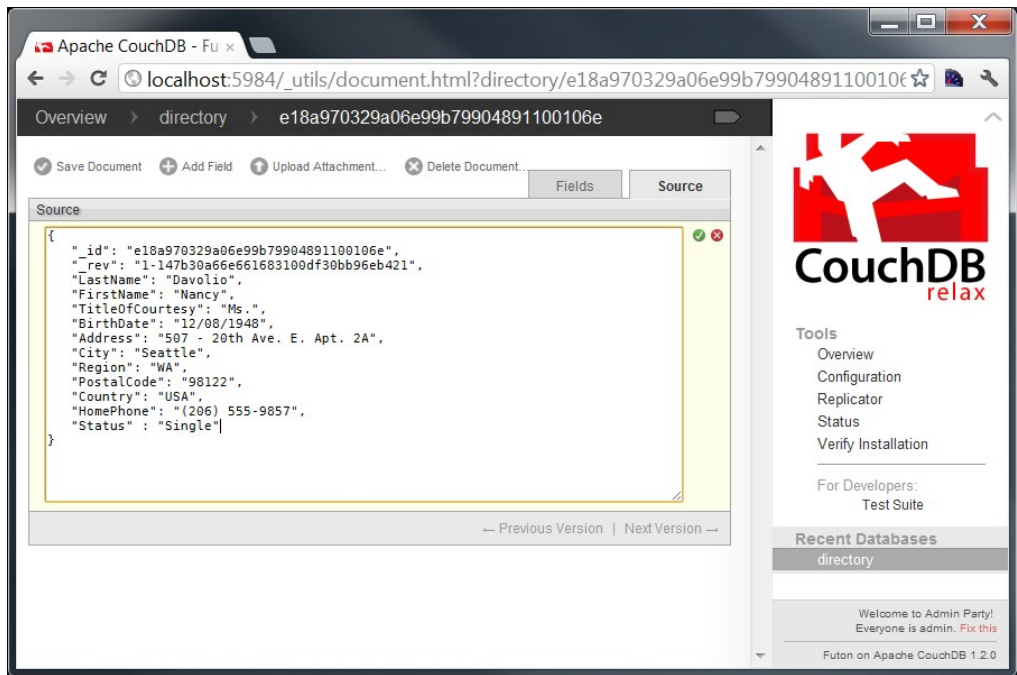
```
{"ok":true,"id":"d9b756d7befc595d821baddc9e00022b","rev":"1f  
8ea929bae4ff36a46883359bfbee945"}
```

Consultation, Modification, Suppression...

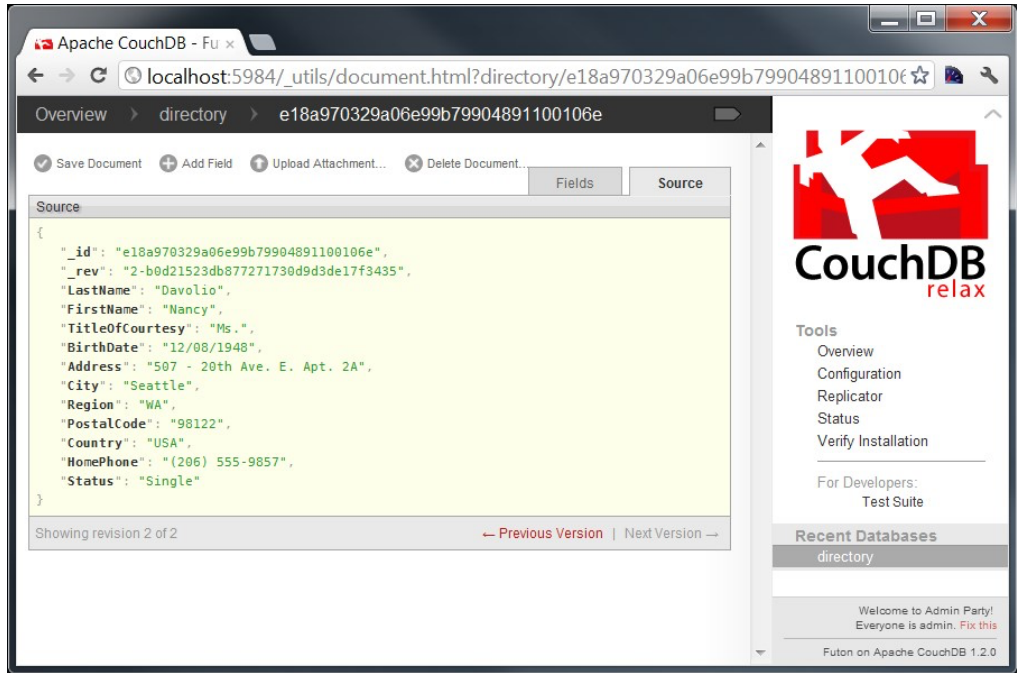
Le rappel de la page d'aperçu/vue d'ensemble (*Overview*) présente la liste des documents disponibles (en l'occurrence un : celui qui vient d'être ajouté). La vue proposée (*View*) comporte tous les documents (*All documents*). Elle propose deux colonnes : **_id** à gauche et **_rev** à droite. Les vues permettent d'accéder aux documents et à leurs données selon les critères choisis et avec des opérations connexes (regroupements, calculs, ordonnancements, etc. Un gros (énorme ?) raccourcis consiste à considérer les vues comme l'équivalent des requêtes `select` en SQL.



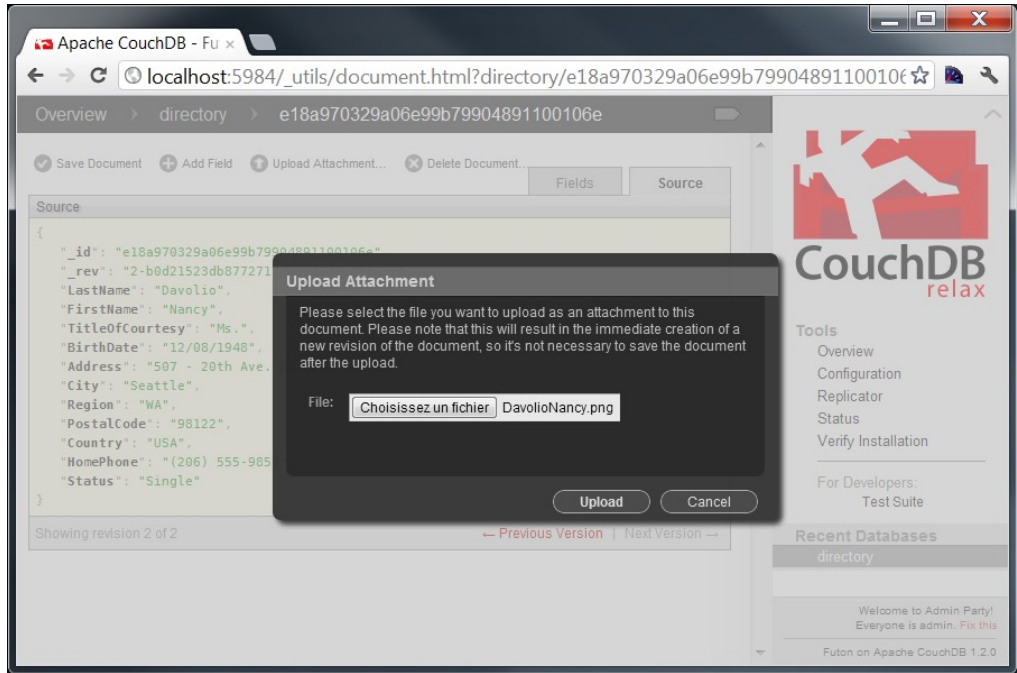
Un clic sur l'identifiant du document à modifier, un clic (sur Source) pour passer en mode JSON et un double-clic dans le corps permet de passer en édition sur le document. À l'exception de `_id` et `_rev`, tout peut être modifié. L'exemple ajoute une caractéristique (*Status*).



La sauvegarde des modifications est identique à celle du document en création : un clic sur la coche verte (validation de la saisie) et un autre sur **Save Document**.

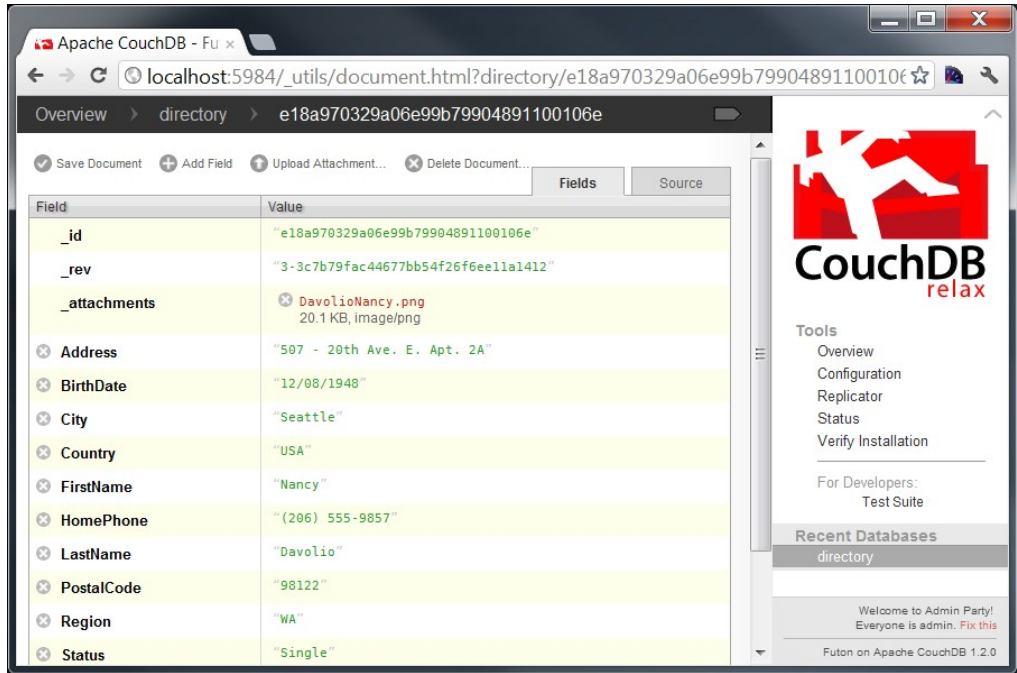


Cerise sur le gâteau, CouchDB permet l'inclusion de pièces jointes au document. Le rattachement d'un fichier se limite à un clic sur **Upload Attachment** et à indiquer le chemin d'accès au fichier à joindre (sans aucune limitation de type). L'ajout de la première pièce jointe provoque la création d'une nouvelle caractéristique (tableau associatif "nom de fichier" : contenu du fichier) nommée **attachments**.



Lors de la consultation, **Futon** précise le nom (ex. "DavolioNancy.png »), la taille (ex. 20.1KB) et le type MIME (ex. image/png) des fichiers attachés.

La dernière option du bandeau **Delete Document** supprime le document (après confirmation).



Consultation du document en HTTP (la mise en forme de la réponse est effectuée par soucis de lisibilité : le flux est renvoyé sur une seule ligne).



```
$curl -X GET
```

```
http://user:pass@localhost:5984/directory/d9b756d7befc595d821bad  
dc9e00022b
```

```
{
```

```
  "_id": "d9b756d7befc595d821baddc9e00022b",  
  "_rev": "1-f8ea929bae4ff36a46883359bfbee945",  
  "LastName": "Daviolo",
```

```
"FirstName": "Nancy",  
"TitleOfCourtesy": "Ms.",  
"BirthDate": "12/08/1948",  
"Address": "507 - 20th Ave. E. Apt. 2A",  
"City": "Seattle",  
"Region": "WA",  
"PostalCode": "98122",  
"Country": "USA",  
"HomePhone": \\(206\\) 555-9857  
}
```

La modification du document consiste à retourner le flux (précédemment collecté) actualisé avec les ajouts/modifications souhaités. L'identifiant **_id** et la révision **_rev** doivent impérativement être renseignés à l'identique (à défaut l'action échoue). En cas de succès, la réponse comporte l'indice de révision : **2321181d3ee4c2be4e1e7992eccf18d5f**.



```
$curl -X PUT -d "{ \"_rev\": \"1-f8ea929bae4ff36a46883359bfbee945\",  
\"LastName\": \"Daviolo\", \"FirstName\": \"Nancy\",  
\"TitleOfCourtesy\": \"Ms.\", \"BirthDate\": \"12/08/1948\", \"Address\":  
\"507 - 20th Ave. E. Apt. 2A\", \"City\": \"Seattle\", \"Region\": \"WA\",  
\"PostalCode\": \"98122\", \"Country\": \"USA\", \"HomePhone\":  
\"(206) 555-9857\", \"Status\": \"Single\" }"  
http://user:pass@localhost:5984/directory/d9b756d7befc595d821bad  
dc9e00022b
```

```
{"ok": true, "id": "e18a970329a06e99b79904891100106e", "rev": "2-  
321181d3ee4c2be4e1e7992eccf18d5f"}
```

L'ajout de pièces jointes en HTTP est également possible. L'action - assez laborieuse - n'est pas abordée ici avec cURL. Il est préférable - et bien plus aisé - d'utiliser **node.js**.

La suppression emploie la méthode HTTP **DELETE**. L'URL appelée doit impérativement comporter l'indice de révision. **ATTENTION** : la suppression ne supprime pas physiquement le document de la base de données. Un document supprimé, comme un document remplacé par une nouvelle version, n'apparaît plus dans les résultats (sauf requête spécifique).



```
$curl -X DELETE
```

```
http://user:pass@localhost:5984/directory/d9b756d7befc595d821bad  
dc9e00022b?rev=2-321181d3ee4c2be4e1e7992eccf18d5f
```

```
{"ok":true,"id":"d9b756d7befc595d821baddc9e00022b","rev":"3f  
83bcf93ff31f2e9cb6c6601fd7b96a1"}
```

L'élimination physique des documents obsolètes s'effectue par compactage de la base de données. L'opération - accessible dans **Futon** - peut être invoquée en HTTP.



```
$curl -X POST -H "Content-Type: application/json"  
http://user:pass@localhost:5984/directory/_compact
```

```
{"ok":true}
```

HTTP (dans le browser)

CouchDB peut être interrogé directement à partir d'un navigateur Web (au hasard Chrome). L'appel de l'URL <http://localhost:5984/directory> (à saisir dans la barre d'adresse) retourne les indications relatives à la base de données **directory**.

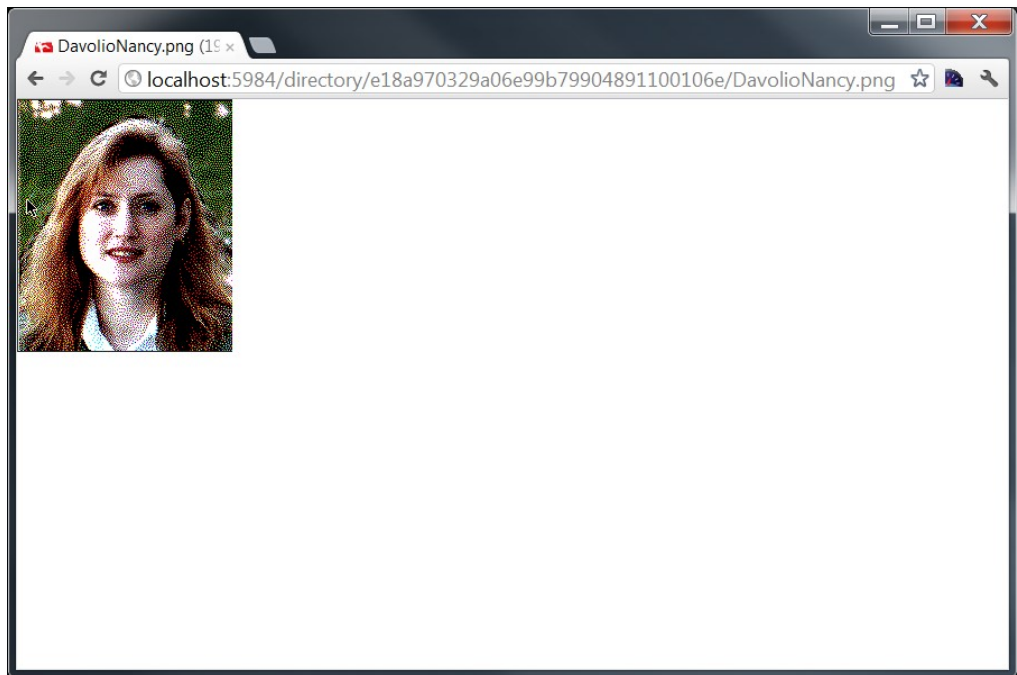
```
{  "db_name":"directory",
  "doc_count":1,
  "doc_del_count":0,
  "update_seq":3,
  "purge_seq":0, "compact_running":false, "disk_size":32866,
  "data_size":21287, "instance_start_time":"1334747552868000",
  "disk_format_version":6, "committed_update_seq":3
}
```

L'URL peut être complétée avec l'identifiant **_id** d'un document. Les informations renvoyées correspondent alors à ce dernier : <http://localhost:5984/directory/e18a970329a06e99b79904891100106e> .

```
{  "_id":"e18a970329a06e99b79904891100106e",
  "_rev":"3-3c7b79fac44677bb54f26f6ee11a1412",
  "LastName":"Davolio", "FirstName":"Nancy", "TitleOfCourtesy":"Ms.",
  "BirthDate":"12/08/1948", "Address":"507 - 20th Ave. E. Apt. 2A",
  "City":"Seattle", "Region":"WA", "PostalCode":"98122",
  "Country":"USA", "HomePhone":"(206) 555-9857", "Status":"Single",
  "_attachments":{"
    "DavolioNancy.png":{"
      "content_type":"image/png", "revpos":3,
      "digest":"md5-Hokv1O9Chuk6V2lUN5j5fA==",
```

```
      "length":20593,  
      "stub":true  
    }  
  }  
}
```

Les pièces jointes sont également accessible en précisant le nom du fichier (clef du tableau associatif **_attachments** : <http://localhost:5984/directory/e18a970329a06e99b79904891100106e/DavolioNancy.png> .



CouchDB à partir de Node.JS

La base de données CouchDB et node.js constituent une (la ?) plate-forme JavaScript Everywhere idéale : un seul langage de la base de données au navigateur Web, entrées/sorties non bloquantes, gestion événementielle, etc. L'excellent module **nano** (de Nuno Job) facilite grandement la mise en œuvre de cette synergie. L'ensemble des opérations, effectuées lors de l'introduction de CouchDB, sont présentées dans ce contexte.

Création d'une base de données

La première ligne charge le module et instancie une nouvelle connexion au serveur de données CouchDB (installé localement avec un administrateur **user** doté du mot de passe **pass**). La seconde opération crée la base de données **directory**. La méthode **create** accepte en premier paramètre le nom de la base de données à créer et en second une fonction **callback** (appelée à l'issue du traitement). L'argument **err** contient des informations relatives à une éventuelle anomalie. En cas de succès, il prend la valeur **null**.

couchcrt.js

```
var nano=require('nano')('http://user:pass@localhost:5984');
nano.db.create('directory',function(err){
  console.log(err);
});
```

Le lancement, en ligne de commandes, s'effectue comme pour toutes les autres applications **node.js**. L'affichage de la valeur **null** confirme la création de la base. **Futon** (http://localhost:5984/_utils) met en évidence la nouvelle base de données dans la liste.



```
$nano couchcrt.js  
null
```

Une nouvelle exécution provoque une erreur car la base de données existe déjà. Le message affiché explicite la nature de l'erreur. Le seul moyen d'exécuter avec succès le script consiste alors à supprimer préalablement la base de données **directory** (sous **Futon**).

```
{ [Error: The database could not be created, the file already exists.]  
  error: 'file_exists', 'status-code': 412,  
  scope: 'couch',  
  request:  
    { method: 'PUT',  
      headers:  
        { 'content-type': 'application/json',  
          accept: 'application/json' },  
      uri: 'http://user:pass@localhost:5984/directory',  
      callback: [Function] } }
```

En pratique, une erreur détectée doit être gérée par un palliatif ou le déclenchement d'une exception. Le test **if (err) throw err** s'inscrit dans cette démarche et protège le corps du traitement "...".

```
var nano=require('nano')('http://user:pass@localhost:5984');
nano.db.create('directory',function(err){      if (err)
throw err;

    ...
});
```

Une alternative consiste à systématiquement tenter de détruire la base à créer. Le traitement peut ainsi être répété à loisir (ce qui peut s'avérer extrêmement dangereux en cas de lancement inopiné si les données sont critiques).

```
var nano=require('nano')('http://user:pass@localhost:5984');
nano.db.destroy('directory',function(){
    nano.db.create('directory',function(err){
        if (err) throw err;
        console.log('Create: Ok');
    });
});
```

Création d'un document

L'objet **db** assure l'interface avec la base de données associée **directory**. L'ensemble des manipulations relatives aux documents (et aux vues) s'effectuent par son intermédiaire. L'ajout et la mise à jour sont perçus comme une seule et même opération : insertion d'un nouveau document (éventuellement en substitution d'une version antérieure). La méthode **insert** s'utilise différemment selon la destination du document (en premier argument). La fonction anonyme (**callback**) appelée à l'issue du traitement apporte de nombreuses précisions : erreur (err), en-tête (hdr) et réponse (bdy). Le premier paramètre de toutes les fonctions de **callback** est usuellement le compte rendu d'erreur.

couchins.js

```
var ano=require('nano')('http://user:pass@localhost:5984');
var db=nano.use('directory');
db.insert({
  LastName: 'Daviolo', FirstName: 'Nancy',
  TitleOfCourtesy: 'Ms.', BirthDate: '12/08/1948',
  Address: '507 - 20th Ave. E. Apt. 2A', City: 'Seattle',
  Region: 'WA', PostalCode: '98122', Country: 'USA',
  HomePhone: '(206) 555-9857' },
function(err,bdy,hdr){
  console.log(err);
  console.log(bdy);
  console.log(hdr);
});
```

Lorsque l'identifiant univoque du document n'est pas précisé, CouchDB en génère un automatiquement. La réponse comporte trois indications : l'indication de succès (ok:true) ou d'échec, l'identifiant univoque (id), l'indice de révision (rev). L'exécution peut être répétée à loisir puisque chaque nouveau document est doté d'un identifiant propre : tous différents mais associés à un contenu identique.



```
$nano couchins.js
null
{ ok: true,
  id: '210d5b439cdb8d3cf2d4161905007eb7',
  rev: '1-5ce7c44364d08a1ac001cde1237fa030' }
{ location:
  'http://localhost:5984/directory/210d5b439cdb8d3cf2d416190500
  7eb7',
    date: 'Sat, 05 May 2012 10:47:56 GMT',
    'content-type': 'application/json',
    'cache-control': 'must-revalidate',
    'status-code': 201,
    uri: 'http://user:pass@localhost:5984/directory' }
```

Le modèle conceptuel mis en place peut nécessiter une identification prédéfinie du document (car elle est signifiante et ne sert pas seulement à l'accès au document). La propriété **_id** est alors dotée de la valeur requise (ex. Employee_1 <=> Employé n° 1).

```
var nano=require('nano')('http://user:pass@localhost:5984');
var db=nano.use('directory'); db.insert({
  _id: 'Employee_1',
  LastName: 'Daviolo',
  FirstName: 'Nancy',
  TitleOfCourtesy: 'Ms.',
  BirthDate: '12/08/1948',
  Address: '507 - 20th Ave. E. Apt. 2A',
  City: 'Seattle',
  Region: 'WA',
  PostalCode: '98122',
  Country: 'USA',
  HomePhone: '(206) 555-9857'
},function(err,bdy,hdr){
  if (err) throw err;
  console.log(bdy);
  console.log('Insert: ok')
});
```

Lors de l'exécution, les valeurs retournées dans la réponse sont toujours les mêmes : statut de l'action, identifiant univoque et indice de révision.

```
{ ok: true,
  id: 'Employee_1',
  rev: '1-5ce7c44364d08a1ac001cde1237fa030' }
Insert: ok
```

L'identifiant, étant par définition, univoque, tout nouvel appel provoque une erreur : le document soumis existe déjà. La seule façon de soumettre un document doté d'un même identifiant et de préciser la révision du document existant. Il ne s'agit alors plus de la création d'un nouveau document mais de la mise à jour du document existant.

```
/Users/jihem/WebstormProjects/instnano/cchdbins.js:16
if (err) throw err;
          ^
Error: Document update conflict.
    at gen_err (/usr/local/lib/node_modules/nano/error.js:94:43)
...
```

Consultation, Modification, Suppression...

La mise à jour d'une propriété d'un document existant nécessite la récupération du document à modifier (d'après sont identifiant univoque). Cette opération permet de manipuler l'objet reçu et de connaître son indice de révision pour le soumettre suite à sa modification. La mécanique lecture, modification, écriture avec la gestion de l'indice de révision permet d'assurer la cohérence des informations. Si la mise à jour communiquée ne comporte pas le bon indice de révision, elle est refusée : une action a été effectuée entre la lecture et la nouvelle soumission. Le document a été modifié par une application tierce. CouchDB propose un second mécanisme de mise à jour coté serveur où un traitement (défini par l'utilisateur) est invoqué avec des valeurs : ce dernier effectue sur le serveur le cycle lecture, modification, écriture (sans interruption possible). Ce procédé est abordé ultérieurement avec les vues. L'exemple ajoute une propriété (Statut avec la valeur Single) au document **Employee_1**.

couchupd.js

```
var nano=require('nano')('http://user:pass@localhost:5984');
var db=nano.use('directory');
db.get('Employee_1',function(err,dta){
  if (err) throw err;
  console.log('_rev -> '+dta._rev);
  dta['Status']='Single';
  db.insert(dta,dta._id,function(err,bdy){
    if (err) throw err;
    console.log(bdy);
    console.log('Update: ok');
  });
});
```

L'exécution met en évidence l'évolution de l'indice de version **_rev** suite à la lecture et en réponse à la mise à jour (insert). L'appel répété met en évidence l'incrémentation des versions : 1_... => 2_..., 2_...=> 3_..., etc.



```
$nano couchupd.js
_rev -> 2-1de7a69bf9b9fd72e065110f90b761c1
{ ok: true, id:
  'Employee_1',
  rev: '3-e9afc9e3b6aaafd21a6759c479658964' }
Update: ok
```

La destruction d'un document revient à le marquer comme supprimé. Le retrait du document (et de ses versions antérieures au sein de la base de données) n'est effectif qu'après compactage. La méthode de suppression se nomme **destroy**. Le mécanisme est identique à la modification (lecture, modification, écriture) si ce n'est qu'aucune modification n'est opérée sur le document lu : l'appel de la méthode **destroy** se substitue à celui de la méthode **insert**.

couchdel.js

```
var nano=require('nano')('http://user:pass@localhost:5984');
var db=nano.use('directory');
db.get('Employee_1',function(err,dta){  if (err) throw
err;      console.log('_rev -> '+dta._rev);
      db.destroy(dta._id,dta._rev,function(){
if (err) throw err;
      nano.db.compact('directory',function(){
      console.log('Delete: ok');
      });
    });
  });
});
```

L'absence de suppression physique du document et de son marquage **supprimé** (qui l'élimine de fait des résultats) sont mis en évidence par la fourniture d'un nouvel indice de révision du document (correspondant à la version dotée du statut supprimé).



```
$nano couchdel.js
```

```
_rev -> 6-e9afc9e3b6aaafd21a6759c479658964 Delete:
ok
```

La suppression ne peut être opérée qu'une seule fois. Un document marqué **supprimé** ne peut pas être à nouveau marqué (puisqu'il l'est déjà).

```
/Users/jihem/WebstormProjects/instnano/cchdbdel.js:4
if (err) throw err;
```

^

Error: deleted

at gen_err (/usr/local/lib/node_modules/nano/error.js:94:43)

...

Lors des opérations de maintenance (ou suite à une sauvegarde), le compactage de la base de données supprime physique les documents qui ne sont plus en usage : supprimés ou obsolètes (remplacé par un autre document doté d'un indice de révision plus élevé).

```
var nano=require('nano')('http://user:pass@localhost:5984');
nano.db.compact('directory',function(err){
if (err) throw err;      console.log('Compact:
ok');
});
```

L'opération réussie...

Compact: ok

ou échoue (si la base n'existe pas par exemple).

```
/Users/jihem/WebstormProjects/instnano/cchdbcmp.js:3  
if (err) throw err;
```

^

```
Error: no_db_file at gen_err  
(/usr/local/lib/node_modules/nano/error.js:94:43)
```

...

La compil...

L'exemple ci-après enchaîne l'ensemble des actions détaillées : suppression de la base, création de la base, ajout d'un nouveau document, modification du document, suppression du document et compactage de la base. L'objet du rappel réside en l'emploi des méthodes **callback** (et leurs enchaînements).

```
var nano=require('nano')('http://user:pass@localhost:5984');
nano.db.destroy('directory',function(){
  nano.db.create('directory',function(){
    var db=nano.use('directory');
    db.insert({
      LastName: 'Daviolo',
      FirstName: 'Nancy',
      TitleOfCourtesy: 'Ms.',
      BirthDate: '12/08/1948',
      Address: '507 - 20th Ave. E. Apt. 2A',
      City: 'Seattle',
      Region: 'WA',
      PostalCode: '98122',
      Country: 'USA',
      HomePhone: '(206) 555-9857'
    },function(err,bdy,hdr){
      console.log(err);
      console.log(bdy);
      console.log(hdr);
      if (err) throw err;
      console.log('_id -> '+bdy.id);
      db.get(bdy.id,function(err,dta){
        if (err) throw err;
        console.log('_rev -> '+dta._rev);
        dta['Status']='Single';
        db.insert(dta,dta._id,function(err,bdy){
          console.log('_rev -> '+bdy.rev);
```

```
        db.destroy(dta._id,bdy.rev,function(){
            if (err) throw err;

            nano.db.compact('directory',function(){
                console.log('ok');
            });
        });
    });
});
});
```

Pièces jointes

L'ajout d'une pièce jointe (attachment) en général issue d'un fichier ou d'un flux (stream) constitue une modification du document (avec évolution de l'indice de révision). La méthode à utiliser est **attachment.insert**. Ses arguments sont l'identifiant du document (conteneur), la désignation de la pièce jointe, le bloc de données la représentant, son type mime (pour l'afficher dans un browser via http), l'indice de révision du document (qui doit exister) et une fonction **callback**.

```
var fs=require('fs');
var nano=require('nano')('http://user:pass@localhost:5984');
var db=nano.use('directory');

fs.readFile('Employee_1.bmp',function(err,ctt){
  if (err) throw err;
  db.get('Employee_1',function(err,dta){
    if (err) throw err;
    console.log('_rev -> '+dta._rev);
    db.attachment.insert(dta._id,'Picture',ctt, 'image/bmp',
{rev:dta._rev},function(err){
      if (err) throw err;
      console.log('Attachment: ok');
    });
  });
});
```

La photo contenue dans le fichier **Employee_1.bmp**.



Compte-rendu d'exécution

```
_rev -> 1-5ce7c44364d08a1ac001cde1237fa030
```

Attachment: ok

Et après...

La maîtrise des notions présentées et un peu de pratique sont requises avant d'aborder les vues et gestionnaires de mises à jour. Le portage d'une base de données SQL vers CouchDB illustre ces principes et en facilite l'assimilation.

SQL vers CouchDB

Les habitudes liées à la pratique de SQL constituent le principal écueil lors de l'usage d'une base de données orientée document. Le projet proposé ci-après met en évidence les singularités des deux approches. L'objectif est de migrer la base de données « **northwind** » de SQL Server vers CouchDB. Cette base servait à découvrir et à illustrer les possibilités de Microsoft SQL Server. Elle n'est plus livrée en standard avec les nouvelles versions de SQL Server (depuis 2003). Une archive (**SQL2000SampleDb.msi**) est mise à disposition par Microsoft à l'adresse suivante : <http://www.microsoft.com/download/en/confirmation.aspx?id=23654>. L'archive comporte différents scripts SQL inhérents aux différentes bases de démonstration (**northwind** et **pubs**). La mise en oeuvre requiert uniquement le fichier **instnwnd.sql** ; les autres peuvent être supprimés. Les opérations s'effectuent directement sur le fichier de script : aucune installation de SQL Server n'est requise.

Create

L'ouverture du fichier de script **instnwnd.sql** dans l'éditeur de texte de votre choix (TextWrangler, Sublime Text, Notepad++, Nano ou autre selon l'environnement) en autorise la consultation. Le script comporte trois types de sections : de la prose propre à SQL Server (non transposable en l'état), des descriptions de table (**CREATE TABLE...**) et des enregistrements à insérer (**INSERT...**).

```
CREATE TABLE "Employees" (  
    "EmployeeID" "int" IDENTITY (1, 1) NOT NULL ,  
    "LastName" nvarchar (20) NOT NULL ,  
    "FirstName" nvarchar (10) NOT NULL ,  
    "Title" nvarchar (30) NULL ,  
    "TitleOfCourtesy" nvarchar (25) NULL ,  
    "BirthDate" "datetime" NULL ,  
    "HireDate" "datetime" NULL ,  
    "Address" nvarchar (60) NULL ,  
    "City" nvarchar (15) NULL ,  
    "Region" nvarchar (15) NULL ,  
    "PostalCode" nvarchar (10) NULL ,  
    "Country" nvarchar (15) NULL ,  
    "HomePhone" nvarchar (24) NULL ,  
    "Extension" nvarchar (4) NULL ,  
    "Photo" "image" NULL ,  
    "Notes" "ntext" NULL ,  
    "ReportsTo" "int" NULL ,  
    "PhotoPath" nvarchar (255) NULL ,  
    CONSTRAINT "PK_Employees" PRIMARY KEY CLUSTERED  
    (  
        "EmployeeID"  
    ),  
    CONSTRAINT "FK_Employees_Employees" FOREIGN KEY
```

```
(  
    "ReportsTo"  
    ) REFERENCES "dbo"."Employees" (  
        "EmployeeID"  
    ),  
    CONSTRAINT "CK_Birthdate" CHECK (BirthDate < getdate())  
)
```

La description des tables explicite le type (int, nvarchar, image, ...) et les caractéristiques (identity, not null, ...) de chaque colonne « **field** ». Un petit rappel pour ceux qui n'auraient pas pratiqué SQL (ou dont la mémoire serait gravement affaiblie) : une table SQL est un tableau à double entrées (voir programme de maternelle). Chaque ligne décrit un enregistrement « **tuple** ». Les colonnes identifient chacune des propriétés. Exemple : tableau d'activité pour les différents jours de la semaine. La table **Employees** n'échappe pas à la règle. Chaque employé est décrit à l'aide de son identifiant (**EmployeeID**), son nom (**LastName**), son prénom (**FirstName**), son titre (**Title**), sa civilité (**TitleOfCourtesy**), sa date de naissance (**BirthDate**), etc. Les contraintes (**CONSTRAINT**) explicitent des conditions à remplir pour qu'un enregistrement soit valide (et puisse être enregistré dans la table). L'identifiant est univoque (**PRIMARY KEY**). En pratique, il s'agit d'un compteur auto-incrémenté géré par SQL Server ("**int**" **IDENTIT (1, 1) NO NULL**).

```
var tables={
  Employee: [
    'EmployeeID',
    'LastName',
    'FirstName',
    'Title',
    'TitleOfCourtesy',
    'BirthDate',
    'HireDate',
    'Address',
    'City',
    'Region',
    'PostalCode',
    'Country',
    'HomePhone',
    'Extension',
    'Picture',
    'Notes',
    'ReportsTo',
    'PhotoPath'
  ]
};
```

Javascript gère la définition dynamique des types. Les déclarations se résument donc aisément à la seule liste des noms de colonnes d'un enregistrement. Un **Employee** se décrit alors à l'aide des noms de ses propriétés (colonnes) référencés par le tableau du même nom (**Employee:**) au sein de la table associative **tables**. L'application du même traitement à l'ensemble des déclarations de table donne le résultat ci-après :

Node.JS – CouchDB sous Ubuntu Server

```
var tables={
  Employee: [ 'EmployeeID', 'LastName', 'FirstName', 'Title',
    'TitleOfCourtesy', 'BirthDate', 'HireDate', 'Address', 'City',
    'Region', 'PostalCode', 'Country', 'HomePhone', 'Extension',
    'Picture', 'Notes', 'ReportsTo', 'PhotoPath' ],
  Category: [ 'CategoryID', 'CategoryName', 'Description',
    'Picture' ],
  Customer: [ 'CustomerID', 'CompanyName', 'ContactName',
    'ContactTitle', 'Address', 'City', 'Region', 'PostalCode',
    'Country', 'Phone', 'Fax' ],
  Shipper: [ 'ShipperID', 'CompanyName', 'Phone' ],
  Supplier: [ 'SupplierID', 'CompanyName', 'ContactName',
    'ContactTitle', 'Address', 'City', 'Region', 'PostalCode',
    'Country', 'Phone', 'Fax', 'HomePage' ],
  Order: [ 'OrderID', 'CustomerID', 'EmployeeID', 'OrderDate',
    'RequiredDate', 'ShippedDate', 'ShipVia', 'Freight', 'ShipName',
    'ShipAddress', 'ShipCity', 'ShipRegion', 'ShipPostalCode',
    'ShipCountry' ],
  Product: [ 'ProductID', 'ProductName', 'SupplierID', 'CategoryID',
    'QuantityPerUnit', 'UnitPrice', 'UnitsInStock', 'UnitsOnOrder',
    'ReorderLevel', 'Discontinued' ],
  Order_Detail: [ 'OrderID', 'ProductID', 'UnitPrice', 'Quantity',
    'Discount' ],
  Region: [ 'RegionID', 'RegionDescription' ],
  Territory: [ 'TerritoryID', 'TerritoryDescription', 'RegionID' ],
  Employee_Territory: [ 'EmployeeID', 'TerritoryID' ]
};
```

Insert

Les tables **CustomerCustomerDemo** et **CustomerDemographics** n'ont pas été conservées car elles sont sans intérêt (vis-à-vis du projet et mises en oeuvres connexes). Les données décrivant chaque enregistrement sont extraites et enregistrées dans des fichiers distincts (un par table). La section décrivant les différentes régions géographiques (de la table **Region**) est exportée vers le fichier **Region.txt**.

```
Insert Into Region Values (1, 'Eastern')
Insert Into Region Values (2, 'Western')
Insert Into Region Values (3, 'Northern')
Insert Into Region Values (4, 'Southern')
```

Region.txt

```
1, 'Eastern'
2, 'Western'
3, 'Northern'
4, 'Southern'
```

Une fois l'ensemble des sections traitées, les fichiers Category.txt, Customer.txt, Employee.txt, Employee_Territory.rel, Order.txt, Order_Detail.rel, Product.txt, Region.txt, Sipper.txt, Supplier.txt, Territory.txt sont disponibles. Par convenance, les désignations sont au singulier. L'extension des fichiers Employee_Territory.txt et Order_Detail.txt a volontairement été modifiée en **.rel**. En effet, ces tables assurent la mise en relation d'enregistrement d'autres tables : Employee_Territory associe des territoires à un d'employé (Employee <=1.n= Territoty), Order_Detail associe

des produits à une commande en spécifiant les quantités, le prix unitaire, ... (Order $\leq 1.n =$ Product). Du fait de l'adoption du modèle document ses données seront intégrées à leur entité de référence (Employee ou Order) et n'existeront plus sous la forme d'une entité distincte indépendante.

Image

La gestion dynamique des types par Javascript permet de s'affranchir de toute contrainte inhérente à la prise en compte de ces derniers... sauf pour le type **SQL Image**. La colonne **Picture** de **Category** comporte des chaînes de caractères de la forme : '0x151C2F00020000...AD05FE'. Elles décrivent, sous la forme d'une suite d'octets notés en hexadécimal, un BLOB (**B**inary **L**arge **O**bject). Le BLOB comporte un en-tête (apparemment de taille fixe) et le contenu du fichier associé. Le type **Image**, tel qu'utilisé, inclut un fichier **BMP** préfixé par l'en-tête. Une démarche empirique s'avère à l'origine de l'algorithme de décodage utilisé. L'apparence de la chaîne '0x151C2F00020000...AD05FE' met en évidence l'emploi de la notation hexadécimale : préfixe « 0x », uniquement des caractères 0 à 9 et A à F. La première tentative s'est résumée à constituer un fichier de l'ensemble des octets. Ce dernier n'a pas pu être ouvert à partir d'un logiciel graphique : erreur à l'ouverture. L'ouverture, à partir d'un éditeur de texte, du fichier généré met en évidence l'en-tête : « !ÿÿÿÿBitmapImagePaint.PicturePBrush) ». Il s'agit clairement d'une description de fichier image de type BMP. Le logiciel Microsoft Paint a ensuite été utilisé pour produire un fichier BMP. L'en-tête de ce dernier a été consulté de la même manière (« BM... »). La recherche de l'en-tête BMP au sein du fichier précédemment généré par l'agrégation des octets met en évidence sa présence à la position 79 (80ième caractère). L'écriture dans le fichier des données à partir de cet offset permet effectivement de produire un fichier image BMP valide.

t2image.js

```
var fs=require('fs');
var image='0x151C2F00020000...AD05FE';
var plainText2Image=function(text,fileName,callback){
var buff=text && text.length>2 && text.substr(0,2)=='0x' ? new
Buffer((text.length-2)/2,encoding='binary') : null;
var indx=0;
if (buff) {
    while (text.length>0) {
        buff[indx++]=parseInt(text.substr(0,2),16);
        text=text.substr(2);
    }
    console.log(buff.toString(encoding='binary').substr(0,79));

    fs.writeFile(fileName,buff.toString(encoding='binary').substr(79),e
ncoding='binary',callback);
}
else
    return callback && callback(null);
} plainText2Image(image,'image.bmp',function(){
    console.log(new Date()+' : done');
});
```

Le programme d'exploration **t2image.js** produit, à partir d'une chaîne BLOB, une image (**image.bmp**). La taille de la chaîne, issue de l'extraction d'une ligne d'insertion comportant une image du fichier **instnwnd.sql**, est de l'ordre de 20Ko (d'où l'emploi de l'abréviation '0x151C2F00020000...AD05FE'). Après vérification de la présence du préfixe « 0x », la chaîne est convertie par paire d'octet (buff[indx++]=parseInt(text.substr(0,2),16)). La présence de la fonction **callback** n'est pas nécessaire (mais c'est un réflexe à adopter rapidement avec Node.JS). L'ajout de la date en sortie sur le compte rendu d'état (**done**) permet de s'assurer que le message affiché correspond à la dernière exécution. L'emploi des

environnements WebStorm et/ou Sublime Text 2 permet de lancer l'exécution directement à partir de l'éditeur et d'afficher la console de résultat (d'où l'intérêt de dater son contenu).



L'image résultante... très « oldies ».

northwind.js

Le programme d'importation des données comporte une première partie liée à la déclaration des modules requis (fs pour l'accès au système de fichier, nano pour l'accès au serveur CouchDB). Une base de données (vide) nommée **northwind** a préalablement été créée sur ce dernier. L'adresse **http://user:pass@localhost:5984** doit être adaptée à votre configuration (en particulier le nom et le mot de passe de l'utilisateur). L'instance **adb** créée donne accès à cette dernière. La table associative **tables**, comportant la description de toutes les tables SQL, termine la première partie du code source, dite « déclarative ».

```
var fs=require('fs');
var nano=require('nano')('http://user:pass@localhost:5984');
var adb=nano.use('northwind');

var tables={
  Employee: [ 'EmployeeID', 'LastName', 'FirstName', 'Title',
    'TitleOfCourtesy', 'BirthDate', 'HireDate', 'Address', 'City',
    'Region', 'PostalCode', 'Country', 'HomePhone', 'Extension',
    'Picture', 'Notes', 'ReportsTo', 'PhotoPath' ],
  Category: [ 'CategoryID', 'CategoryName', 'Description',
    'Picture' ],
  Customer: [ 'CustomerID', 'CompanyName', 'ContactName',
    'ContactTitle', 'Address', 'City', 'Region', 'PostalCode',
    'Country', 'Phone', 'Fax' ],
  Shipper: [ 'ShipperID', 'CompanyName', 'Phone' ],
  Supplier: [ 'SupplierID', 'CompanyName', 'ContactName',
    'ContactTitle', 'Address', 'City', 'Region', 'PostalCode',
    'Country', 'Phone', 'Fax', 'HomePage' ],
  Order: [ 'OrderID', 'CustomerID', 'EmployeeID', 'OrderDate',
    'RequiredDate', 'ShippedDate', 'ShipVia', 'Freight', 'ShipName',
    'ShipAddress', 'ShipCity', 'ShipRegion', 'ShipPostalCode',
    'ShipCountry' ],
  Product: [ 'ProductID', 'ProductName', 'SupplierID', 'CategoryID',
```

```
'QuantityPerUnit', 'UnitPrice', 'UnitsInStock', 'UnitsOnOrder',  
'ReorderLevel', 'Discontinued' ],  
  Order_Detail: [ 'OrderID', 'ProductID', 'UnitPrice', 'Quantity',  
  'Discount' ],  
  Region: [ 'RegionID', 'RegionDescription' ],  
  Territory: [ 'TerritoryID', 'TerritoryDescription', 'RegionID'  
],  
  Employee_Territory: [ 'EmployeeID', 'TerritoryID' ]  
};
```

La fonction **getFilesList** retourne (en callback) la liste des fichiers correspondants à l'expression régulière fournie (**regexp**) du répertoire indiqué (**directory**). La liste **list** des fichiers compatibles avec la sélection est initiée sous forme de l'instance d'un tableau vide []. La méthode **readDir** renvoie la liste des entrées du répertoire à la fonction lambda en argument. La fonction **loop** parcourt cette liste (de façon récursive). Lorsque la désignation de l'entrée répertoriée est compatible, l'examen du son type (via **stat**) permet (ou non) de l'ajouter à la liste (selon qu'il s'agisse d'un fichier ou d'un répertoire). L'entrée n'est pas suffisante, elle doit être préfixé du répertoire avant communication à **fs.stat**. L'emploi de `process.nextTick()` n'est pas obligatoire : l'appel de **loop** est suffisant. Lors d'itération, il est recommandé de déléguer la prochaine occurrence au prochain Tick (permet au autres tâches d'être effectuées pendant ce cycle d'exécution $\Leftrightarrow$ tick). Le modèle de programmation présenté est typique de la démarche à adopter avec Node.JS.

```
var getFilesList=function(directory,regexp,callback) {
  var list = [];
  fs.readdir(directory, function(err, files){
    var loop = function() {
      if (files.length>0) {
        file = files.shift();
        if (regexp && file.match(regexp)) {
          fs.stat(directory + '/' + file,
            function(err,stats){
              if (!err && stats.isFile())
                list.push(file);
              process.nextTick(loop);
            });
        }
        else
          process.nextTick(loop);
      }
      else
        return callback && callback(list);
    }
    process.nextTick(loop);
  });
}
```

La conversion du type image est assurée par la fonction **plainText2Image**. Elle évolue légèrement afin de retourner l'image à la fonction **callback** si aucun nom de fichier n'est précisé (filename=null).

```
var plainText2Image=function(text,fileName,callback) {
    var buff=text && text.length>2 && text.substr(0,2)=='0x'
    ?
        new Buffer((text.length-2)/2,encoding='binary') : null;
    var indx=0;
    if (buff) {
        while (text.length>0) {
            buff[indx++]=parseInt(text.substr(0,2),16);
            text=text.substr(2);
        }
        if (fileName) {
            fs.writeFile(fileName,buff.toString(encoding='binary').substr(79),
            encoding='binary',callback)
        }
        else
            return callback && callback(buff.slice(79));
    }
    else
        return callback && callback(null);
}
```

L'insertion des données au sein de la base CouchDB s'effectue par ajout d'un document suivi (ou non) de l'ajout de l'image associée en pièce jointe (**attachment**). L'ajout d'une pièce jointe nécessite l'identifiant univoque et l'indice de révision du document manipulée. Ce dernier est récupéré lors de la lecture (**adb.get**) suivant l'ajout (**adb.insert**) dans **rec._rev**. Le type MIME de l'image est 'image/bmp'. L'enchaînement de ces opérations est essentiel afin d'assurer la cohérence des documents à chaque action. Plusieurs actions peuvent être effectuées sur un même document : ajout du document, modification pour ajout de l'image, modification pour ajout d'une donnée relative (fichier **.rel**). Chaque action doit impérativement être menée à son terme avant d'initier la suivante. A défaut, lors d'un processus, l'indice de révision mémorisé peut ne plus correspondre à la réalité : document altéré par un processus connexes.

```
var insertWithPicture=function(rec,picture,callback)
{
    adb.insert(rec,rec._id,function(err){
        if (err) throw err;
        if (picture){
            adb.get(rec._id,function(err,rec){
                if (err) throw err;
                console.log(rec._id+' : '+rec._rev);
                adb.attachment.insert(rec._id,'Picture',picture,
'image/bmp', {rev:rec._rev},function(err){
                    if (err) throw err;
                    return callback && callback();
                });
            });
        }
        else
            return callback && callback();
    });
}
```

Le respect de la cohérence implique une gestion séquentielle des insertions à effectuer. La fonction **insertLine** répète le traitement ligne à ligne. Les propriétés du document à ajouter sont mémorisées dans **rec**. La variable **att** comporte les données de l'image à convertir (ou null si aucune). Le nom du fichier précise la nature du document : **document**, **document_complément**. La présence du caractère « **_** » signale des données à ajouter dans la propriété **complément** du **document**. Exemple : **Order** est un document ; **Order_Detail** est un complément à ajouter à la propriété **Detail** du document **Order** référencé. Dans l'ensemble des fichiers, quelle que soit l'entité considérée, l'identifiant univoque (...**ID**) est en première position (d'où l'usage des clefs `table.split('_')[0]+'_'+value[0]` et `table+'_'+value[0]`). La présence des identifiants univoques en première position n'est pas le fruit du hasard mais correspond aux bonnes pratiques en matière de

gestion de base de données. Manifestement, la base de démonstration (**northwind**) a été conçue dans les règles de l'art. L'usage implique qu'un enregistrement SQL puisse être identifié de façon univoque par sa première caractéristique (primary key). Dès lors qu'une clef primaire est composée d'un ensemble de propriétés, elle est exploitée par contrainte d'unicité et non d'identification univoque (même si dans de nombreux cas la confusion de ces deux aspects peut être entretenue sans aléas autres que la spécialisation des traitements gérant l'entité). Sans entrer dans un débat (qui dépasse le cadre de l'ouvrage et s'avérerait soporifique), notons seulement que l'administrateur de la base de données SQL à migrer, de part la qualité de son travail, facilite grandement la tâche des personnes appelées à manipuler celle-ci.

```
var
insertLine=function(table,value,callback){
    var rec={};
    var att=null;
    for (var i in tables[table]) {
        if (tables[table][i].match(/Picture/)) {
            att=value[i];
        }
        else
            if (i>0)
                rec[tables[table][i]]=value[i];
    }

    if (table.indexOf('_')>0) {
        adb.get(table.split('_')[0]+'_'+value[0],function(err,upd){
            if (err) throw err;
            var
            rel=upd[table.split('_')[1]];

```

```
        if (typeof(rel)=='undefined') {
            rel=[];
        }
        rel.push(rec);
        upd[table.split('_')[1]]=rel;
        adb.insert(upd,upd._id,function(err){
            if (err) throw err;
            return callback && callback();
        });
    });
}
else {
    rec['_id']=table+'_'+value[0];
    console.log(rec['_id']);
    rec['type']=table;
    rec[tables[table][0]]=value[0];
    if (att)
    {
        plainText2Image(att,null,function(picture){
            insertWithPicture(rec,picture,callback);
        });
    }
    else
        insertWithPicture(rec,att,callback);
}
}
```

La récupération de la liste des fichiers à traiter, leurs lectures et le découpage ligne à ligne pour l'itération du traitement d'insertion **inserLine** est effectuée par **insertFile**. La fonction requiert l'expression régulière filtrant la liste des fichiers du répertoire courant et la méthode **callback** (à appeler à l'issue du traitement).

```
var insertFile=function(regex,callback){
    getFilesList('.',regex,function(files){
        var eachFile = function() {
            if (files.length>0) {
                file = files.shift();
```

```
fs.readFile(file, function(err, data) {
  if (err) throw err;

  console.log(new Date().getTime() +
    '->' + file);
  var lines = data.toString().split('\n');
  var eachLine = function() {
    if (lines.length > 0) {
      var line = lines.shift();
      insertLine(file.substr(0,
        file.lastIndexOf('.')),
        eval('[' + line + ']'),
        eachLine)
    }
    else
      process.nextTick(eachFile);
  }
  process.nextTick(eachLine);
});
}
else
  return callback && callback();
}
process.nextTick(eachFile);
});
}
```

Le corps du programme principal se limite à 2 appels, imbriqués via **callback**, de **insertFile**. Le premier traite tous les fichiers relatifs aux documents (avec ou sans image) doté de l'extension **.txt**. Le second intègre, au sein des documents précédemment intégrés, les compléments (fichier **rel**).

```
insertFile(/\.txt$/, function() {
  insertFile(/\.rel$/, null);
});
```

En l'état le dossier du projet comporte les fichiers : `Category.txt`, `Customer.txt`, `Employee.txt`, `Employee_Territory.rel`, `Order.txt`, `Order_Detail.rel`, `Product.txt`, `Region.txt`, `Shipper.txt`, `Supplier.txt`, `Territory.txt` et `northwind.js`. Le lancement du programme par **node northwind.js** ajoute les données dans la base **northwind** (à créer préalablement). La consultation des données s'effectue à l'aide de l'interface **Futon** (http://localhost:5984/_utils) en sélectionnant la base **northwind**. La consultation de **Order_10248** met en évidence l'inclusion des données complémentaires issue de **Detail.rel**. L'indice de révision 4-

`f7c23f94bd57f272541cbbf77e006beb` explicite le nombre d'actions requises à la constitution du document : 4 (1 ligne de **Order.txt** et 3 lignes de **Detail.rel**).

```
{
  "_id": "Order_10248",
  "_rev": "4-f7c23f94bd57f272541cbbf77e006beb",
  "CustomerID": "VINET",
  "EmployeeID": 5,
  "OrderDate": "7/4/1996",
  "RequiredDate": "8/1/1996",
  "ShippedDate": "7/16/1996",
  "ShipVia": 3,
  "Freight": 32.38,
  "ShipName": "Vins et alcools Chevalier",
  "ShipAddress": "59 rue de l'Abbaye",
  "ShipCity": "Reims",
  "ShipRegion": null,
  "ShipPostalCode": "51100",
  "ShipCountry": "France",
  "type": "Order",
  "OrderID": 10248,
  "Detail": [
    {
      "ProductID": 11,
```

```
        "UnitPrice": 14,  
        "Quantity": 12,  
        "Discount": 0  
    },  
    {  
        "ProductID": 42,  
        "UnitPrice": 9.8,  
        "Quantity": 10,  
        "Discount": 0  
    },  
    {  
        "ProductID": 72,  
        "UnitPrice": 34.8,  
        "Quantity": 5,  
        "Discount": 0  
    }  
]  
}
```

L'indice de révision de **Category_1** (2-e791c05f27c5a6988859f630d3f5d9f4) est lié aux deux actions requises à l'ajout du document (doté d'une image en pièce jointe, voir **_attachments**).

```
{  
  "_id": "Category_1",  
  "_rev": "2-e791c05f27c5a6988859f630d3f5d9f4",  
  "CategoryName": "Beverages",  
  "Description": "Soft drinks, coffees, teas, beers, and ales",  
  "type": "Category",  
  "CategoryID": 1,  
  "_attachments": {  
    "Picture": {  
      "content_type": "image/bmp",  
      "revpos": 2,  
      "digest": "md5-e/iungK6D6oAPWflb6hDcA==",  
      "length": 10667,  
      "stub": true  
    }  
  }  
}
```

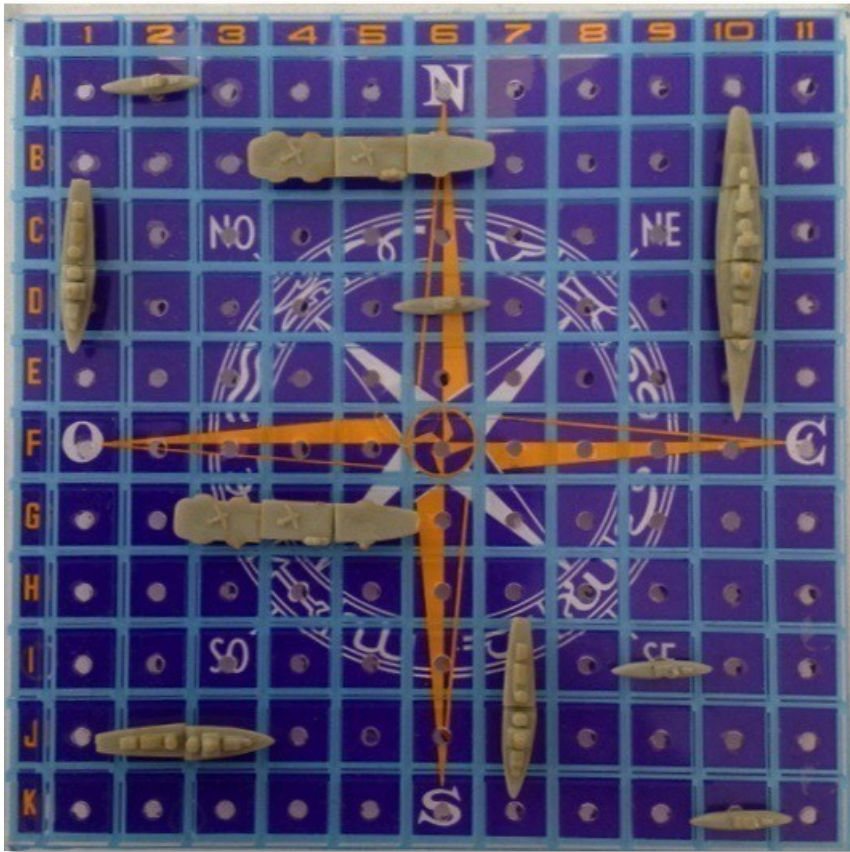
```
}
```

Futon permet de naviguer dans les différentes versions d'un document afin de consulter les révisions précédentes. Le processus d'ajout est alors clairement explicité dans les deux cas présentés (**Order_10248** et **Category_1**). Ces documents sont respectivement directement accessibles aux adresses : http://127.0.0.1:5984/northwind/Order_10248, http://127.0.0.1:5984/northwind/Category_1. L'image de la catégorie peut aussi être consultée : http://127.0.0.1:5984/northwind/Category_1/Picture.

Views

L'accès aux données d'une base de données orientée document ne se limite pas à la seule récupération d'un document à l'aide de son identifiant univoque. Les requêtes basées sur les caractéristiques des documents sont également prises en charge. L'agrégation des caractéristiques discriminantes constitue la clef d'une vue (view) listant les entités compatibles avec la recherche : document ou portion de document. La base de données utilisée pour illustrer ces notions décrit le plateau d'un jeu de bataille navale. Le plateau est découpé en 11 lignes (de A à K) de 11 colonnes (de 1 à B, 11 en hexadécimal). Les 121 cases sont identifiées de façon univoque par leur emplacement : A2, B4, etc. Les navires sont disposés horizontalement ou verticalement sur le plateau. Ils occupent de 1 à 4 cases selon leur type : sous-marins S1, S2, S3 et S4 (1), croiseurs C1, C2 et C3 (2), porte-avions A1 et A2 (3), cuirassé B1 (4).

Views, Map & Reduce



Le programme node.js **shipinit.js** crée la base de données **ship** et insère les documents décrivant chaque navire : identification univoque (**_id**), type (**type**) et cases occupées (**location**). La fonction **dbShipInit** détruit la base **ship** puis la crée avec les données fournies en argument (**aoShip**). La fonction **callback** est appelée

à l'issue du traitement. Le serveur CouchDB doit préalablement être activé sur le poste local. L'URL **http://user:pass@localhost:5984** est à adapter aux conditions réelles d'usage : emploi d'un système distant ou local, actualisation des identifiants de connexion, etc.

shipinit.js

```
var nano=require('nano')('http://user:pass@localhost:5984');
var dbShipInit=function(aoShip,callback) {
    nano.db.destroy('ship',function(){
        nano.db.create('ship',function(err){
            if (err) return callback && callback(err);
            var db=nano.use('ship');
            var shipInsert=function(){
                if (aoShip.length>0) {
                    loShip=aoShip.pop();
                    console.log(loShip);
                    db.insert(loShip,function(err){
                        if (err) return callback
                        callback(err);
                        shipInsert();
                    });
                }
                else
                    return callback && callback();
            };
            shipInsert(aoShip);
        });
    });
};
```

Node.JS – CouchDB sous Ubuntu Server

```
};  
// 4 cuirassé - battleship  
// 3 porte avion - aircraft carrier  
// 2 croiseur - cruiser  
// 1 sous-marin -  
submarine dbShipInit([  
  { _id:'S1', type:'Submarine', location:[ 'A2' ]},  
  { _id:'S2', type:'Submarine', location:[ 'D6' ]},  
  { _id:'S3', type:'Submarine', location:[ 'I9' ]},  
  { _id:'S4', type:'Submarine', location:[ 'KA' ]},  
  { _id:'C1', type:'Cruiser', location:[ 'C1','D1' ]},  
  { _id:'C2', type:'Cruiser', location:[ 'I7','J7' ]},  
  { _id:'C3', type:'Cruiser', location:[ 'J2','J3' ]},  
  { _id:'A1', type:'Aircraft Carrier', location:[ 'B4','B5','B6' ]},  
  { _id:'A2', type:'Aircraft Carrier', location:[ 'G3','G4','G5' ]},  
  { _id:'B2', type:'Battleship', location:[ 'BA','CA','DA','EA']}  
],function(err) {  
  if (err) throw err;  
  console.log('Create:  
Ok')  
});
```

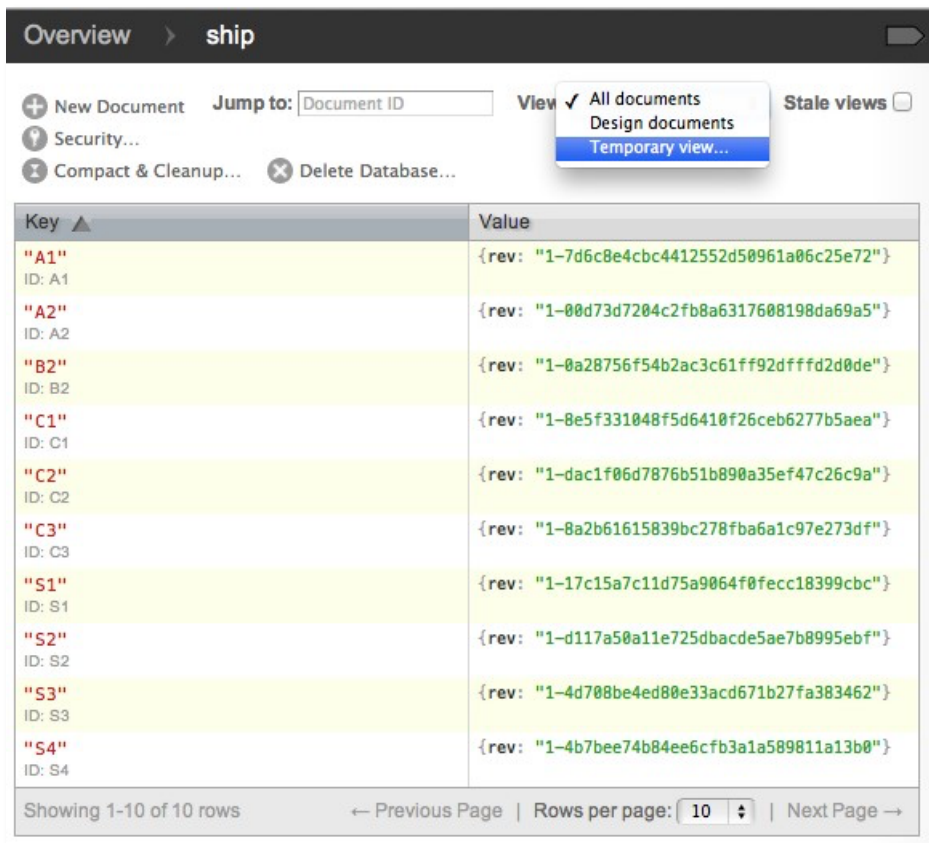
En cas de succès, l'exécution en ligne de commande affiche le compte rendu **Create: Ok**.



\$node shipinit.js

Create: Ok

L'accès à **Futon** (http://localhost:5984/_utils dans le cas de l'exemple) référence la nouvelle base de données dans l'aperçu (**Overview**). Un clic sur le nom de la base de données (**ship**) permet d'en consulter les documents. Les informations sousmises sont complétées par l'indice de révision (**rev**). La clef (**Key**) comporte l'identifiant univoque de chaque navire (A1 à S4). Le document associé est celui soumis par le programme. La consultation (**View**) porte par défaut sur tous les documents (**All documents**). Le mode **Temporary view...** donne accès à l'écran de création des vues temporaires. Une vue est décrite à l'aide de deux fonctions javascript : Map, Reduce. La première est obligatoire. Elle cartographie les éléments à extraire des documents de la base de données et la nature de leur restitution : composant de la clef (**Key**) ou de valeur (**Value**). La seconde, Reduce, est facultative. Elle manipule le résultat de la fonction Map pour produire un décompte, une somme, une synthèse ou l'enrichir.



Overview > ship

+ New Document Jump to: View ☒ All documents Design documents Temporary view... Stale views ☐

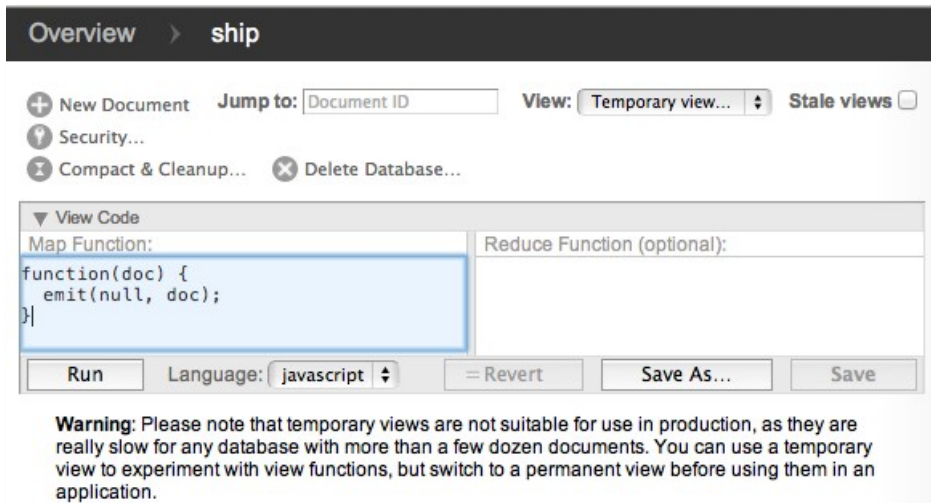
🔒 Security... 🗑️ Compact & Cleanup... ✖ Delete Database...

Key ▲	Value
"A1" ID: A1	{rev: "1-7d6c8e4cbc4412552d50961a06c25e72"}
"A2" ID: A2	{rev: "1-00d73d7204c2fb8a6317608198da69a5"}
"B2" ID: B2	{rev: "1-0a28756f54b2ac3c61ff92dffd2d0de"}
"C1" ID: C1	{rev: "1-8e5f331048f5d6410f26ceb6277b5aea"}
"C2" ID: C2	{rev: "1-dac1f06d7876b51b890a35ef47c26c9a"}
"C3" ID: C3	{rev: "1-8a2b61615839bc278fba6a1c97e273df"}
"S1" ID: S1	{rev: "1-17c15a7c11d75a9064f0fecc18399cbc"}
"S2" ID: S2	{rev: "1-d117a50a11e725dbacde5ae7b8995ebf"}
"S3" ID: S3	{rev: "1-4d708be4ed80e33acd671b27fa383462"}
"S4" ID: S4	{rev: "1-4b7bee74b84ee6cfb3a1a589811a13b0"}

Showing 1-10 of 10 rows ← Previous Page Rows per page: 10 | Next Page →

La fonction **Map** est pré-remplie avec une fonction par défaut. Elle retourne l'ensemble des documents associés à clef **null**. La fonction anonyme accepte en paramètre (**doc**) le document à manipuler. Elle est appelée pour chaque document de la base de données. Le résultat est produit par la méthode **emit**. Ses arguments sont la clef (**Key**) et la valeur associée (**Value**) pour le document considéré. L'appel à **emit** peut être conditionné : tous les documents ne complètent alors pas forcément le résultat. L'appel à **emit** peut être itéré : un document peut ainsi

produire plusieurs couples (clef, valeur) au sein du résultat. Le résultat est toujours le cumul de l'ensemble des itérations de la fonction **Map** (appliquée à chaque document de la base de données).



Le bouton **run** permet d'exécuter la vue et d'en consulter le résultat. La fonction **Map** par défaut, appliquée à la base **ship**, retourne le résultat ci-après. L'appel pour chaque document de la fonction a produit une ligne de résultat : **Key** : null, **Value** : document en l'état (avec indice de révision). L'affichage précise l'identifiant univoque du document à l'origine des données dans la colonne **Key** (ex. ID: A1).

Key ▲	Value
null ID: A1	{_id: "A1", _rev: "1-7d6c8e4cbc4412552d50961a06c25e72", type: "Aircraft Carrier", location: ["B4", "B5", "B6"]}
null ID: A2	{_id: "A2", _rev: "1-00d73d7204c2fb8a6317608198da69a5", type: "Aircraft Carrier", location: ["G3", "G4", "G5"]}
null ID: B2	{_id: "B2", _rev: "1-0a28756f54b2ac3c61ff92dffffd2d0de", type: "battleship", location: ["BA", "CA", "DA", "EA"]}
null ID: C1	{_id: "C1", _rev: "1-8e5f331048f5d6410f26ceb6277b5aea", type: "Cruiser", location: ["C1", "D1"]}
null ID: C2	{_id: "C2", _rev: "1-dac1f06d7876b51b890a35ef47c26c9a", type: "Cruiser", location: ["I7", "J7"]}
null ID: C3	{_id: "C3", _rev: "1-8a2b61615839bc278fba6a1c97e273df", type: "Cruiser", location: ["J2", "J3"]}
null ID: S1	{_id: "S1", _rev: "1-17c15a7c11d75a9064f0fecc18399cbc", type: "Submarine", location: ["A2"]}
null ID: S2	{_id: "S2", _rev: "1-d117a50a11e725dbacde5ae7b8995ebf", type: "Submarine", location: ["D6"]}
null ID: S3	{_id: "S3", _rev: "1-4d708be4ed80e33acd671b27fa383462", type: "Submarine", location: ["I9"]}
null ID: S4	{_id: "S4", _rev: "1-4b7bee74b84ee6cfb3a1a589811a13b0", type: "Submarine", location: ["KA"]}
Showing 1-10 of 10 rows	
← Previous Page Rows per page: 10 Next Page →	

La liste des cases occupées et du navire associé s'obtient facilement en parcourant pour chaque document la liste des emplacements. Chaque emplacement (**doc.location[idx]**) constitue une clef à laquelle est rattaché l'identifiant univoque du navire occupant la case (**doc._id**).

Map

Liste des emplacements utilisés (avec indication de l'occupant)

Map:

```
function(doc) {  
  for (var idx in doc.location) {  
    emit(doc.location[idx], doc._id);  
  }  
}
```

Le résultat peut être trié par ordre de clef de façon ascendante (par défaut) ou descendante.

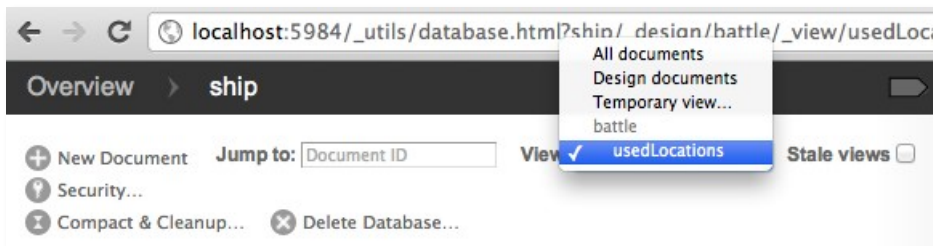
Key ▲	Value
"A2" ID: S1	"S1"
"B4" ID: A1	"A1"
"B5" ID: A1	"A1"
"B6" ID: A1	"A1"
"BA" ID: B2	"B2"
"C1" ID: C1	"C1"
"J2" ID: C3	"C3"
"J3" ID: C3	"C3"
"J7" ID: C2	"C2"
"KA" ID: S4	"S4"

Showing 1-20 of 20 rows ← Previous Page Rows per page: 100 Next Page →

La vue réalisée (**usedLocations**) peut être enregistrée dans un document de conception (**_design/battle**). Un même document de conception peut contenir une multitude de vues (dotées de noms distincts). Comme les autres documents de la base de données, il est associé à un identifiant univoque (son nom) et un indice de révision.



À la suite de l'enregistrement, le menu **View** référence le document de conception et les vues associées. La sélection s'applique immédiatement et le résultat correspondant est affiché dans la fenêtre.



Des conditions peuvent être appliquées au sein de la fonction **Map**. La fonction présentée peut très facilement évoluer pour ne conserver que les emplacements

occupés par les croiseurs. Le test peut être effectué en sélectionnant à nouveau la vue **Temporary View**. Cet écran propose toujours la dernière saisie opérée. L'action se limite à ajouter la ligne **if (doc.type=='Cruiser')** et à presser le bouton **run** (nul besoin d'enregistrer la vue pour en obtenir le résultat).

Map:

```
function(doc) { if
  (doc.type=='Cruiser') for (var idx
in doc.location) {
  emit(doc.location[idx], doc._id);
  }
}

{
  { Key: "C1", Value: "C1" },
  { Key: "D1", Value: "C1" },
  { Key: "I7", Value: "C2" },
  { Key: "J2", Value: "C3" },
  { Key: "J3", Value: "C3" },
  { Key: "J7", Value: "C2" }
}
```

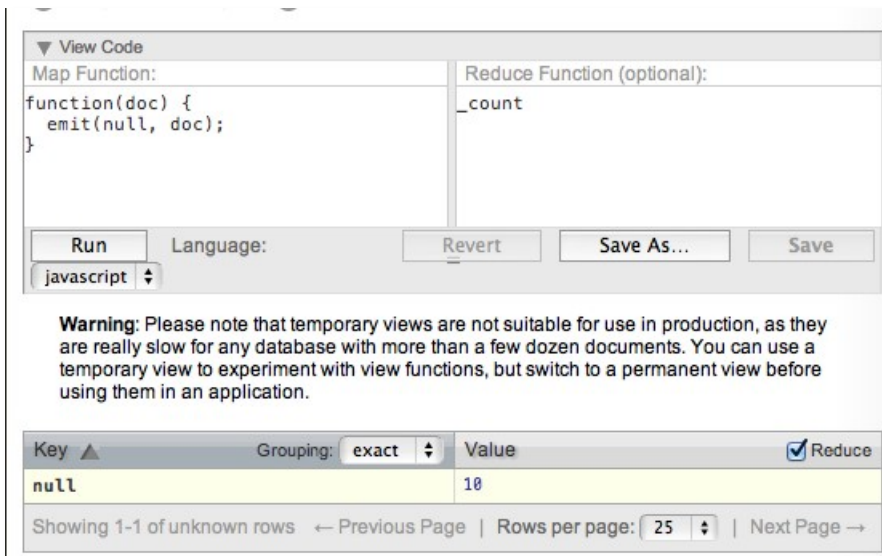
La condition peut à nouveau être modifiée pour ne lister que les cases occupées par un navire requérant au moins deux emplacements : tout sauf les sous-marins. En pratique, la fonction **Map** n'a que 2 limites : son temps d'exécution doit être raisonnable (car elle est appliquée à tous les documents de la base), javascript doit être employé [est-ce vraiment une limite ?].

Map:

```
function(doc) {  
  if (doc.location.length>=2)  
    for (var idx in doc.location) {  
      emit(doc.location[idx], { _id:doc._id, type:doc.type });  
    }  
}  
  
{  
  {Key:"B4", Value:{_id:"A1", type:"Aircraft Carrier"}},  
  {Key:"B5", Value:{_id:"A1", type:"Aircraft Carrier"}},  
  {Key:"B6", Value:{_id:"A1", type:"Aircraft Carrier"}},  
  {Key:"BA", Value:{_id:"B2", type:"Battleship"}},  
  {Key:"C1", Value:{_id:"C1", type:"Cruiser"}},  
  {Key:"CA", Value:{_id:"B2", type:"Battleship"}},  
  {Key:"D1", Value:{_id:"C1", type:"Cruiser"}},  
  {Key:"DA", Value:{_id:"B2", type:"Battleship"}},  
  {Key:"EA", Value:{_id:"B2", type:"Battleship"}},  
  {Key:"G3", Value:{_id:"A2", type:"Aircraft Carrier"}},  
  {Key:"G3", Value:{_id:"A2", type:"Aircraft Carrier"}},  
  {Key:"G3", Value:{_id:"A2", type:"Aircraft Carrier"}},  
  {Key:"G3", Value:{_id:"C2", type:"Cruiser"}},  
  {Key:"G3", Value:{_id:"C3", type:"Cruiser"}},  
  {Key:"G3", Value:{_id:"C3", type:"Cruiser"}},  
  {Key:"G3", Value:{_id:"C2", type:"Cruiser"}},  
}
```

Reduce

La production de résultats à l'aide de la fonction **Map** offre de nombreuses possibilités. La fonction **Reduce** permet d'aller encore plus loin en manipulant les résultats issus de la fonction **Map**. Cet enchaînement évite d'avoir à transmettre d'important flux de données à destination de l'application utilisatrice pour des opérations élémentaires : comptage, somme, etc.



The screenshot shows the CouchDB MapReduce interface. At the top, there's a 'View Code' section with two text areas: 'Map Function:' and 'Reduce Function (optional):'. The Map Function contains the code:

```
function(doc) {  
  emit(null, doc);  
}
```

. The Reduce Function contains the code:

```
_count
```

. Below these are buttons for 'Run', 'Language:' (set to 'javascript'), 'Revert', 'Save As...', and 'Save'. A warning message states: 'Warning: Please note that temporary views are not suitable for use in production, as they are really slow for any database with more than a few dozen documents. You can use a temporary view to experiment with view functions, but switch to a permanent view before using them in an application.' Below the warning is a table with columns 'Key', 'Grouping:', 'Value', and a checkbox for 'Reduce'. The table shows one row with 'null' as the key and '10' as the value. The 'Reduce' checkbox is checked. At the bottom, it says 'Showing 1-1 of unknown rows', '← Previous Page', 'Rows per page: 25', and 'Next Page →'.

Key	Grouping:	Value	Reduce
null	exact	10	☒

Les fonctions **Map**, **Reduce** ci-après indiquent le nombre de bateaux. La case à cocher **Reduce** de l'en-tête de colonne **Value** peut être décochée pour inhiber l'appel à la fonction **Reduce**. Les données brutes (présentées en argument de la fonction **Reduce**) sont alors consultables. Pour mémoire, la fonction **Map** s'applique à chaque document. La fonction **Reduce** s'applique à chaque groupe (**Grouping**) de clefs identiques (**exact**).

Map:

```
function(doc) {  
    emit(null, doc);  
}
```

Reduce:

```
_count
```

{ Key:null, Value:10 }

Le même principe donne le nombre d’emplacements occupés...

Map:

```
function(doc) {  
    for (var idx in doc.location) {  
        emit(null, doc.location[idx]);  
    }  
}
```

Reduce:

```
_count
```

{ Key:null, Value:20 }

Ou le nombre d’emplacements par ligne.

Map:

```
function(doc) {  
    for (var idx in doc.location) {  
        emit(doc.location[idx][0], doc.location[idx]);  
    }  
}
```

```
}
```

Reduce:

```
_Count
```

```
{  
  {Key: "A", Value: 1},  
  {Key: "B", Value: 4},  
  {Key: "C", Value: 2},  
  {Key: "D", Value: 3},  
  {Key: "E", Value: 1},  
  {Key: "G", Value: 3},  
  {Key: "I", Value: 2},  
  {Key: "J", Value: 3},  
  {Key: "K", Value: 1}  
}
```

Le changement du **Grouping** de **exact** à **none** applique la fonction **Reduce** à l'ensemble (sans distinction de groupe). La valeur correspond alors à la somme du nombre de cases occupées pour chaque ligne = nombre total de cases occupées. L'opération de regroupement de groupe correspond à un **rereduce** (alors à vrai).

```
{Key: null, Value: 20}
```

La fonction interne **_count** peut donc s'exprimer la façon suivante :

Map:

```
function(keys, values, rereduce)
{
  if (rereduce) {
    return
    sum(values);
  }
  else {
    return
    values.length;
  }
}
```

La clef (**Key**) est utilisée comme critère de regroupement. La détermination du nombre d'emplacements par type de bateau implique de constituer la clef sur le type. La modification de la fonction **Map** donne immédiatement le résultat attendu.

Map:

```
function(doc) {  
  for (var idx in doc.location) {  
    emit(doc.type, doc.location[idx]);  
  }  
}
```

Reduce:

```
_count  
  
{  
  {Key:"Aircraft Carrier", Value:6},  
  {Key:"Battleship", Value:4},  
  {Key:"Cruiser", Value:6},  
  {Key:"Submarine", Value:4}  
}
```

La clef n'est pas obligatoirement une valeur atomique. Pour proposer différent niveau de groupement, il suffit de renvoyer un tableau de valeur pour clef. La fonction **Map** proposée constitue une clef dotée de deux niveaux : le premier est la ligne (A-K), le second la colonne (1-B).

Map:

```
function(doc) {  
  for (var idx in doc.location) {
```

```
emit (
    [doc.location[idx][0], doc.location[idx][1]],
    { type:doc.type, name:doc._id }
);
}
}
Reduce:
_count
```

Lorsque le **Grouping** est **exact** (ou **level2**), l'ensemble de la clef est pris en compte (soit les 2 premiers éléments du tableau). La valeur (**Value**) est toujours de 1. Une seule case correspond à un emplacement défini par un indice de ligne et un indice de colonne. Seules les cases occupées sont comptabilisées (les autres ne sont pas répertoriées).

```
{
  {Key: ["A", "2"], Value: 1},
  {Key: ["B", "4"], Value: 1},
  ...
  {Key: ["K", "A"], Value: 1}
}
```

Lorsque le niveau de groupe est abaissé à 1 (**Groupe level1**), seule la première caractéristique est prise en compte pour itérer la fonction **Reduce**. Cette dernière est donc appelée une seule fois pour chaque ligne avec l'ensemble des valeurs associées (liste des cases de colonnes distinctes de cette même ligne), d'où les résultats : 1 case pour la ligne A, 4 cases pour la ligne B, etc. Les valeurs affichées correspondent aux nombres d'emplacements par ligne déjà évoqués. Seul le format de la clef a changé : il s'agit maintenant d'un tableau de chaîne (en l'occurrence une seule dotée de l'indice de la ligne).

level1

```
{  
  {Key: ["A"], Value: 1},  
  {Key: ["B"], Value: 4},  
  ...  
  {Key: ["K"], Value: 1}  
}
```

Le passage du **Grouping** à **none** indique le nombre total de cases occupées (qui s'avère identique à celui déjà calculé, soit : 20).

none

```
{Key: null, Value: 20}
```

La fonction **Reduce** ne se limite pas à compter les résultats (**_count**) ou sommer leurs valeurs (**_sum**). L'exemple suivant détermine la liste des croiseurs qui sont sur la même ligne qu'un sous-marin. La fonction **Map** produit, à partir des documents dont le type est **Cruiser** ou **Submarine**, une liste. Pour chaque emplacement occupé, elle associe l'indice de ligne à l'identifiant du navire (avec précision de type).

Map:

```
function(doc) {  
  if (!!~['Cruiser','Submarine'].indexOf(doc.type))  
    for (var idx in doc.location)  
      emit(doc.location[idx][0], { _id:doc._id,type:doc.type });  
}
```

Pour chaque ligne, la fonction **Reduce** collecte le nom et le type de tous les croiseurs et sous-marin présents. La table de hachage **ship** est utilisée pour référencer les représentants par catégorie. Lorsque chaque catégorie est pourvue, la liste des croiseurs est renvoyée. La fonction accepte pour argument les clefs et valeurs (sous forme de tableaux) des entités présentent dans le résultat pour le groupement (de clefs) considéré (avec **true** ou sans **false** rereduce).

Reduce:

```
function(keys, values, rereduce) {  
  var ship={Cruiser:[],Submarine:[]};  
  for (var idx in values)  
    ship[values[idx].type].push(values[idx]._id);  
}
```

```
    if (ship.Cruiser.length>0 &&
ship.Submarine.length>0) {
        return ship.Cruiser;
    }
    else
        return
null;
}
```

Le résultat indique pour chaque groupement considéré (ligne occupée par au moins un croiseur et/ou un sous-marin) l'identifiant des croiseurs compatibles avec le critère (avoir un sous-marin sur la même ligne). Une valeur à **null** correspond à une ligne disposant soit d'un croiseur, soit d'un sous-marin mais pas des deux. Les identifiants **C1** et **C2** correspondent aux croiseurs recherchés, respectivement en ligne **D** et **I**.

```
{
  {Key: "A", Value: null},
  {Key: "C", Value: null},
  {Key: "D", Value: ["C1"]},
  {Key: "I", Value: ["C2"]},
  {Key: "J", Value: null},
  {Key: "K", Value: null}
}
```

La modification du **Grouping** de **exact** à **none** renvoie deux fois l'identifiant de tous les croiseurs (et non seulement **C1** et **C2**). L'absence de prise en compte du regroupement par ligne induit un seul appel à la fonction **Reduce** avec tous les résultats. L'ensemble parcouru par la fonction ne correspond plus à une ligne mais à l'ensemble des lignes. La table de hachage **ship** comporte donc l'ensemble des croiseurs et sous-marins présents (d'où la présence de **C3**). La répétition des identifiants est liée au nombre de cases occupées par chaque croiseur (2).

```
{Key:null, ["C3", "C3", "C2", "C2", "C1", "C1"]}
```

Pour obtenir le résultat souhaité (**C1**, **C2**), la fonction **Reduce** doit évoluer afin de gérer une table de hachage par ligne (et non une l'ensemble des valeurs en argument). L'indication de la ligne est extraite de la liste des clefs (**keys**) des éléments communiqués. Le parcours des valeurs de ces éléments permet de référencer les navires par ligne et par type (dans **ship**). Le résultat (**result**) est constitué ligne après ligne en s'assurant que l'identifiant du croiseur trouvé n'est pas déjà référencé. Les résultats issus de la fonction **Map** sont communiqués en deux parties distinctes : clefs (**keys**) et valeurs (**values**). L'indice fait le lien entre les clefs et les valeurs pour un même élément du résultat.

Reduce:

```
function(keys, values, rereduce)
{
    var result=[];
```

```
var ship={};

for (var idx in keys)
    ship[keys[idx][0]]={Cruiser:[],Submarine:[]};
for (var idx in values)

ship[keys[idx][0]][values[idx].type].push(values[idx]._id);
for (var idx in keys)
    if (ship[keys[idx][0]].Cruiser.length>0 &&
        ship[keys[idx][0]].Submarine.length>0) {
        for (var itm in ship[keys[idx][0]].Cruiser)
            if (!~result.indexOf(ship[keys[idx][0]].Cruiser[itm]))
                result.push(ship[keys[idx][0]].Cruiser[itm]);
    }
return
result;
}
```

Lorsque le **Grouping** est **exact**. La vue obtenue est similaire à celle produite par la fonction **Reduce** précédente.

```
{
  {Key: "A", Value: []},
  {Key: "C", Value: []},
  {Key: "D", Value: ["C1"]},
  {Key: "I", Value: ["C2"]},
  {Key: "J", Value: []},
  {Key: "K", Value: []}
}
```

Cependant l'évolution se manifeste avec la suppression de la gestion des regroupements de clefs (**Grouping none**). Seuls les identifiants des croiseurs C1 et C2 sont retournés.

```
{Key:null, ["C2", "C1"]}
```

Views et node.js

Les vues préparées à l'aide de **Futon** peuvent être appelées à partir de node.js. La méthode **view** requiert à cet effet trois arguments : le nom du document de conception (sans le prefix **_design**), le nom de la vue (regroupant les fonctions **Map** et **Reduce**), la fonction de **callback**. Le script **shipview.js** affiche la réponse (en cas de succès) sinon l'exception **err** est levée.

shipview.js

```
var
nano=require('nano')('http://user:pass@localhost:5984');
var db=nano.use('ship');
db.view('battle', 'usedLocations', function(err, data) {
  if (err) throw err;
  console.log(data);
});
```

La réponse est comporte de trois propriétés : le nombre total de lignes du résultat (**total_rows**), le décallage (**offset**) et les lignes du résultat (**rows**).

```
{ total_rows:
20,
  offset: 0,
  rows:
    [ { id: 'S1', key: 'A2', value: 'S1' },
      { id: 'A1', key: 'B4', value: 'A1' },
      { id: 'A1', key: 'B5', value: 'A1' },
```

```
{ id: 'A1', key: 'B6', value: 'A1' },
{ id: 'B2', key: 'BA', value: 'B2' },
{ id: 'C1', key: 'C1', value: 'C1' },
{ id: 'B2', key: 'CA', value: 'B2' },
{ id: 'C1', key: 'D1', value: 'C1' },
{ id: 'S2', key: 'D6', value: 'S2' },
{ id: 'B2', key: 'DA', value: 'B2' },
{ id: 'B2', key: 'EA', value: 'B2' },
{ id: 'A2', key: 'G3', value: 'A2' },
{ id: 'A2', key: 'G4', value: 'A2' },
{ id: 'A2', key: 'G5', value: 'A2' },
{ id: 'C2', key: 'I7', value: 'C2' },
{ id: 'S3', key: 'I9', value: 'S3' },
{ id: 'C3', key: 'J2', value: 'C3' },
{ id: 'C3', key: 'J3', value: 'C3' },
{ id: 'C2', key: 'J7', value: 'C2' },
{ id: 'S4', key: 'KA', value: 'S4' } ]
```

En complément des indications requises, **view** autorise la description de paramètres supplémentaires. L'ordre descendant (**descending:true**) change l'ordre de tri des lignes selon leurs clefs respectives. Le nombre de ligne du résultat peut être limité (**limit:5**). Cette indication associée à une valeur de décallage (**skip** à préciser) permet de paginer le résultat de la vue. L'étendue de l'ensemble résultat peut être conditionnée à partir des clefs **startkey** et **endkey**. Les paramètres non utilisés n'ont pas à être spécifiés. L'indication **skip:0** est inutile.

shipview.js

```
var
nano=require('nano')('http://user:pass@localhost:5984');
var db=nano.use('ship');
db.view('battle', 'usedLocations',
{
    descending: true,
    limit: 5,
    skip: 0,
    startkey: 'J7',
    endkey: 'D1'
}, function(err, data) {
    if (err) throw err;
    for (var liInd in data.rows)
        console.log(data.rows[liInd]); })
});
```

Les modifications apportées à **shipview.js** liste les emplacements situés entre D1 et J7 en ordre inverse. Seules les 5 premières lignes de cet ensemble sont retournées.

```
{ id: 'C2', key: 'J7', value: 'C2' }
{ id: 'C3', key: 'J3', value: 'C3' }
{ id: 'C3', key: 'J2', value: 'C3' }
{ id: 'S3', key: 'I9', value: 'S3' }
{ id: 'C2', key: 'I7', value: 'C2' }
```

Lorsque l'étendue ne concerne qu'une seule clef, il est possible de définir une étendue à l'aide d'un caractère unicode hors étendue **\u9999**.

```
startkey: val,  
endkey: val + "\u9999"
```

La méthode `JSON.stringify(...)` s'avère également intéressante pour transformer une réponse quelconque (au format JSON) en une chaîne de texte (à stocker ou transférer via HTTP).

Les documents de conception **_design/...** sont des documents comme les autres. Le programme de création de la base de données et d'insertion des documents peut donc prendre en charge l'ajout de la vue.

shipinit.js

...

```
dbShipInit([  
  { _id:'S1', type:'Submarine', location:[ 'A2' ]},  
  { _id:'S2', type:'Submarine', location:[ 'D6' ]},  
  { _id:'S3', type:'Submarine', location:[ 'I9' ]},  
  { _id:'S4', type:'Submarine', location:[ 'KA' ]},  
  { _id:'C1', type:'Cruiser', location:[ 'C1','D1' ]},  
  { _id:'C2', type:'Cruiser', location:[ 'I7','J7' ]},  
  { _id:'C3', type:'Cruiser', location:[ 'J2','J3' ]},  
  { _id:'A1', type:'Aircraft Carrier', location:[ 'B4','B5','B6' ]},  
  { _id:'A2', type:'Aircraft Carrier', location:[ 'G3','G4','G5' ]},  
  { _id:'B2', type:'Battleship', location:[ 'BA','CA','DA','EA' ]},  
])
```

```
{
  "_id": "_design/battle",
  "language": "javascript",
  "views": {
    "usedLocations": {
      "map": "function(doc) {\n for (var idx in
doc.location) {\n emit(doc.location[idx],doc._id);\n }\n}"
    }
  }
},function(err) {
  if (err) throw err;
  console.log('Create: Ok')
});
```

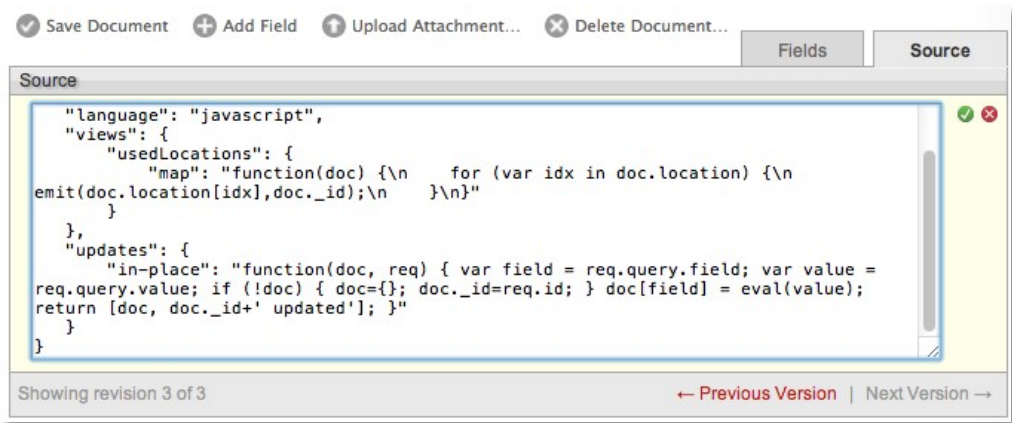
Update handler

La modification d'un document ne peut être effectuée par une application que si elle a connaissance du dernier indice de révision de ce dernier. Ce mécanisme évite les écrasements car seul un document doté de l'indice de révision correct peut se substituer au document existant. Il est doté d'un nouvel indice de révision lors de cette opération. En pratique, il s'avère souvent nécessaire de lire préalablement le document à modifier avant toutes altérations (avec le risque d'avoir à effectuer une nouvelle tentative si le document a évolué entre-temps).

CouchDB propose un mécanisme de mise à jour coté serveur : **updates** handler. La propriété **updates** d'un document de conception peut accueillir des fonctions (identifiées selon le même principe que les vues) de gestion des mises à jour. La fonction accepte deux arguments : le document à modifier (s'il a pu être retrouvé d'après l'identifiant communiqué), l'ensemble des paramètres de la requête. Lorsque aucun document n'a été trouvé, la fonction peut en créer un à la volée.

L'accès à la base de données **ship** sous **Futon** permet de sélectionner la vue (**view**) des documents de conception (**Design documents**). La liste présente le document **_design/battle** (colonne **Key**). Un clic sur son nom, puis le passage en mode **Source** (onglet en haut à droite) en autorise l'édition (cliquer sur le corps du texte). Le code relatif au gestionnaire de mises à jour (**updates**) peut être ajouté au document (JSON). Pour valider les modifications, il faut effectuer un clic sur la coche verte, puis sur l'option **Save Document**.

```
{
  "_id": "_design/battle",
  "_rev": "1-e75d3a7348f9866f4369f40328b8c0a1",
  "language": "javascript",
  "views": {
    "usedLocations": {
      "map": "function(doc) {\n for (var idx in doc.location) {\n
emit(doc.location[idx], doc._id);\n }\n}"
    }
  },
  "updates": {
    "in-place": "function(doc, req) { var field = req.query.field; var
value = req.query.value; if (!doc) { doc={}; doc._id=req.id; } doc[field]
= eval(value); return [doc, doc._id+' updated']; }"
  }
}
```



La méthode **updateWithHandler** invoque le gestionnaire de mise à jour **in-place** du document **_desing/battle**. L'identifiant du document à modifier est communiqué

('A1') puis l'ensemble des paramètres de la requête : le nom de la propriété (**field**) à modifier (location) et sa valeur (**value**). L'exécution se poursuit au sein de CouchDB par l'affectation de la valeur à la propriété : `doc[field]=eval(value)`.

shipupdt.js

```
var nano=require('nano')('http://user:pass@localhost:5984');
var db=nano.use('ship');

db.updateWithHandler("battle", "in-place", 'A1', { field: "location",
value: '([\B5\','\B6\','\B7\'] )' }, function(err,val) {
    if (err) throw err;
    console.log('Updated. ');
});
```

Northwind.js (extra)

Le mécanisme **Map/Reduce** permet de réaliser toutes les opérations usuelles de SQL. Le portage d'une requête d'un environnement à l'autre nécessite toutefois, une certaine gymnastique intellectuelle. A titre d'exemple les requêtes SQL ci-après sont ré-exprimées (à partir de la base northwind déjà présentée et portée vers CouchDB).

Exemple n°1

```
create view "Customer and Suppliers by City" AS
SELECT City, CompanyName, ContactName, 'Customers' AS Relationship
FROM Customers
UNION SELECT City, CompanyName, ContactName, 'Suppliers'
FROM Suppliers
--ORDER BY City, CompanyName
GO
```

Map:

```
function(doc) {
  if (!!~['Customer','Supplier'].indexOf(doc.type))
    emit({City:doc.City,CompanyName:doc.CompanyName},
        {ContactName:doc.ContactName,type:doc.type});
}
```

Exemple n°2

```
create view "Alphabetical list of products" AS
SELECT Products.*, Categories.CategoryName
FROM Categories INNER JOIN Products ON Categories.CategoryID =
Products.CategoryID
WHERE (((Products.Discontinued)=0))
GO
```

Map:

```
function(doc) {
  var idx= ['Category','Product'].indexOf(doc.type);
```

```
if (idx>-1)
  emit([doc.CategoryID,doc.ProductID],idx==0 ?
    {ProductID:null,Name:doc.CategoryName}:
    {ProductID:doc.ProductID,Name:doc.ProductName});
}
Reduce:
function(keys,values,rereduce)
{
  var CategoryName='';
  var list=[];
  for (i in values) {
    if (keys[i][0][1]==null) {
      CategoryName=values[i].Name;
    }
  }
  for (i in values) {
    if (keys[i][0][1]!=null)
  {
    list.push([CategoryName,keys[i][0][1],values[i].Name]);
  }
  }
  return list;
}
```

A suivre...

➔Réplication et local repository (npm)...



Compilation de notes et d'articles de « jihem », _rev : 2.

```
--[[
o|'-|o this file is licensed under cc4-by-sa international license.
( 1 ) to view this license, visit https://creativecommons.org/licenses/by-sa/4.0/
=      author: jean-marc "jihem" quere 2011 - jean-marc.quere@codyssea.com
--]]
```